Acoustic Direction Finding System



Presented by:
Morena Mofokeng

Prepared for:
Associate Prof. A. J. Wilkinson
Department of Electrical Engineering
University of Cape Town

Submitted to the Department of Electrical Engineering at the University of Cape Town in partial fulfilment of the academic requirements for a Bachelor of Science degree in Mechatronics Engineering

October 28, 2026

Declaration

Plagiarism Declaration

1. I know that plagiarism is wrong. Plagiarism is to use another's work and pretend that it is one's own.
2. I have used the ..IEEE.. convention for citation and referencing. Each contribution to, and quotation in, this report from the work(s) of other people has been attributed, and has been cited and referenced. Any section taken from an internet source has been referenced to that source.
3. This report is my own work, and is in my own words (except where I have attributed it to others).
4. **I have not paid a third party to complete my work on my behalf. My use of artificial intelligence software has been limited to ...editing.... (specify precisely how you used AI to assist with this assignment).**
5. I have not allowed and will not allow anyone to copy my work with the intention of passing it off as his or her own work.
6. I acknowledge that copying someone else's assignment or essay, or part of it, is wrong, and declare that this is my own work.

Signed .....Word count: **11400**

Acknowledgments

I thank my supervisor, Associate Prof. A. J. Wilkinson, for his guidance, feedback and the time he gave to this project. I also thank my family and friends for the great support they gave me throughout the year, which I really needed. I thank my bursar for believing in me and investing in my future.

Terms of Reference

These terms of reference were agreed between the author and the project supervisor, Associate Prof. A. J. Wilkinson, at the start of the EEE4022S final year project, Acoustic Direction Finding System. They set out the problem to be investigated, the scope of the work, and what is expected of the author.

The aim of the project is to design, build and test an acoustic direction finding system for signals in the audio range. The system must use a minimum of three microphones arranged in a suitable geometry, with the captured signals amplified, sampled and transferred to a laptop for processing and display. Sampling is to be implemented using a microcontroller-based acquisition system. Software must provide a real-time display of the received signals and the estimated direction of arrival of the source(s) across a range of frequencies. A simulator, written in Julia, must model the array geometry, the emitting source(s) and channel noise, so that simulated data sets can be used to develop and test the processing algorithms. All simulation and signal processing is to be done in Julia.

System performance is to be evaluated for different types of sound emitter, including continuous tones, impulsive sounds, and human speech, using both simulated and real recorded data. The project allows scope to emphasise hardware, direction finding algorithms, two- or three-dimensional estimation, or multiple sources; the chosen focus is hardware (the microphone array and microcontroller-based data capture system) and direction finding algorithms for a single sound source.

The deliverables are a working simulation and direction estimation algorithm in Julia, a working data capture system with real-time display, and an analysis of direction finding performance on simulated and real data.

Abstract

This report describes the design, and testing of a low-cost acoustic direction finding system that estimates the azimuth of a single sound source over 360 degrees. The system has three SPH0645 digital microphones on an equilateral triangle with a 17 mm side, a Teensy 4.0 microcontroller that samples the three channels together at about 44.1 kHz and sends the samples over USB, and a Julia program that estimates and displays the direction in which the source emits the sound from. Three algorithms were implemented and compared: Trig + FFT, MUSIC, and a look-up table with a cost function. A simulator was used to develop them.

The complete system was tested at known angles on a turntable at the distances 90 cm and 180 cm away from the source with a 4 kHz sine, a speech phrase and a hand clap, and a Monte Carlo analysis on the recorded speech measured the sensitivity to noise, microphone position errors, gain and delay mismatch, and frame selection.

In simulation, all three algorithms meet the accuracy limit of 15 degrees (10 degrees for the system and 5 degrees for the uncertainty of the known angle), with a mean absolute error of 0.6 to 0.8 degrees. On the real array the limit is met in only 13 of 72 test conditions. The errors are repeatable (standard deviation 0.8 degrees on average), so they are systematic, not random. Speech gives lower errors than the sine, and the best result is a mean absolute error of 14.8 degrees for the LUT + cost function on speech at 90 cm, against 45.9 degrees averaged over all real recordings.

The Monte Carlo analysis shows that the phase-based methods are more robust than MUSIC, and that a delay error of 1 microsecond between the channels shifts the estimate by 20 to 37 degrees. The study concludes that the algorithms and software are correct for an ideal array, and that the remaining work is to find and remove the systematic error of the real array, which is proposed as future work together with calibration of the array.

Contents

1 Introduction **1**

1.1 Background to the study . . . 1
1.2 Objectives of this study . . . 1
1.2.1 Problems to be investigated . . . 1
1.2.2 Purpose of the study . . . 2
1.3 Scope and Limitations . . . 2
1.4 Plan of development . . . 2

2 Literature Review **4**

2.1 Fundamentals of Sound and Acoustic Propagation . . . 4
2.1.1 Sound as a pressure wave . . . 4
2.1.2 Frequency, wavelength and the speed of sound . . . 5
2.1.3 Propagation effects . . . 6
2.1.4 Characteristics of sound sources . . . 10
2.2 Signal processing foundations . . . 11
2.2.1 Sampling theory . . . 11
2.2.2 Quantisation . . . 14
2.2.3 Discrete-time signal analysis . . . 17
2.2.4 Phase and time delay . . . 20
2.2.5 Correlation and estimation . . . 23
2.2.6 Noise models . . . 25
2.2.7 Estimator bias and variance . . . 27
2.2.8 Narrowband and wideband signals . . . 30
2.3 Sound source localisation . . . 31
2.3.1 Overview and applications . . . 31
2.3.2 Problem formulation . . . 32
2.3.3 Classes of localisation . . . 36
2.3.4 SSL method taxonomy . . . 39
2.4 Direction finding and angle of arrival . . . 41
2.4.1 Definition of angle of arrival . . . 41
2.4.2 Geometric principle: from delay to angle . . . 42
2.4.3 Ambiguities . . . 45
2.4.4 Performance measures . . . 47

2.5 Microphone array geometry . . . 49
2.5.1 Array manifold and steering vectors . . . 49
2.5.2 Common geometries . . . 50
2.5.3 Spacing and spatial aliasing . . . 51
2.5.4 Geometry-dependent effects . . . 52
2.5.5 Justification of the three-microphone case . . . 53
2.6 Direction-of-arrival algorithms . . . 54
2.6.1 Time delay estimation . . . 54
2.6.2 TDOA-to-DOA conversion . . . 57
2.6.3 Beamforming . . . 58
2.6.4 Adaptive beamforming (MVDR/Capon) . . . 59
2.6.5 Subspace methods . . . 60
2.6.6 Maximum likelihood methods . . . 61
2.6.7 Multi-source localisation . . . 62
2.6.8 Learning-based approaches . . . 62
2.6.9 Comparative analysis . . . 63
2.7 Hardware subsystems . . . 64
2.7.1 Microphones . . . 64
2.7.2 Analogue front end . . . 64
2.7.3 Acquisition options . . . 65
2.7.4 Digital audio interfaces . . . 65
2.7.5 Microcontroller platforms . . . 66
2.7.6 Data transfer to a host . . . 67
2.7.7 Existing commercial and research arrays . . . 68
2.8 Simulation of acoustic arrays . . . 68
2.9 Sources of error and performance evaluation . . . 70
2.9.1 Error sources . . . 70
2.9.2 Calibration . . . 71
2.9.3 Evaluation methodology . . . 71
2.10 Summary and implications for the design . . . 72

3 System Design **74**
3.1 Requirements and Specifications Analysis . . . 74
3.2 Design Approach and Selection Criteria . . . 76
3.3 Array Geometry and Spacing . . . 76
3.4 Microphone Selection . . . 78
3.5 Acquisition Hardware and Digital Interface . . . 78
3.6 Data Transfer to the Host . . . 79
3.7 Sampling Parameters . . . 80
3.8 Signal Conditioning . . . 81
3.9 Direction-of-Arrival Algorithm Selection . . . 81
3.10 Software Design . . . 83
3.10.1 Software architecture . . . 83

3.10.2 Firmware . . . 83
3.10.3 Host acquisition . . . 84
3.10.4 Signal processing pipeline . . . 84
3.10.5 Simulators and Monte Carlo framework . . . 86
3.10.6 Real-time display . . . 87
3.10.7 Software verification . . . 88
3.11 Final System Design . . . 89
3.11.1 Physical design . . . 89
3.11.2 Test fixture . . . 91
3.11.3 Operation of the complete system . . . 91
3.12 System Integration . . . 92
3.12.1 Interaction between design choices . . . 92
3.12.2 Latency and data budget . . . 92
3.13 Design Summary . . . 93

4 Experimental Setup and Analysis Methodology 94
4.1 Objectives and Overview of the Experiments . . . 94
4.2 Test Environment . . . 95
4.3 Equipment . . . 95
4.4 Experimental Layout and Procedure . . . 96
4.4.1 Layout . . . 96
4.4.2 Procedure . . . 96
4.5 Test Signals . . . 97
4.6 Data Processing Parameters . . . 97
4.7 Performance Metrics . . . 98
4.8 Simulation and Monte Carlo Experiments . . . 98
4.8.1 Simulator for the known-angle tests . . . 98
4.8.2 Monte Carlo set-up on the measured recordings . . . 99
4.8.3 Why only the speech recordings were used . . . 99
4.8.4 Experiment A: noise sensitivity . . . 99
4.8.5 Experiment B: microphone position errors . . . 100
4.8.6 Experiment C: channel gain and delay mismatch . . . 100
4.8.7 Experiment D: frame selection and observation length . . . 100
4.9 Calibration, Uncertainty and Validity . . . 100
4.10 Chapter Summary . . . 101

5 Results and Discussion 102
5.1 Simulation Results . . . 102
5.1.1 Known-angle accuracy . . . 102
5.2 Measured Results . . . 103
5.2.1 Accuracy and repeatability on the real array . . . 103
5.2.2 Discussion of the real-array accuracy . . . 104
5.2.3 Effect of signal type . . . 105

5.2.4 Hand clap . . . 105
5.2.5 Effect of distance . . . 105
5.2.6 Monte Carlo analysis of the measured speech recordings . . . 106
5.2.7 Noise sensitivity . . . 107
5.2.8 Microphone position errors . . . 108
5.2.9 Gain and delay mismatch . . . 108
5.2.10 Frame selection . . . 109
5.2.11 Observation length . . . 110
5.2.12 Error budget . . . 110
5.3 Simulated Against Measured Results . . . 111
5.4 Algorithm Comparison . . . 111
5.5 Real-Time Performance . . . 112
5.6 Acceptance Test Results . . . 112
5.7 Limitations . . . 113

6 Conclusions **114**
6.1 Summary of the Work . . . 114
6.2 Conclusions . . . 114
6.3 Contribution . . . 115

7 Recommendations **116**
7.1 Design Recommendations . . . 116
7.2 Operating Recommendations . . . 116
7.3 Evaluation Recommendations . . . 117

8 Future Work **118**

A Pseudocode for the Simulations **124**
A.1 Signal and Noise Simulation . . . 124
A.2 Trig + FFT Algorithm . . . 124
A.3 LUT + Cost Function Algorithm . . . 125
A.4 MUSIC Algorithm . . . 125
A.5 Noise, Mismatch and Geometry Perturbations . . . 126
A.6 Monte Carlo Driver . . . 126

B Pseudocode for the Firmware and Real-Time Program **128**
B.1 Teensy Firmware . . . 128
B.2 Host Acquisition . . . 129
B.3 Real-Time Direction Estimate . . . 129
B.4 Display . . . 130

C CAD Model of the Final System **131**
C.1 CAD Model of the Enclosure . . . 131
C.2 CAD Model of the Turntable . . . 134

D Datasheets of the Selected Components **136**
D.1 SPH0645 Microphone . . . 136
D.2 Teensy 4.0 Board and Processor . . . 136
D.3 Other Components . . . 137

E Code Repository and Use of AI **138**
E.1 Code Repository . . . 138
E.2 Use of AI in the Report . . . 138

F Ethics Clearance **140**

G Graduate Attribute Appendix **142**
G.1 GA1: Problem Solving . . . 142
G.2 GA4: Investigations, Experiments and Data Analysis . . . 143
G.3 GA5: Use of Engineering Tools . . . 143
G.4 GA6: Professional and Technical Communication . . . 144
G.5 GA8: Individual Working . . . 144
G.6 GA9: Independent Learning Ability . . . 145

List of Figures

2.1 A sound pulse and a periodic wave in a tube driven by a piston or loudspeaker: (a) the arrangement, (b) diaphragm displacement against time, (c) pressure against distance, showing the wavelength λ, and (d) pressure against time, showing the period T. 5

2.2 Wavelength against frequency in air at 25 °C across the audible range, from about 17.3 m at 20 Hz to about 17.3 mm at 20 kHz. 6

2.3 Level change against distance for spherical spreading (6 dB per doubling of distance) and cylindrical spreading (3 dB per doubling). 7

2.4 Near field and far field: strongly curved wavefronts across the array close to the source, and an effectively plane wavefront at large distance. 8

2.5 Reflection from a surface, with the reflected sound modelled as coming from an image of the source behind the surface. 8

2.6 Comb-filter magnitude response $|H(f)|$ for a reflection of relative amplitude $a = 0.5$ and a path difference of 1.7 m (delay about 4.9 ms). 9

2.7 A 30 kHz tone sampled at 48 kHz, with the 18 kHz alias overlaid. 12

2.8 Spectrum of a sampled signal: the replicas are separated for $f_s > 2f_{max}$ and overlap (aliasing) for $f_s < 2f_{max}$. 14

2.9 Anti-alias filter masks on one frequency axis at $f_s = 48\,kHz$ and $f_s = 192\,kHz$, showing the wider transition band at the higher rate. 14

2.10 A sinusoid quantised at (a) 1 bit, (b) 2 bits, (c) 4 bits and (d) 16 bits. The approximation improves as the number of levels grows. 15

2.11 Spectrum of a tone that fits the frame exactly and of one that does not, both with a rectangular window, showing spectral leakage. 18

2.12 Four common windows in the time domain, with their magnitude spectra in dB. 19

2.13 Speech waveform and spectrogram of the vowel “A” spoken by a female speaker[4]. . . . 20

2.14 A sinusoid and a delayed copy with the phase shift marked, and the unwrapped phase of a broadband signal against frequency, whose slope gives the delay. 21

2.15 Phase difference between a microphone pair against arrival angle for $d = 0.1\,m$, at a frequency below and one above the 1.73 kHz aliasing limit. 22

2.16 Autocorrelations of a tone, white noise and a short pulse, showing the periodic peaks of the tone against the single sharp peak of the others. 24

2.17 Cross-correlation of two simulated channels with a known delay at high and low SNR, with the true lag marked. 25

2.18 Bias and variance: estimates scattered around a true value for low and high bias against low and high variance. 28

2.19 Angle estimates over many Monte Carlo trials at several SNRs, showing the mean error (bias) and standard deviation, with the theoretical prediction overlaid. 30
2.20 Geometry of the array model: coordinate axes, reference microphone, unit vector $\mathbf{u}$, angles θ and ϕ, and a plane wavefront reaching the microphones at different times. . . . 33
2.21 Block diagram of the signal model: source, propagation with delays, additive noise, A/D conversion and the estimator producing $\hat{\theta}$. 34
2.22 Common array layouts: binaural pair, tri-aural triangle, linear array, circular array and non-coplanar tetrahedron, with the three-microphone array highlighted. 37
2.23 Passive localisation (the source emits and the array listens) contrasted with active localisation (the system emits and the echo returns). 39
2.24 Map of localisation methods: classical (time-delay, beamforming, Capon, subspace, maximum likelihood) and learning-based, with the time-delay and delay-and-sum branches highlighted. 39
2.25 Geometry of a microphone pair: baseline d, parallel rays arriving at the microphones, plane wavefront through microphone 1, extra path $d\cos\theta$ and angle θ measured from the axis. 43
2.26 Delay against arrival angle for the array, with the integer-sample delays marked, showing the angle steps that widen towards endfire. 45
2.27 Two mirrored source positions across the baseline of a microphone pair that give the same delay (front–back ambiguity). 47
2.28 Simulated RMSE against SNR for the array, with the numerically computed Cramér–Rao bound overlaid. 49
2.29 Wavefront deviation $D^2/(8r)$ across the aperture for a source at range r. 50
2.30 Polar beampatterns for the array at a frequency below and one above the aliasing limit. . 52
2.31 Angular accuracy σ_θ against azimuth for a pair, a three-microphone line and the equilateral triangle. 53
2.32 GCC-PHAT correlation functions for an impulse, a tone and a speech-like signal, showing sharp and repeated peaks (the speech-like signal is synthetic, amplitude-modulated band-limited noise). 56
2.33 Angle error against delay-interpolation method (integer, parabolic and upsampled). . . . 56
2.34 SRP-PHAT spatial spectrum for the triangular array. 59
2.35 Data path from the microphones to the display, with the latency terms marked. 68
2.36 Measured error at each test angle on the real array (RMS over the three algorithms and both distances, from Table 5.3) against the simulated mean absolute error of 0.6 to 0.8 degrees (Table 5.1). The simulation is a single band because it is reported for all angles together. 72

3.1 Geometry of the three-microphone array 77
3.2 Format of one serial packet 80
3.3 Software architecture 83
3.4 Firmware flow 84
3.5 Signal processing pipeline 85
3.6 Flow diagrams of the three algorithms 85

3.7 Structure of the Monte Carlo framework 86
3.8 Real-time display: (a) waiting for a trigger; (b) after a hand clap at 0 degrees, with the array layout and estimated direction, the angle readout, the three recorded waveforms and the status line. 88
3.9 Block diagram of the complete system 89
3.10 CAD model of the enclosure 90
3.11 The completed system: (a) enclosure with the three microphones, angle markings, status LED and USB cable, seen from above; (b) the same enclosure from an angle; (c) the two parts of the turntable; (d) the turntable assembled. 90
3.12 CAD model of the turntable, with the top plate lifted above the base 91

5.1 Shift of the estimate under noise, position error, gain mismatch and delay mismatch (solid: 180 cm, dashed: 90 cm; the dotted line is 10 degrees) 107
5.2 Error of the combined estimate with frame gating (a) and with the observation length (b) (solid: 180 cm, dashed: 90 cm) 110

C.1 Enclosure, front view with dimensions 131
C.2 Enclosure, back view 132
C.3 Enclosure, top view with dimensions, showing the microphone cut-outs and the angle markings 132
C.4 Enclosure, left and right views 133
C.5 Enclosure, isometric views from the front and from the back 133
C.6 Turntable, isometric view with the top plate lifted above the base 134
C.7 Turntable, front views with dimensions: base (left) and top plate (right) 134
C.8 Turntable, top views: base with the markings every 22.5 degrees (left) and top plate (right) 135
C.9 Turntable, side view with the top plate lifted above the base 135

D.1 Pinout of the Teensy 4.0, front (left) and back (right) [65] 137

List of Tables

2.1 Test emitters and their implications for direction finding . . . 11
2.2 Ideal precision limits for common bit depths . . . 16
2.3 Common windows (standard values [9]) . . . 19
2.4 Noise models and their effect on delay estimation . . . 27
2.5 Application areas of SSL . . . 31
2.6 Assumptions of the basic localisation model . . . 36
2.7 Classes of localisation and the position of this project . . . 38
2.8 Families of localisation methods . . . 40
2.9 What each microphone arrangement determines . . . 46
2.10 Performance measures and their use in this project . . . 48
2.11 Common array geometries and trade-offs . . . 51
2.12 Comparison of direction-of-arrival methods for this project's conditions (qualitative, from [37, 54] and the analysis above) (a) . . . 63
2.12 Comparison of direction-of-arrival methods for this project's conditions (qualitative, from [37, 54] and the analysis above) (b) . . . 63
2.13 Microphone types . . . 64
2.14 Acquisition options . . . 65
2.15 Microcontroller platforms (typical values; see the manufacturers' documentation) . . . 67
2.16 Representative arrays (check current specifications) . . . 68
2.17 Error sources, effect and mitigation . . . 70

3.1 User requirements of the acoustic direction finding system . . . 74
3.2 Functional requirements of the acoustic direction finding system . . . 75
3.3 Specifications for the acoustic direction finding system . . . 75
3.4 Acceptance test procedures for the acoustic direction finding system . . . 76
3.5 Array geometry options . . . 77
3.6 Microphone options . . . 78
3.7 Acquisition platform options . . . 79
3.8 Digital interface options . . . 79
3.9 Data transfer options . . . 80
3.10 Sampling parameters . . . 81
3.11 Signal conditioning steps . . . 81
3.12 Direction-of-arrival algorithm options . . . 82
3.13 Software components . . . 83

3.14 Monte Carlo experiments . . . 87
3.15 Software verification checks . . . 89
3.16 Final design parameters . . . 92
3.17 Latency budget and measured time for each stage . . . 92
3.18 Design decision for each specification . . . 93

4.1 Overview of the experiments . . . 95
4.2 Equipment used in the real-array tests . . . 96
4.3 Test signals . . . 97
4.4 Processing parameters . . . 97

5.1 Accuracy on simulated signals (6 angles, 6 runs each, errors in degrees) . . . 102
5.2 Accuracy and repeatability on the real array (errors in degrees) . . . 103
5.3 Mean wrapped error at each true angle on the real array (degrees) . . . 104
5.4 Error of the clean estimates in the Monte Carlo experiments (degrees) . . . 106
5.5 Shift of the estimate against the SNR (RMSE in degrees; * = some trials without an estimate) . . . 108
5.6 Shift of the estimate against the microphone position error (RMSE in degrees) . . . 108
5.7 Shift of the estimate against the gain mismatch and the delay mismatch (RMSE in degrees) 109
5.8 Mean absolute error against the true angle with frame gating (degrees, angles within 15 degrees in brackets) . . . 109
5.9 RMSE against the true angle for windows of different length (degrees) . . . 110
5.10 Error budget: shift of the estimate for representative levels (RMSE in degrees) . . . 111
5.11 Comparison of the three algorithms . . . 112
5.12 Results of the acceptance test procedures . . . 113

6.1 Aims, findings and evidence . . . 115

D.1 Datasheets and the sections that cite them . . . 136

E.1 Contents of the code repository . . . 138

Nomenclature

Symbol	Meaning and unit
c	Speed of sound in air (343 m/s)
f	Frequency (Hz)
f_0	Frequency of a tone (Hz)
f_s	Sampling frequency (Hz)
λ	Wavelength, $\lambda = c/f$ (m)
b	Side length of the equilateral microphone array (17 mm)
d	Spacing between two microphones (m)
M	Number of microphones
K	Number of sources
N	Number of samples in a frame (FFT length)
$x[n]$	Sampled signal at sample index n
θ	True azimuth (direction of arrival) of the source (degrees)
$\hat{\theta}$	Estimated azimuth (degrees)
ϕ	Phase of a signal (rad)
τ	Time delay between two microphones (s)
τ_{21}, τ_{31}	Delays of microphones 2 and 3 relative to microphone 1 (s)
$\mathbf{u}$	Unit vector pointing to the source
$\mathbf{R}$	Spatial covariance matrix of the microphone signals
σ^2	Noise variance

Abbreviations

ADC	Analogue-to-digital converter
ATP	Acceptance test procedure
CAD	Computer-aided design
CI	Confidence interval
CPU	Central processing unit
CRLB	Cramér–Rao lower bound
DFT	Discrete Fourier transform
DMA	Direct memory access
DOA	Direction of arrival
ESPRIT	Estimation of signal parameters via rotational invariance techniques
FFT	Fast Fourier transform
FPGA	Field-programmable gate array
FREQ	Functional requirement (identifier prefix)
GCC	Generalised cross-correlation
I2S	Inter-IC Sound (digital audio interface)
LUT	Look-up table
MAE	Mean absolute error
MC	Monte Carlo
MEMS	Micro-electromechanical system
MUSIC	Multiple signal classification
MVDR	Minimum variance distortionless response
PDM	Pulse-density modulation
PHAT	Phase transform (weighting)
RAM	Random-access memory
RMSE	Root-mean-square error
SD	Standard deviation
SNR	Signal-to-noise ratio
SRP	Steered response power
SSL	Sound source localisation
STFT	Short-time Fourier transform
TDM	Time-division multiplexing
TDOA	Time difference of arrival
UCT	University of Cape Town
UREQ	User requirement (identifier prefix)
USB	Universal serial bus
WAV	Waveform audio file format

Chapter 1

Introduction

1.1 Background to the study

Knowing the direction from which a sound arrives is useful in applications such as acoustic threat detection [40]. Sound source localisation is commonly reduced to estimating the direction of the source (azimuth and elevation) without its range, which allows cheaper and simpler systems than full three-dimensional localisation [39, 40]. The direction is found from the small differences in time at which a sound wave reaches an array of microphones. For a wave arriving from a distant source, these time differences depend only on the direction of arrival and on the spacing of the microphones.

Published direction finding methods range from simple time-delay estimators, which are favoured for low-cost real-time systems [45], to subspace methods such as MUSIC, which give better resolution but need more computation and a well-calibrated array. Most reported systems estimate azimuth only, perform worse in real rooms with noise and reverberation than in simulation, and handle several sources poorly [41]. Much of the work also uses larger arrays than the three microphones considered here, so there is less guidance on how well a very small, low-cost array performs on different kinds of sound.

This project builds and tests such a system. It uses three digital microphones on an equilateral triangle with a side of 17 mm, a Teensy 4.0 microcontroller that obtains sound samples from the microphones and sends the data over USB, and a program written in Julia that estimates and displays the azimuth of a single source. The project was set by the supervisor, Associate Prof. A. J. Wilkinson, as the EEE4022S final year project, Acoustic Direction Finding System.

1.2 Objectives of this study

1.2.1 Problems to be investigated

The study investigates the following questions.

- How can a low-cost array of three microphones, sampled by a microcontroller on a shared clock, be designed and built to capture sound for direction finding and transfer it to a laptop in real time?
- How accurately can the azimuth of a single source be estimated with three microphones, three algorithms of different complexity (Trig + FFT, MUSIC and LUT + cost function), first in simulation and then on the real array?

- Does the accuracy depend on the angle, the distance to the source or the type of sound (a continuous tone, speech and an impulsive hand clap)?
- How sensitive are the algorithms to noise, to errors in the microphone positions, to gain and delay mismatch between the channels and to the choice of frames and observation length?

1.2.2 Purpose of the study

Direction finding with few microphones is attractive because it is cheap and simple, but the accuracy that a three-microphone array reaches in a real room, and the way that this depends on the algorithm and the sound, is not well documented. The study gives a working reference system with its design, a simulator and a test method with a known angle, and it separates what an ideal array can achieve from what the real array achieves. This shows where the remaining error comes from, which is the information needed to improve a low-cost system. The accuracy limit used throughout is 15 degrees, made up of 10 degrees for the system and 5 degrees for the uncertainty of the known angle.

1.3 Scope and Limitations

The scope of the study covers the design of the three-microphone array and its data capture hardware, a Julia simulator, the implementation of the three algorithms, and their evaluation with simulated and recorded data. The system estimates the azimuth of a single source in the horizontal plane over the full 360 degrees. The real-array tests used a 4 kHz sine, a speech phrase and a hand clap, played or made at known angles on a turntable at distances 90 cm and 180 cm between the source and the system. A Monte Carlo analysis on the recorded speech tests the sensitivity of the algorithms. The study does not cover elevation or three-dimensional estimation, several simultaneous sources, moving sources, or calibration of the real array.

The limitations include an array that has a fixed size (17 mm side), so the spacing cannot be adjusted for different frequencies. All real tests were done in one laboratory of about 30 square metres that is not anechoic, so reflections were present, and the known angle has an uncertainty of about 5 degrees because of the way the source and turntable were aligned. The distances were measured with a tape measure. The hand clap was recorded once for each condition, so no repeatability is reported for it.

1.4 Plan of development

The report has eight chapters and multiple appendices.

Chapter 2 reviews the literature on sound, signal processing, sound source localisation, array geometry, direction-of-arrival algorithms, hardware and simulation, and ends with the implications for the design.

Chapter 3 gives the requirements and specifications, and the design of the array, the acquisition hardware, the transfer of data and the software, including the three algorithms.

Chapter 4 describes the experimental setup and the analysis methods: the test environment, equipment, procedure, signals, processing parameters, performance metrics and the Monte Carlo experiments.

Chapter 5 presents and discusses the simulated and measured results, compares the algorithms and checks the results against the acceptance tests.

Chapter 6 gives the conclusions.

Chapter 7 the recommendations and Chapter 8 the future work.

The appendices give the pseudocode of the simulations and of the firmware and real-time program, the CAD model, the datasheets, and the code repository and the use of AI.

The code, the detailed CAD drawings and the figures used in this report are available in the project repository at `https://github.com/MFKMOR011/Acoustic-Direction-Finding-System`.

Chapter 2

Literature Review

This chapter reviews the theory and existing work needed to design an acoustic direction-finding system. It builds from the physics of sound and its propagation (Section 2.1), through the signal-processing foundations (Section 2.2), to sound source localisation in general (Section 2.3) and the principles of direction-of-arrival estimation (Section 2.4). It then covers microphone array geometry (Section 2.5), the estimation algorithms (Section 2.6), the hardware subsystems (Section 2.7), the simulation of acoustic arrays (Section 2.8), and the sources of error and the evaluation of performance (Section 2.9). The chapter closes with a summary of what the literature implies for the design (Section 2.10). The theory presented here underpins the design decisions made in later chapters.

2.1 Fundamentals of Sound and Acoustic Propagation

2.1.1 Sound as a pressure wave

Acoustic technologies cover applications as varied as surface acoustic wave (SAW) filters in telecommunications devices, non-destructive testing (NDT) of solid components in expensive machinery and ultrasonic medical imaging, and sound waves lie at the heart of all of them [1]. A sound wave is the combination of pressure and velocity fluctuations propagating through a medium, which may be solid, liquid or gas, and the terms “acoustic wave” and “sound wave” are used interchangeably [1].

In air, sound consists of localised variations in pressure above and below the ambient atmospheric pressure, referred to as compressions and rarefactions [2]. Slower pressure changes, such as those caused by weather systems or altitude, are too gradual to be heard as sound [2]. Toole et al.[2] illustrates the mechanism with a piston at the end of an air-filled tube (Figure 2.1). When the piston moves inward, it compresses the adjacent layer of air, which passes the compression to the next layer in turn while each layer returns to its resting state [2]. The result is a longitudinal pulse, since the air moves along the direction of propagation [2]. The speed at which the disturbance travels is the speed of sound, the pressure rise is proportional to the velocity of the source, and the perceived loudness relates to the amplitude of the pressure change above ambient [2]. If the source instead moves back and forth continuously, alternating compressions and rarefactions produce a periodic wave, the simplest case being a pure tone [2]. For a plane wave travelling in the $+x$ direction, the pressure fluctuation about ambient is

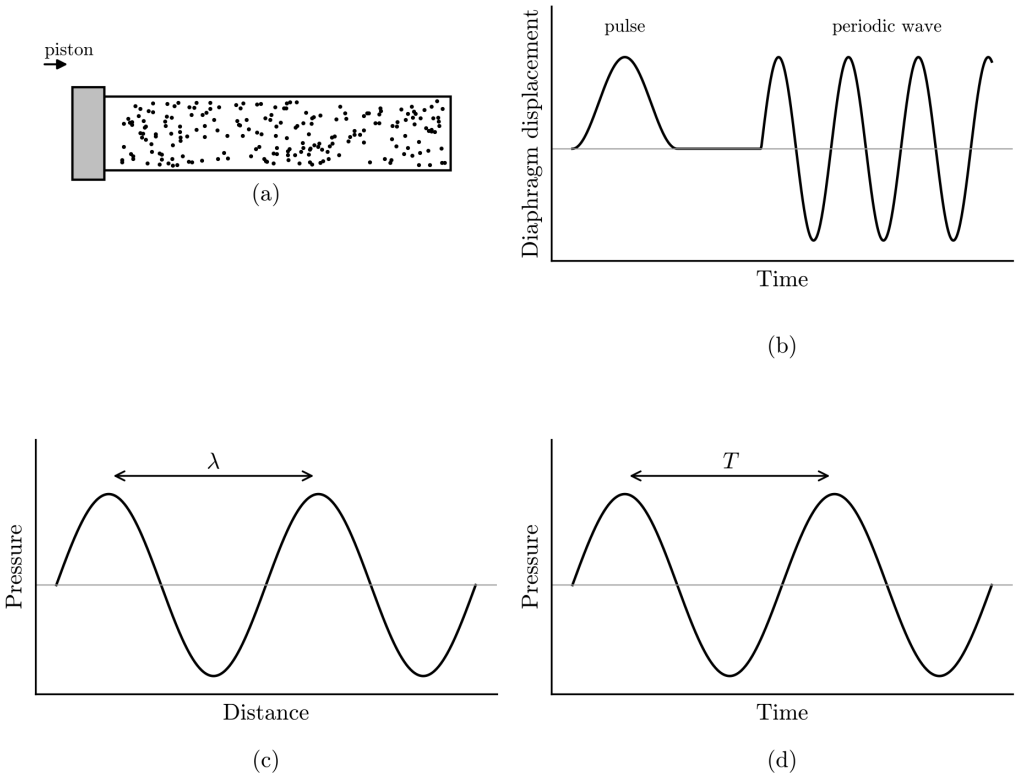


Figure 2.1: A sound pulse and a periodic wave in a tube driven by a piston or loudspeaker: (a) the arrangement, (b) diaphragm displacement against time, (c) pressure against distance, showing the wavelength λ, and (d) pressure against time, showing the period T.

$$p(x,t) = P\sin(\omega t - kx) \tag{2.1}$$

$$\omega = 2\pi f, \qquad k = \frac{2\pi}{\lambda} = \frac{\omega}{c} \tag{2.2}$$

where P is the pressure amplitude, ω the angular frequency, k the wavenumber, f the frequency, λ the wavelength and c the speed of sound.

Sound is commonly associated with human hearing, which spans roughly 20 Hz to 20 kHz, but the term is not limited to human perception. Ultrasound lies above this range and infrasound below it, and animals such as bats and whales use frequencies of up to 200 kHz to locate targets by echo [1]. This project is concerned with sound propagating in air within the audible range, where the aim is to find the direction of a source rather than to echo-locate a target.

2.1.2 Frequency, wavelength and the speed of sound

A periodic sound wave repeats over a distance called the wavelength, λ, when observed at a single instant, and over a time called the period, T, when observed at a single point [2]. The frequency is the reciprocal of the period, $f = 1/T$, and the three quantities are linked by the speed of sound, c [2]:

$$\lambda = cT = \frac{c}{f} \tag{2.3}$$

The speed of sound in air depends on temperature. Toole et al.[2] gives

$$c = 331.29 + 0.607\,t \quad \text{m/s} \tag{2.4}$$

where t is the air temperature in °C. This yields 346.47 m/s at 25 °C. Over the audible range, Equation 2.3 gives wavelengths from about 17.32 m at 20 Hz down to about 17.32 mm at 20 kHz (Figure 2.2). This wide span matters for array design, since the spacing of the microphones relative to the wavelength determines which frequencies can be processed without ambiguity (Section 2.5.3). The temperature dependence also matters because a direction-finding system converts measured delays to angles using c, so an error in c carries directly into the angle estimate. From Equation 2.4, a 10 °C error in the assumed temperature changes c by about 6 m/s, roughly 1.8 %.

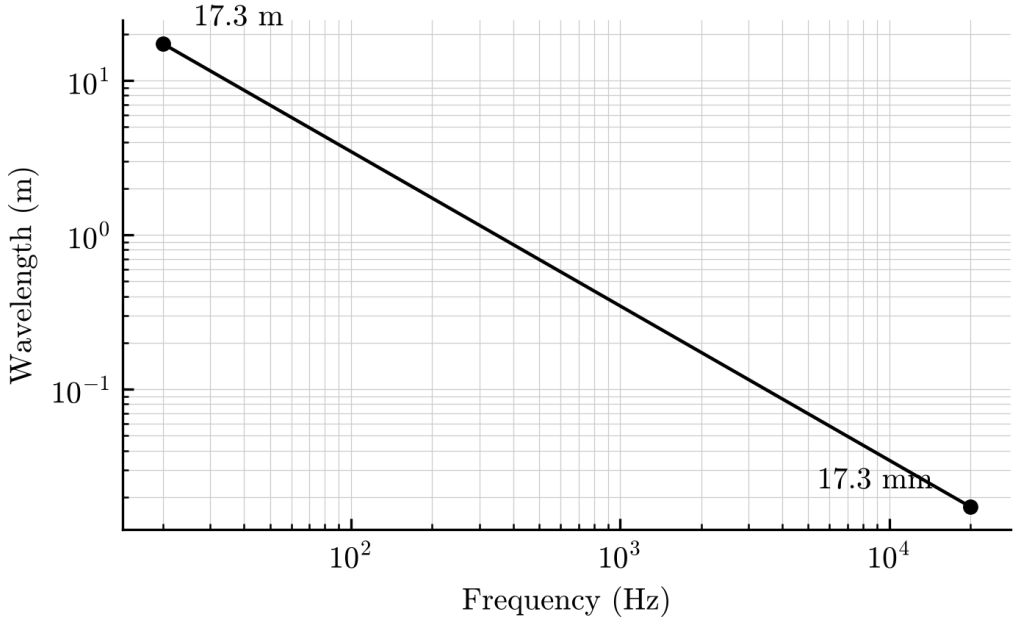


Figure 2.2: Wavelength against frequency in air at 25 °C across the audible range, from about 17.3 m at 20 Hz to about 17.3 mm at 20 kHz.

2.1.3 Propagation effects

When an acoustic wave meets the boundary between two media, part of it is reflected and part is transmitted, with the proportions set by the difference in acoustic impedance and the directions set by Snell's law [1]:

$$\frac{\sin\theta_1}{c_1} = \frac{\sin\theta_2}{c_2} \tag{2.5}$$

$$R = \frac{Z_2 - Z_1}{Z_2 + Z_1}, \qquad Z = \rho c \tag{2.6}$$

Here θ_1, θ_2 are the angles to the surface normal in each medium, R is the pressure reflection coefficient, Z the specific acoustic impedance, and ρ the density. The wavelength in the second medium also changes with the local wave velocity [1]. Because this project concerns sound in air, the effects that follow are those of a single medium with reflecting surfaces in it.

Spreading

A source that is small relative to the wavelength radiates a spherical wavefront, so its power W is spread over a sphere of area $4\pi r^2$ and the intensity falls with the square of the distance [2]:

$$I = \frac{W}{4\pi r^2}, \qquad \frac{I_1}{I_2} = \frac{r_2^2}{r_1^2} \tag{2.7}$$

The level difference between two distances is therefore

$$\Delta L = 10\log_{10}\frac{r_2^2}{r_1^2} = 20\log_{10}\frac{r_2}{r_1}\ \text{dB} \tag{2.8}$$

which gives about 6 dB for every doubling of distance [2] (Figure 2.3). A line source, such as a long row of loudspeakers, produces a cylindrical wavefront with $I \propto 1/r$, so that

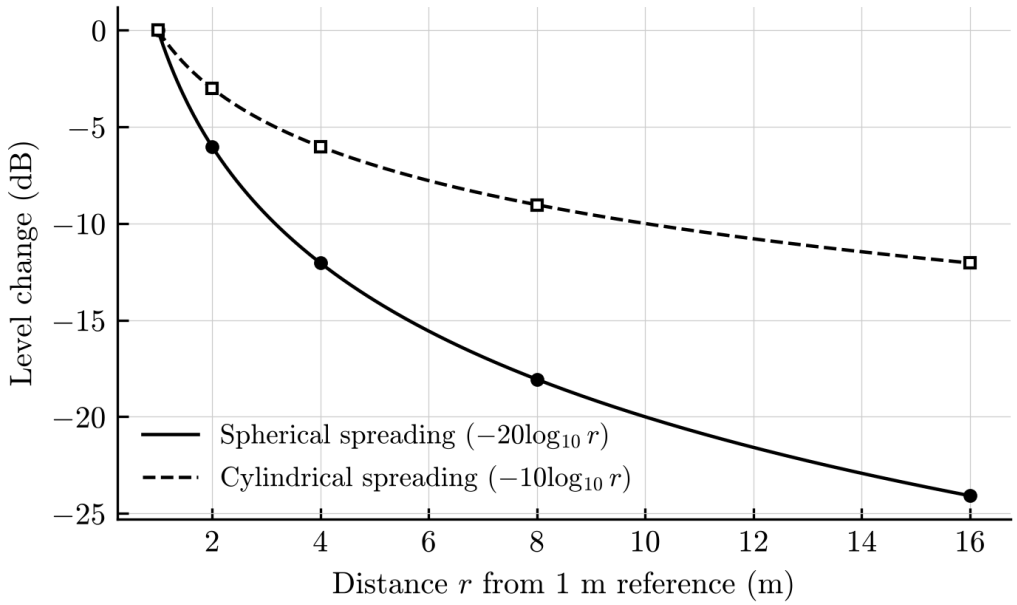


Figure 2.3: Level change against distance for spherical spreading (6 dB per doubling of distance) and cylindrical spreading (3 dB per doubling).

$$\Delta L = 10\log_{10}\frac{r_2}{r_1}\ \text{dB} \tag{2.9}$$

and the level falls by only about 3 dB per doubling [2]. Real environments depart from the ideal, since absorption by air and ground alters the prediction outdoors, and reflections sustain levels at greater distances indoors [2].

Near field and far field

Close to a source, the wavefront is strongly curved and has not yet formed an orderly pattern, so measurements there do not represent the source's total output [2]. For a sensor array, the same curvature means the arrival direction differs from one sensor to the next. Further away, the curvature across any small region becomes negligible, and the wave can be treated locally as a plane wave travelling in a single direction. This is what allows direction-finding methods to assume the same arrival direction at every sensor for a distant source. How far is "far enough" depends on the array size and the wavelength,

and is quantified in Section 2.5.1. Figure 2.4 illustrates both cases.

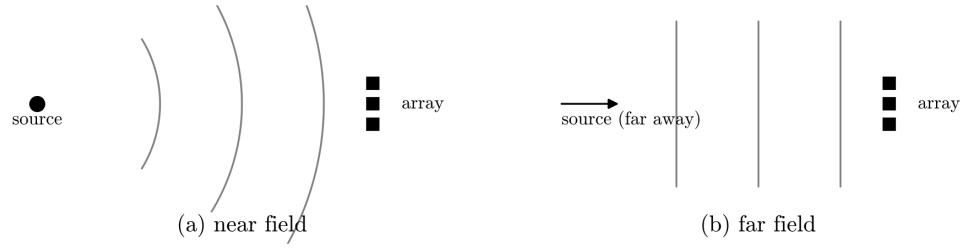


Figure 2.4: Near field and far field: strongly curved wavefronts across the array close to the source, and an effectively plane wavefront at large distance.

Reflection and absorption

Where a reflecting surface is large compared with the wavelength, the angle of reflection equals the angle of incidence, and this still holds when the surface irregularities are much smaller than the wavelength [2]:

$$\theta_i = \theta_r \tag{2.10}$$

If the irregularities are comparable to the wavelength, the sound is scattered in many directions [2]. A perfect reflection can be modelled as coming from an image of the source behind the surface, emitting the same power [2] (Figure 2.5). In practice the surface absorbs some of the energy, described by an absorption coefficient that depends on frequency and angle of incidence [2]:

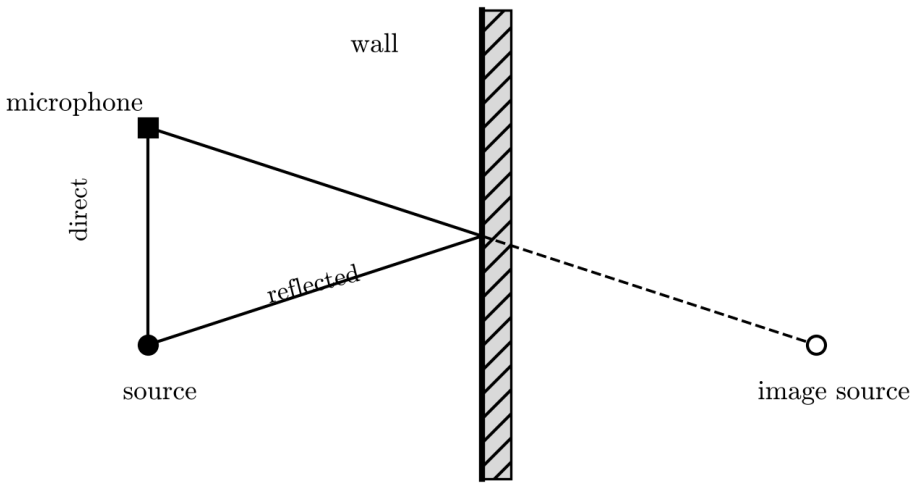


Figure 2.5: Reflection from a surface, with the reflected sound modelled as coming from an image of the source behind the surface.

$$\alpha = 1 - \frac{I_r}{I_i} \tag{2.11}$$

where I_i and I_r are the incident and reflected intensities. A value of $\alpha = 0$ is a perfect reflector and $\alpha = 1$ a perfect absorber [2].

Interference

Sound waves add linearly where they meet [2]. Two equal waves in phase reinforce to give twice the amplitude, which is 6 dB higher ($20\log_{10}2 \approx 6.02$ dB), while two waves 180° out of phase cancel [2]. At a microphone, the direct sound and its reflections interfere in this way, and whether the result is constructive or destructive depends on the extra distance travelled by the reflection relative to the wavelength [2]. For a reflection with path difference Δr and relative amplitude $a \leq 1$, the delay and the resulting magnitude response are

$$\tau = \frac{\Delta r}{c} \tag{2.12}$$

$$|H(f)| = \left|1 + a e^{-j2\pi f\tau}\right| = \sqrt{1 + a^2 + 2a\cos(2\pi f\tau)} \tag{2.13}$$

with maxima of $1+a$ at $f = k/\tau$ and minima of $1-a$ at $f = (2k+1)/(2\tau)$, for integer k (Figure 2.6). The response alternates between peaks and dips at regular intervals of $1/\tau$ across frequency, producing a comb-shaped response known as comb filtering [2]. Because the reflection is weakened by absorption and by its longer path ($a < 1$), the peaks are smaller than 6 dB and the nulls are imperfect [2]. For example, $\Delta r = 1.7$ m gives $\tau \approx 4.9$ ms at $c = 346.47$ m/s and a comb spacing of about 204 Hz. With $a = 0.5$ the peaks reach about $+3.5$ dB and the dips fall to -6 dB.

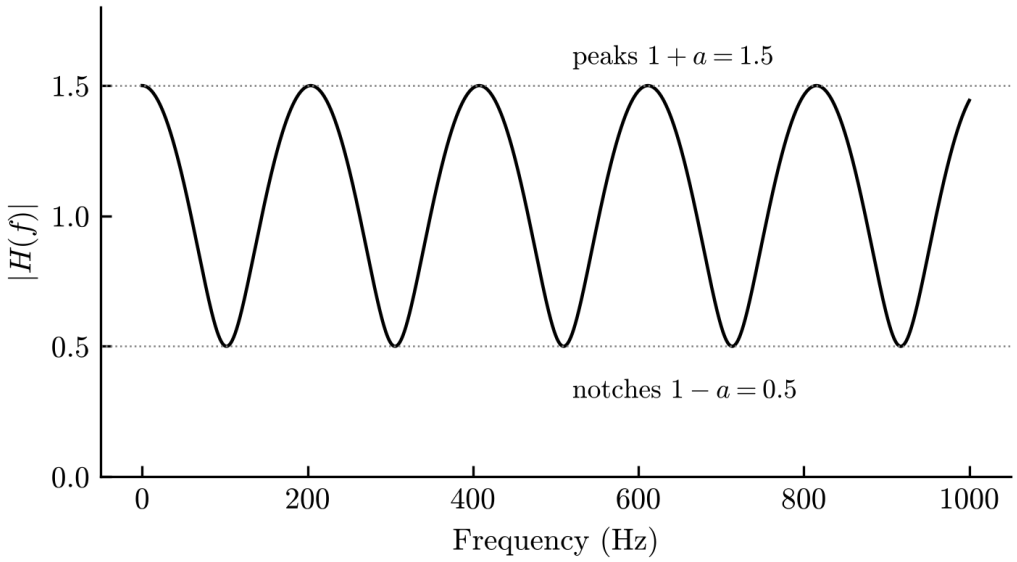


Figure 2.6: Comb-filter magnitude response $|H(f)|$ for a reflection of relative amplitude $a = 0.5$ and a path difference of 1.7 m (delay about 4.9 ms).

Diffraction and directivity

Sound bends around the edges of openings and obstacles, which is why acoustic shadows are only partial [2]. Whether an obstacle of size a shadows or is bypassed depends on the product

$$ka = \frac{2\pi a}{\lambda} \tag{2.14}$$

When $ka \ll 1$ (an obstacle small relative to the wavelength) the waves spread around it and shadowing is slight, whereas for $ka \gg 1$, at higher frequencies, the waves continue in their original direction and

shadowing is significant [2]. Sources are also directional to a degree that depends on frequency. Low-frequency sound from most sources radiates in all directions, while higher-frequency components can have strong directional biases, so no single direction represents the total output of a complex source [2].

Refraction

Sound travels faster in warmer air and with the wind, so over long distances, vertical gradients of temperature or wind bend the propagation path [2]. Equations 2.4 and 2.5 together describe the effect, since a changing c with height changes the ray angle. Downward refraction occurs downwind or in a temperature inversion, and upward refraction occurs upwind or when temperature falls with height [2].

A direction-finding array assumes a single direct path whose wavefront is effectively planar at the microphones. The far-field behaviour described above supports that assumption, and the conditions for it are set out in Section 2.5.1. Reflections break it, since each arrives from a different direction with its own delay τ (Equation 2.12 and interferes with the direct sound differently at each microphone, which biases the delay and angle estimates. This is why indoor testing is harder than free-field testing and why reverberation is treated as an error source in Section 2.9. Refraction is negligible at the short ranges used here, and source directivity matters mainly for choosing and orienting the loudspeaker in the experiments.

2.1.4 Characteristics of sound sources

The simplest sound is a pure tone, a single sinusoid at one frequency,

$$x(t) = A\sin(2\pi f_0 t + \phi) \tag{2.15}$$

with amplitude A, frequency f_0 and phase ϕ. Most real sounds are more complex, consisting of several tones of different frequencies, often with transient components added [2]. Speech, for example, is a mixture of approximately periodic vowel sounds and abrupt consonant sounds [2]. A complex periodic sound has a fundamental frequency f_0, which normally sets the perceived pitch, and higher components at whole-number multiples of it, called harmonics [2]:

$$x(t) = \sum_{n=1}^{N} A_n \sin(2\pi n f_0 t + \phi_n) \tag{2.16}$$

The harmonics are usually lower in amplitude than the fundamental [2].

Sources also differ in how their energy is spread across frequency, and this matters for direction finding. A tone is narrowband, with all its energy at f_0. An impulsive sound such as a gunshot is the opposite. By Fourier theory, a pulse of duration T_p spreads its energy over a bandwidth of roughly

$$B \approx \frac{1}{T_p} \tag{2.17}$$

so shorter pulses are more broadband, and an ideal impulse has a flat spectrum [3]. Speech is broadband but also non-stationary, since its spectrum changes continuously and can be treated as stationary only over short frames of tens of milliseconds [3]. The sound of a mosquito is a tonal wingbeat with a fundamental in the hundreds of hertz and harmonics above it, so it is narrowband-like but weak[5].

These differences are why the emitters named in the project brief form a meaningful test set (Table 2.1). They stress the direction-finding methods of Section 2.6 in different ways, and the performance on one type does not predict the performance on another.

Table 2.1: Test emitters and their implications for direction finding

Source	Spectrum	Time behaviour	Consequence for direction finding
Continuous tone	Narrowband, single line	Stationary	Delay is only recoverable from phase, which is ambiguous when the spacing exceeds half a wavelength, and the correlation peak is not unique
Impulse (gunshot)	Broadband	Very short	Sharp correlation peak, but little data to average, so timing precision and noise matter most
Speech	Broadband, harmonic plus noise-like	Non-stationary, with gaps	Needs frame-based, short-time processing, and silent frames must be handled

2.2 Signal processing foundations

This section reviews the signal-processing concepts on which the direction-finding methods depend: sampling and quantisation, spectral analysis, the relation between phase and time delay, correlation, noise and estimator performance, and the distinction between narrowband and wideband signals.

2.2.1 Sampling theory

Digital computers operate on sequences of numbers, whereas sound exists as a continuous-time signal, so audio must be sampled before it can be processed [3]. The condition under which this loses no information is given by the sampling theorem, associated with the work of Nyquist and Shannon [3].

The usual way of obtaining a discrete-time signal is periodic sampling, in which the continuous-time signal $x_c(t)$ is measured at uniform intervals to give the sequence $x[n]$ [3]:

$$x[n] = x_c(nT), \qquad -\infty < n < \infty \tag{2.18}$$

where T is the sampling period. Its reciprocal is the sampling frequency, which can be written in hertz or in radians per second [3]:

$$f_s = \frac{1}{T}, \qquad \Omega_s = \frac{2\pi}{T} = 2\pi f_s \tag{2.19}$$

In hardware, sampling is performed by an analog-to-digital (A/D) converter, which only approximates the ideal operation above. Its practical limits include quantisation of the output values, non-uniform quantisation steps, the need for a sample-and-hold stage and a maximum sampling rate [3]. Quantisation is treated in Section 2.2.2.

Effect of sampling on the spectrum

Sampling alone cannot be reversed in general, because many different continuous-time signals produce the same samples [3]. The ambiguity is removed by restricting the input signal's bandwidth. In the frequency domain, sampling makes copies of the signal's spectrum $X_c(j\Omega)$ and places them at every integer multiple of the sampling frequency [3]:

$$X_s(j\Omega) = \frac{1}{T}\sum_{k=-\infty}^{\infty} X_c\big(j(\Omega - k\Omega_s)\big) \tag{2.20}$$

If the signal is bandlimited, so that $X_c(j\Omega) = 0$ for $|\Omega| > \Omega_N$, and the copies do not overlap, the original spectrum can be recovered with an ideal low-pass filter [3]. This gives the **Nyquist sampling theorem**: the signal is uniquely determined by its samples if

$$\Omega_s > 2\Omega_N \quad \Longleftrightarrow \quad f_s > 2f_{max} \tag{2.21}$$

where Ω_N is the highest frequency present [3]. In the source's terminology, Ω_N is the Nyquist frequency and $2\Omega_N$ is the Nyquist rate [3]. In practice, the term "Nyquist frequency" is also widely used for $f_s/2$, so this report writes $f_s/2$ explicitly where that meaning is intended. For the audible range, $f_{max} = 20$ kHz requires $f_s > 40$ kHz, which is why 44.1 kHz and 48 kHz are standard audio rates.

Aliasing

If $\Omega_s \leq 2\Omega_N$, the copies overlap and the original spectrum can no longer be recovered by filtering. This distortion is called aliasing [3]. For a cosine of frequency $\Omega_0 > \Omega_s/2$, the reconstructed signal is a lower-frequency cosine $\cos(\Omega_s - \Omega_0)t$, so the higher frequency takes on the identity of a lower one [3]. In hertz, a tone at f_0 appears at

$$f_a = \big|f_0 - k f_s\big|, \qquad k = \operatorname{round}\left(\frac{f_0}{f_s}\right) \tag{2.22}$$

For example, a 30 kHz tone sampled at $f_s = 48$ kHz appears at 18 kHz, inside the audible band (Figure 2.7). Once aliasing has occurred, the alias cannot be distinguished from a genuine component at that frequency, so it must be prevented before sampling.

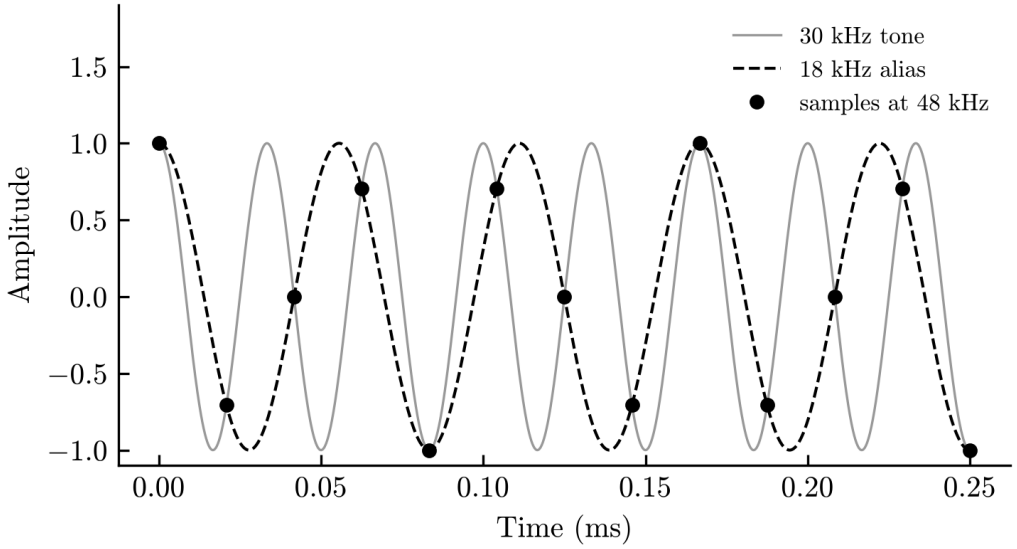


Figure 2.7: A 30 kHz tone sampled at 48 kHz, with the 18 kHz alias overlaid.

Anti-aliasing filtering

The prevention is an analog low-pass filter placed before the A/D converter, which removes content above the band of interest [3]. An ideal filter would have a cutoff Ω_c between Ω_N and $\Omega_s - \Omega_N$, but real filters have a finite transition band. To keep aliased components out of the band of interest, the passband must extend to f_{max} and the stopband must begin by $f_s - f_{max}$. For $f_{max} = 20$ kHz and $f_s = 48$ kHz, the transition band is therefore only 20 to 28 kHz. This calls for a steep, high-order analog filter, and the stopband attenuation must be large enough to push aliased energy below the converter's quantisation noise, which means roughly 6 dB per bit of converter resolution (Section 2.2.2). Such filters are costly and can distort phase, which matters here because direction finding depends on inter-channel timing.

Oversampling

The demands on the analog filter relax if the signal is sampled well above the Nyquist rate. With an oversampling ratio $M = f_s/(2f_{max})$, the stopband need only begin at $f_s - f_{max}$, so the transition band widens as f_s rises. For $f_s = 192$ kHz it spans 20 to 172 kHz, so a low-order analog filter suffices [3]. The sharp filtering is then done digitally, followed by decimation by M back to the working rate. Oversampling also reduces in-band quantisation noise, because the noise power is spread over a bandwidth M times wider and the digital filter keeps only a fraction $1/M$ of it. Assuming white quantisation noise, the SNR improvement is

$$\Delta\text{SNR} = 10\log_{10} M \text{ dB} \tag{2.23}$$

which is about 3 dB, or half a bit, per doubling of the sampling rate [3].

The sampling interval limits timing resolution directly. Adjacent samples are $1/f_s$ apart, so the smallest resolvable change in path difference is

$$\Delta d = \frac{c}{f_s} \tag{2.24}$$

which is about 7.2 mm at $f_s = 48$ kHz and $c = 346.47$ m/s. A microphone pair spaced d apart sees a maximum delay of d/c, so a small array yields only a few distinct integer-sample delays. This motivates the sub-sample interpolation methods in Section 2.6. The sampling clock must also be common to all channels, because any time skew between channels acts as a delay error and biases the angle estimate. Finally, the anti-aliasing and oversampling trade-off is relevant to the choice of microphone and sampling hardware in Section 2.7. Digital MEMS microphones commonly perform oversampling and decimation internally, so the filtering that the front end must supply depends on the chosen part.

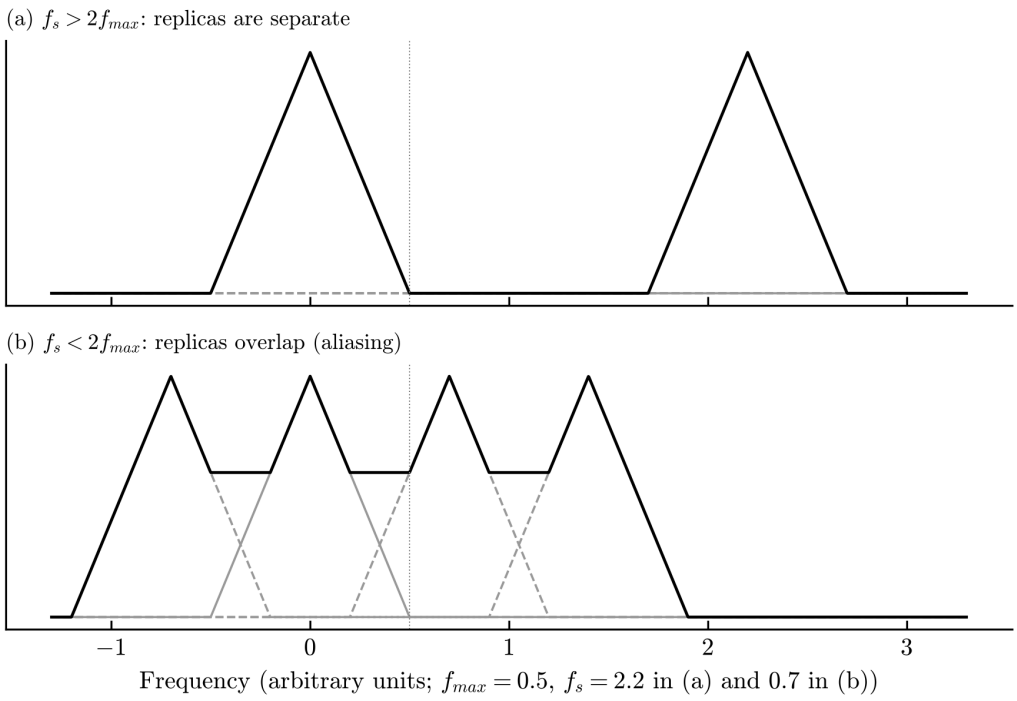


Figure 2.8: Spectrum of a sampled signal: the replicas are separated for $f_s > 2f_{max}$ and overlap (aliasing) for $f_s < 2f_{max}$.

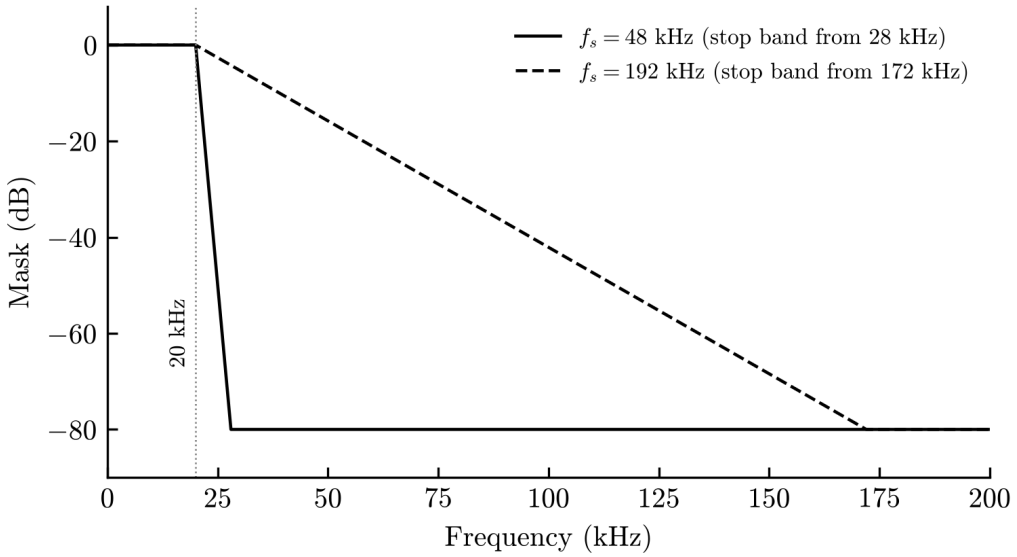


Figure 2.9: Anti-alias filter masks on one frequency axis at $f_s = 48\,kHz$ and $f_s = 192\,kHz$, showing the wider transition band at the higher rate.

2.2.2 Quantisation

A digital audio signal is described by two properties, its sampling rate and its precision, and quantisation concerns the precision with which each sample is represented [6]. Precision, or bit depth, is the number of bits used to store each sample. Where a physical signal takes continuous values, a computer must draw each sample from a finite set of 2^N levels for an N-bit representation [6]. A higher bit depth gives more levels and so a more faithful representation of small differences, at the cost of storing and transmitting more data [6]. Figure 2.10 shows a sinusoid quantised at several bit depths. The approximation is close

at 16 bits and deteriorates as the precision falls, and in the extreme 1-bit case the sinusoid becomes a square wave [6].

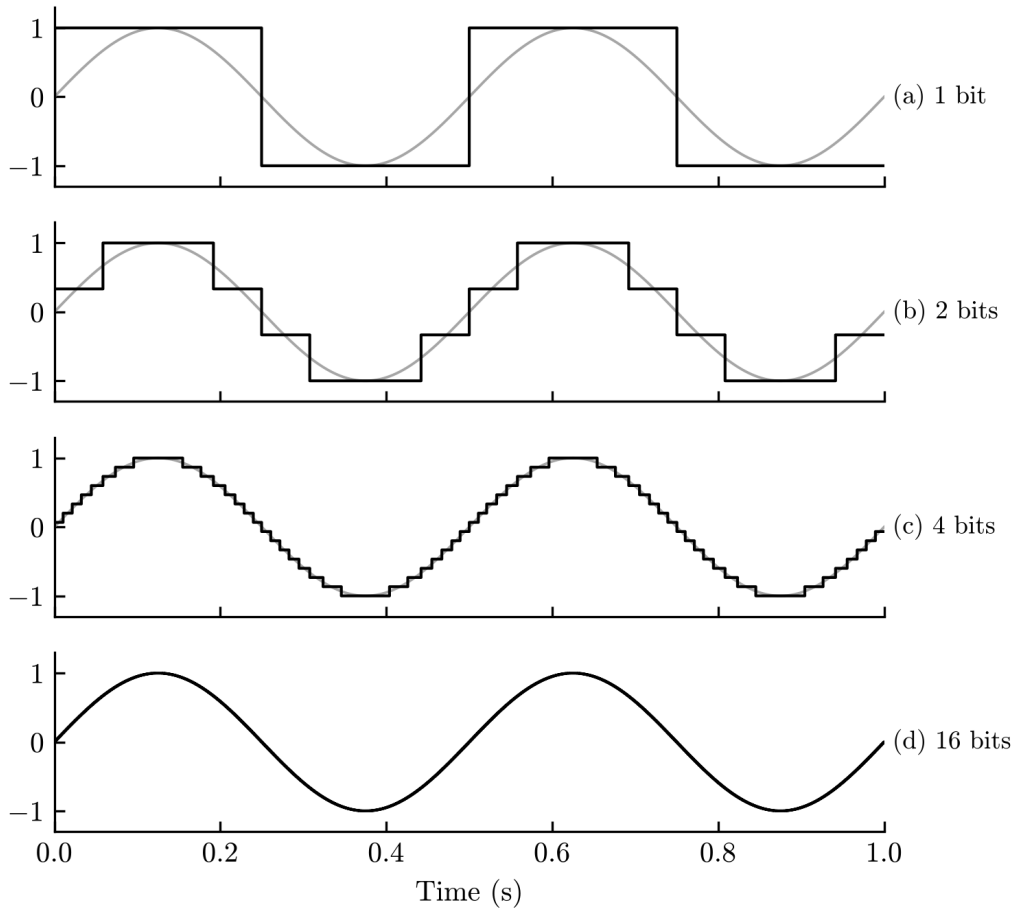


Figure 2.10: A sinusoid quantised at (a) 1 bit, (b) 2 bits, (c) 4 bits and (d) 16 bits. The approximation improves as the number of levels grows.

For a converter with full-scale range $\pm V_{FS}$ and an N-bit signed integer output, the quantisation step is

$$\Delta = \frac{2V_{FS}}{2^N} \tag{2.25}$$

and a stored integer $q \in \{-2^{N-1}, \ldots, 2^{N-1}-1\}$ represents the value $x_q = q\Delta$. The process is lossy, since most continuous input values cannot be represented exactly and are rounded to the nearest level. The difference is the quantisation error $e = x_q - x$.

Dynamic range

One measure of a quantiser is its dynamic range, the ratio of its largest to its smallest non-zero value in decibels [6]. Because signal power is proportional to the square of voltage, amplitudes are squared in the ratio [6]. The smallest non-zero magnitude is one step, Δ, and the largest is $2^{N-1}\Delta$, so

$$DR = 10\log_{10}\left(\frac{x_{max}^2}{x_{min}^2}\right) = 20\log_{10}\left(2^{N-1}\right) \approx 6.02\,(N-1)\ \text{dB} \tag{2.26}$$

This gives the well-known rule of thumb that each additional bit adds about 6 dB of dynamic

range [6]. Values for common bit depths appear in Table 2.2. Dynamic range is not the same as signal-to-noise ratio (SNR), which is a separate and more involved quantity [6].

Quantisation noise and SNR

The rounding error can be modelled as noise. If the input is large and varies enough that the error is spread evenly over one step, e is uniformly distributed over $[-\Delta/2,\ \Delta/2]$ and its power is [3]

$$\sigma_e^2 = \frac{\Delta^2}{12} \tag{2.27}$$

For a full-scale sinusoid of amplitude $A = 2^{N-1}\Delta$, the signal power is $A^2/2 = 2^{2N-3}\Delta^2$, so

$$\mathrm{SNR} = 10\log_{10}\frac{2^{2N-3}\Delta^2}{\Delta^2/12} = 6.02N + 1.76 \text{ dB} \tag{2.28}$$

This is the ideal limit, and a real converter reaches less because of analogue noise and non-linearity. Two consequences follow. First, a signal well below full scale has a proportionally worse SNR, and each factor of two in amplitude lost costs 6 dB. Second, the noise power is spread over the whole band up to $f_s/2$, which is why oversampling improves the in-band SNR as in Equation 2.23.

Table 2.2: Ideal precision limits for common bit depths

Bits N	**Dynamic range, Eq. 2.26 (dB)**	**Ideal SNR, Eq. 2.28 (dB)**	**Typical use**
8	42.14	49.9	Low-quality audio
16	90.31	98.1	Compact disc standard [6]
24	138.47	146.2	Music production and high-fidelity capture [6]
32	186.64	194.4	Integer storage

Quantisation as distortion

Quantisation introduces discontinuities, since a smooth sinusoid becomes a staircase that jumps between levels. Low-frequency waves cannot change abruptly, so severe quantisation must add high-frequency content to the signal [6]. At the 1-bit extreme this appears as the sinusoid turning into a square wave, which sounds brighter than the original while keeping the same fundamental [6].

Integer and floating-point representation

Audio is commonly stored as integers, for example in WAV files, but computers process numerical data as floating-point numbers, which use a non-uniform spacing similar to scientific notation [6]. Floating-point values are still technically quantised, but the steps are so fine that they are treated as continuous [6]. A 32-bit float spans a dynamic range of about 1529 dB, against about 187 dB for a 32-bit integer, which far exceeds the range of human hearing [6]. For this reason the usual practice is to convert the data to floating point, do the processing, and quantise again only when saving or playing back [6]. This is also how the processing in this project proceeds. Quantisation is imposed once, at the converter, and all later processing is in double-precision floating point in Julia, so the extra error added by the arithmetic is negligible.

Quantisation sets a floor on the signal that can be distinguished from noise, and this floor limits direction finding. Timing estimates degrade as SNR falls (Section 2.4.4), and a distant or quiet source produces a signal that spans only a few quantisation levels, which in the limit makes the channel noise-dominated. The resolution of the microphone's converter therefore matters alongside its acoustic noise floor, and the two should be compared when the hardware is chosen (Section 2.7).

2.2.3 Discrete-time signal analysis

The discrete Fourier transform (DFT) determines the frequency content of a finite sequence of samples, and it is one of the two most widely used procedures in digital signal processing, alongside digital filtering [7]. It is the discrete counterpart of the continuous Fourier transform, which maps a time-domain signal $x(t)$ to a frequency-domain function $X(f)$ [7]. For a sequence $x[n]$ of N samples, the DFT is

$$X[k] = \sum_{n=0}^{N-1} x[n]\, e^{-j2\pi kn/N}, \qquad k = 0, 1, \ldots, N-1 \tag{2.29}$$

Bin k corresponds to the frequency

$$f_k = \frac{k f_s}{N}, \qquad \Delta f = \frac{f_s}{N} \tag{2.30}$$

so the bin spacing Δf is set by the sampling rate and the number of samples analysed. For real signals the spectrum is symmetric, so only bins up to $f_s/2$ carry independent information. Each output value is complex, with a magnitude and a phase angle [7]:

$$|X[k]| = \sqrt{X_{re}[k]^2 + X_{im}[k]^2}, \qquad \angle X[k] = \tan^{-1}\frac{X_{im}[k]}{X_{re}[k]} \tag{2.31}$$

The squared magnitude is the power spectrum [7]:

$$P[k] = |X[k]|^2 = X_{re}[k]^2 + X_{im}[k]^2 \tag{2.32}$$

Evaluating Equation 2.29 directly takes on the order of N^2 complex multiplications. The fast Fourier transform (FFT) computes the same result in the order of $N \log_2 N$ operations, which makes spectral analysis practical in real time [8, 3].

Windowing and leakage

The DFT works on a finite block of samples, but real signals such as speech or music are very long or effectively endless. A finite-length window $w[n]$ is therefore applied to the signal before the DFT is computed [3]:

$$x_w[n] = x[n]\, w[n], \qquad n = 0, \ldots, N-1 \tag{2.33}$$

Taking a block without any weighting amounts to using a rectangular window. The DFT treats the block as one period of a repeating signal. If a tone completes a whole number of cycles in the block, the repeated signal is continuous and all its energy falls in a single bin. For any other frequency, the edges

of the block do not match, which creates a discontinuity, and the energy spreads across many bins. This is called spectral leakage (Figure 2.11) [9].

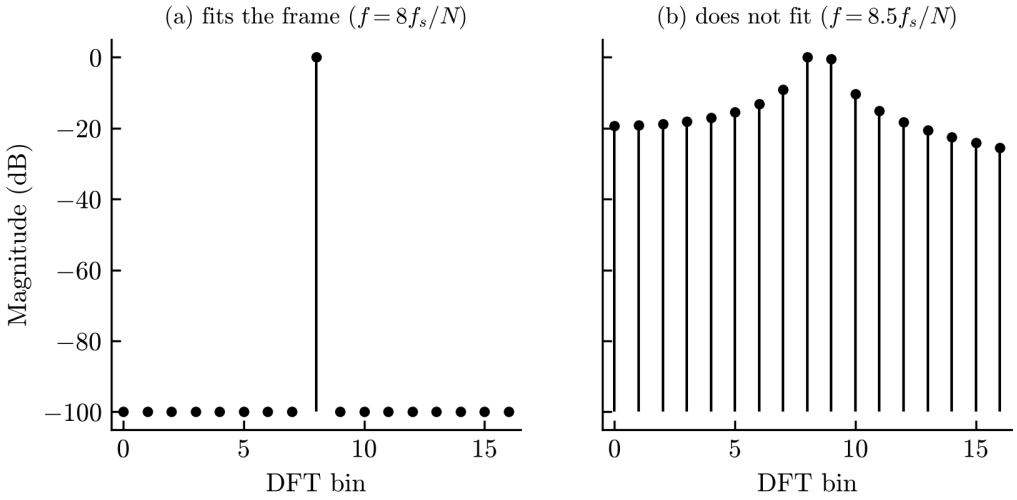


Figure 2.11: Spectrum of a tone that fits the frame exactly and of one that does not, both with a rectangular window, showing spectral leakage.

In the frequency domain, windowing convolves the true spectrum with the window's own spectrum, so leakage is determined by the shape of that spectrum: a main lobe whose width limits how close two tones can be before they merge, and side lobes that leak energy into distant bins and can mask weak components next to a strong one [9].

Tapering the window towards zero at its edges lowers the side lobes, at the cost of a wider main lobe. The rectangular window has the narrowest main lobe and the highest side lobes. Other windows trade resolution for leakage suppression, as shown in Table 2.3 and Figure 2.12.

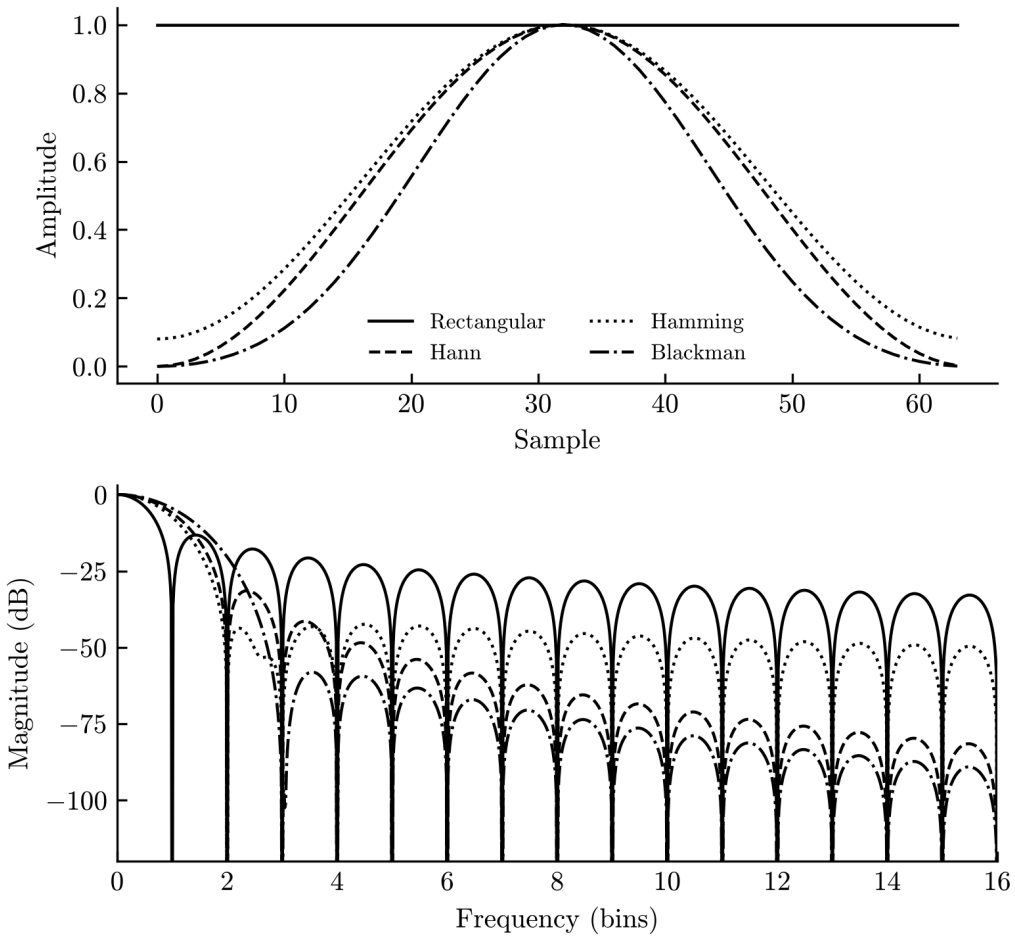


Figure 2.12: Four common windows in the time domain, with their magnitude spectra in dB.

Table 2.3: Common windows (standard values [9])

Window	**Main-lobe width (null to null, bins)**	**Peak side lobe (dB)**	**Typical use**
Rectangular	2	−13	Best resolution, or signals fitting the frame exactly
Hann	4	−31	General-purpose spectral analysis
Hamming	4	−41 to −43	Speech processing
Blackman	6	−58	Strong components next to weak ones

Short-time Fourier transform

A single DFT describes the whole block and says nothing about when each component occurred. For signals whose spectrum changes in time, such as speech, the signal is divided into overlapping frames, each is windowed, and a DFT is taken of each one. This is the short-time Fourier transform (STFT) [3]:

$$X[m,k] = \sum_{n=0}^{N-1} x[n+mH]\, w[n]\, e^{-j2\pi kn/N} \tag{2.34}$$

where m is the frame index and H is the hop size, the number of samples between frame starts. The squared magnitude $|X[m,k]|^2$ displayed against time and frequency is the spectrogram (Figure 2.13).

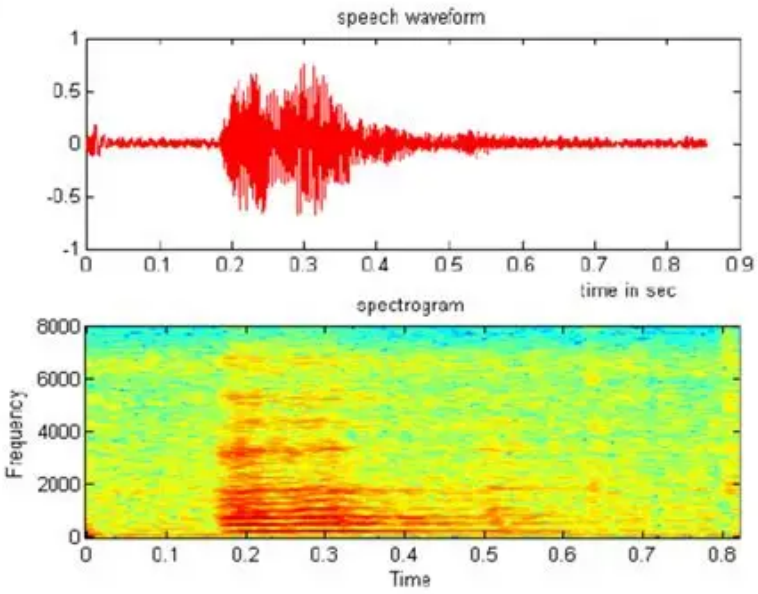


Figure 2.13: Speech waveform and spectrogram of the vowel "A" spoken by a female speaker[4].

The window length N sets a trade-off between time and frequency resolution. The frequency resolution is roughly f_s/N (widened by the window's main lobe), while the time resolution is the frame duration N/f_s. A longer window resolves close frequencies but blurs fast changes in time, and a shorter window does the reverse, so both cannot be improved together [3]. For example, $N = 1024$ at $f_s = 48$ kHz gives about 47 Hz bins and a 21 ms frame. Speech is commonly analysed in frames of a few tens of milliseconds, over which it is approximately stationary [3].

The frequency domain is also where direction information is most easily handled. Two channels with a relative delay τ have the same magnitude spectrum, but the phase of the second is shifted by $2\pi f\tau$ at each frequency (Section 2.2.4). The cross-spectrum of channels 1 and 2 captures this directly:

$$G_{12}[k] = X_1[k]X_2^*[k], \qquad \angle G_{12}[k] = 2\pi f_k \tau \tag{2.35}$$

up to a sign set by the convention used, and the delay estimate follows from the phase when $|2\pi f_k\tau| < \pi$. This is the basis of the generalised cross-correlation methods in Section 2.6. The choice of frame length therefore matters in practice. An impulsive sound such as a gunshot is captured by a short frame placed on the event, while a tone benefits from a long frame, which narrows the main lobe and improves the SNR of the bin it occupies.

2.2.4 Phase and time delay

A complex periodic waveform can be described as a sum of harmonically related sine waves, and its shape depends on both their amplitudes and their relative timing [2]. This relative timing is expressed as phase, in which one period of a sine wave is divided into 360°, so a shift of 90° corresponds to a quarter of a period [2]. The same time shift therefore produces a different phase shift at each frequency, and a delayed complex waveform has phase shifts that grow with the frequency of each component [2]. For a delay τ, the phase shift at frequency f is

$$\varphi = 2\pi f\tau \text{ rad} = 360^\circ f\tau \tag{2.36}$$

A special case is polarity reversal, where the waveform is inverted and every component shifts by 180° [2]. This is relevant to arrays, since a microphone wired or mounted with reversed polarity produces a 180° error at all frequencies.

Delay in the frequency domain

Delaying a signal multiplies its spectrum by a complex exponential. In continuous and discrete form respectively [3],

$$x(t-\tau) \;\longleftrightarrow\; X(f)\,e^{-j2\pi f\tau}, \qquad x[n-D] \;\longleftrightarrow\; X[k]\,e^{-j2\pi kD/N} \tag{2.37}$$

The magnitude spectrum is unchanged, and only the phase changes, in proportion to frequency. A pure delay therefore appears as a phase that is a straight line against frequency, and the delay can be read from the slope [3]:

$$\angle X_2(f) - \angle X_1(f) = -2\pi f\tau \quad \Rightarrow \quad \tau = -\frac{1}{2\pi}\frac{d\varphi}{df} \tag{2.38}$$

Equation 2.37 also gives a way to delay a signal by a non-integer number of samples. Choosing a fractional D in the discrete form applies a delay finer than the sample interval, which is the basis of the sub-sample methods in Section 2.6.

Phase ambiguity

A measured phase is known only modulo 2π, so the phase difference between two channels is the wrapped value

$$\varphi_{meas} = \left(2\pi f\tau + \pi\right) \bmod 2\pi - \pi \tag{2.39}$$

and a delay can be recovered uniquely from a single frequency only if $|\tau| < 1/(2f)$. For a longer delay, the phase has wrapped and several delays, separated by one period $1/f$, give the same measurement. This is the same ambiguity seen for tones in the correlation view (Section 2.2.5). A broadband signal avoids it, because the phase slope across many frequencies (Equation 2.38 fixes the delay unambiguously, even when the phase at each frequency is wrapped (Figure 2.14).

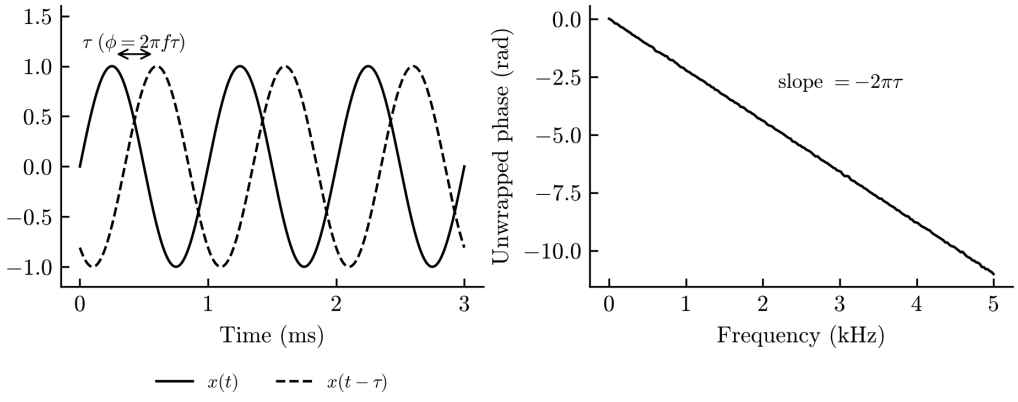


Figure 2.14: A sinusoid and a delayed copy with the phase shift marked, and the unwrapped phase of a broadband signal against frequency, whose slope gives the delay.

Application to a microphone pair

For a plane wave arriving at angle θ to the line joining two microphones spaced d apart, the extra path to the farther microphone is $d\cos\theta$, which gives a delay and phase difference (derived in Section 2.4.2) of

$$\tau = \frac{d\cos\theta}{c}, \qquad \Delta\varphi = \frac{2\pi f\, d\cos\theta}{c} \tag{2.40}$$

The largest delay a pair can see is $\tau_{max} = d/c$, which occurs at endfire. The phase difference stays unambiguous for all angles only if $|\Delta\varphi| \le \pi$ at $\theta = 0^\circ$, so

$$d \le \frac{\lambda}{2} \quad \Longleftrightarrow \quad f \le \frac{c}{2d} \tag{2.41}$$

Above this frequency, several angles give the same phase difference, which is spatial aliasing and is treated with array geometry in Section 2.5.3 (Figure 2.15). For example, $d = 0.1$ m and $c = 346.47$ m/s give $\tau_{max} \approx 289\ \mu$s, which is about 13.9 samples at 48 kHz, and an unambiguous limit of about 1.73 kHz. A tone above 1.73 kHz on this pair cannot be assigned a single angle from phase alone.

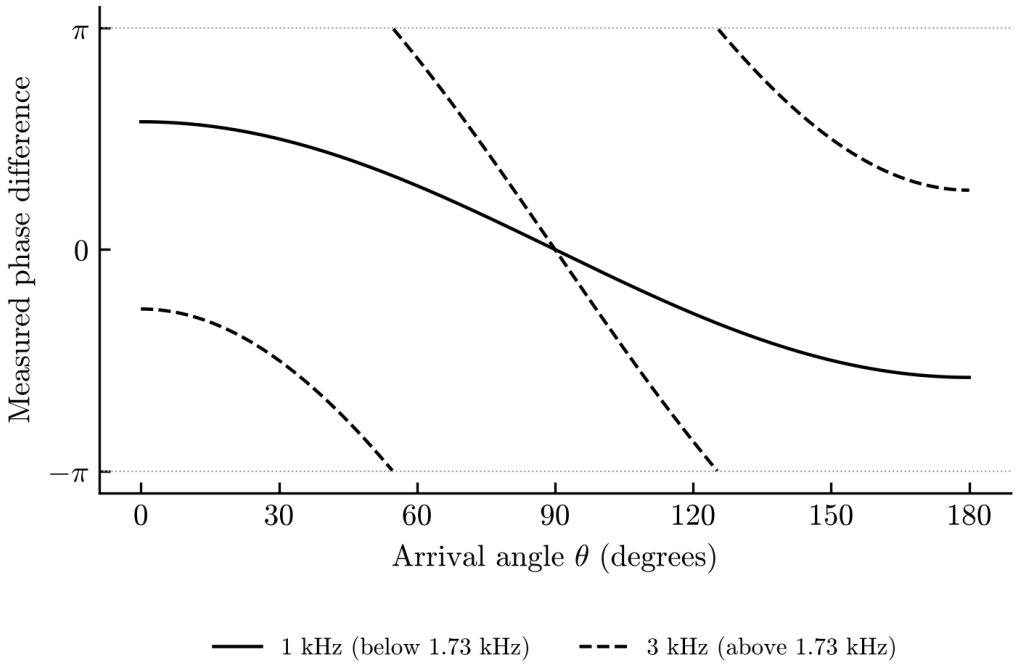


Figure 2.15: Phase difference between a microphone pair against arrival angle for $d = 0.1\,m$, at a frequency below and one above the 1.73 kHz aliasing limit.

Phase error and angle error

Any error in the measured phase becomes a delay error $\sigma_\tau = \sigma_\varphi/(2\pi f)$, and through Equation 2.56 an angle error

$$\sigma_\theta \approx \frac{c\,\sigma_\varphi}{2\pi f\, d\,|\sin\theta|} \tag{2.42}$$

so a given phase error matters less at high frequency and large spacing, which is a second reason to prefer larger spacing, alongside the timing-error relation of Section 2.2.7. A fixed phase mismatch

between two microphones or their amplifier channels, however, acts as a constant bias. For example, a 2° mismatch at 1 kHz is equivalent to a delay error of 5.6 μs and, with $d = 0.1$ m at broadside, an angle error of about 1.1°. The mismatch must therefore be calibrated or measured (Section 2.9), and it matters most in digital MEMS microphones, where it comes from the interface and clocking as well as the sensor.

Phase and the narrowband assumption

Equation 2.40 treats the delay as a single phase shift, which is exact for one frequency. For a signal with bandwidth B, the phase shift differs across the band, so a delay can be replaced by one phase factor only if the signal is narrowband, meaning $B\tau_{max} \ll 1$. This distinction is developed in Section 2.2.8 and separates the narrowband methods, which work on one phase shift per frequency, from wideband methods, which must use true delays.

2.2.5 Correlation and estimation

Correlation measures how similar two signals are as one is shifted relative to the other, and it is used in communications, radar, control and medicine [11]. It resembles convolution, since one function slides over the other and the overlap is accumulated, but it involves no time reversal [11]. It divides into cross-correlation, between two different signals, and autocorrelation, between a signal and itself [11].

For discrete-time signals the cross-correlation is [11]

$$R_{xy}[n] = \sum_{k=-\infty}^{\infty} x[k]\, y[k+n] \tag{2.43}$$

where n is the lag of y relative to x [11]. When $y = x$ this becomes the autocorrelation [11]:

$$R_{xx}[n] = \sum_{k=-\infty}^{\infty} x[k]\, x[k+n] \tag{2.44}$$

Correlation is not commutative, so $R_{xy}[n] \neq R_{yx}[n]$ in general [11]. The two are related by $R_{xy}[n] = R_{yx}[-n]$, and the autocorrelation is an even function of lag, $R_{xx}[n] = R_{xx}[-n]$ [11]. Also, $|R_{xx}[n]| \leq R_{xx}[0]$, so the autocorrelation always peaks at zero lag, where it equals the signal energy [12]. Dividing by the energies gives a normalised form bounded between -1 and 1:

$$\rho_{xy}[n] = \frac{R_{xy}[n]}{\sqrt{R_{xx}[0]\, R_{yy}[0]}} \tag{2.45}$$

Time delay as a correlation peak

Suppose $y[n] = x[n-D]$, a copy delayed by D samples. Then $R_{xy}[n] = R_{xx}[n-D]$, which is the autocorrelation shifted to peak at $n = D$. The lag of the cross-correlation peak therefore reveals the delay between two channels, and this is the basis of time-delay estimation in Section 2.6. For two microphones receiving the same source,

$$x_i[n] = s[n - D_i] + w_i[n], \qquad i = 1, 2 \tag{2.46}$$

where s is the source signal and w_i is noise. If the noise is zero-mean, uncorrelated between channels and uncorrelated with the source, the expected cross-correlation is $R_{ss}[n-(D_2-D_1)]$, so the noise does not move the peak but only adds fluctuations around it [13].

Source type and peak shape

The autocorrelation of a signal shows what the cross-correlation peak will look like. A broadband signal, such as an impulse or noise, has an autocorrelation that falls quickly away from zero lag, with a peak whose width is about $1/B$ (Equation 2.17, so the delay is sharply defined. A pure tone has a periodic autocorrelation with equal peaks every period $1/f_0$, so the delay can only be found modulo that period and is ambiguous when the true delay exceeds it [14]. This is the correlation view of the tone limitation noted in Table 2.1 (Figure 2.16).

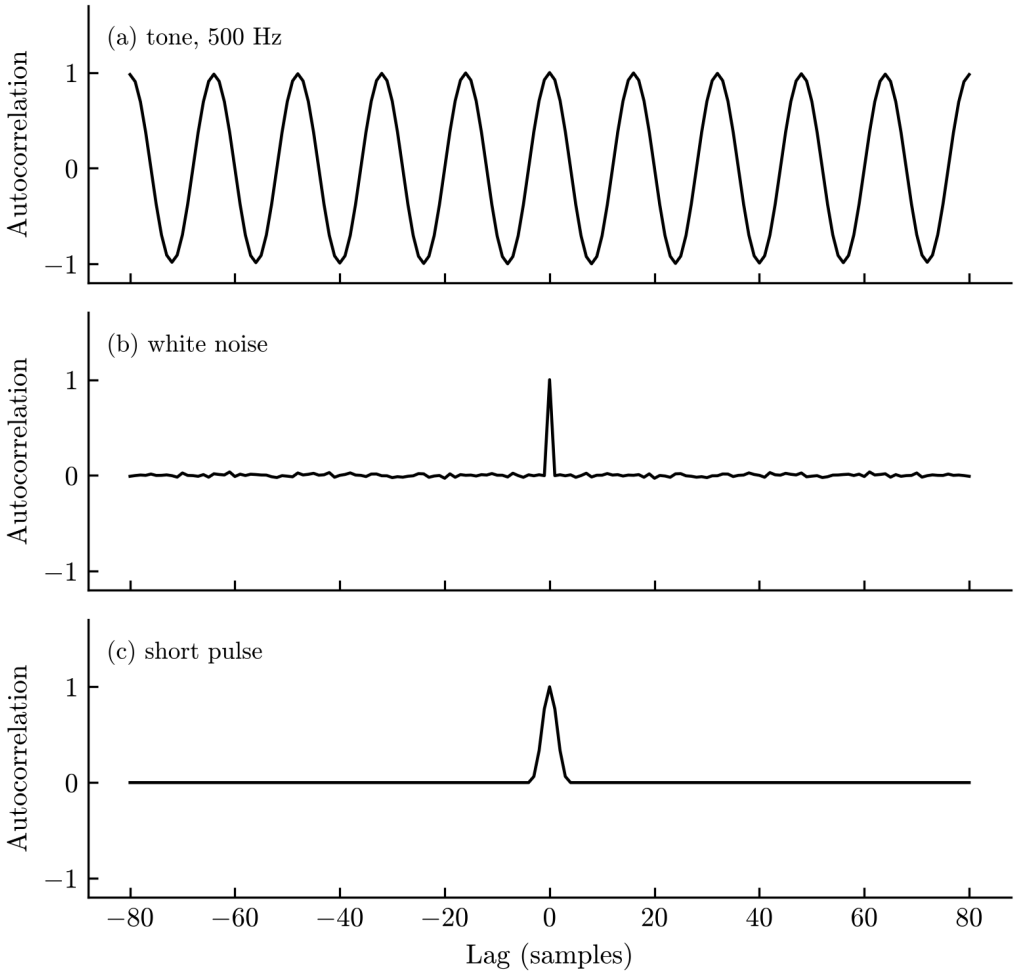


Figure 2.16: Autocorrelations of a tone, white noise and a short pulse, showing the periodic peaks of the tone against the single sharp peak of the others.

Estimation from finite data

In practice $x[n]$ is a noisy record of finite length N, so the correlation must be estimated. A common estimator is

$$\hat{R}_{xy}[n] = \frac{1}{N} \sum_{k=0}^{N-1-n} x[k]\, y[k+n], \qquad 0 \le n < N \tag{2.47}$$

with a lag of $-n$ handled by exchanging the roles of x and y. Dividing by N gives a biased estimator that shrinks towards zero at large lags, whereas dividing by $N-n$ is unbiased but has a larger variance at large lags [12]. Any estimator is judged by its bias, its variance and the combined root-mean-square error. The variance of a correlation estimate falls as the record length grows and as the SNR rises, and this is why a short impulsive event with little data, or a weak source, gives a less reliable delay estimate (Figure 2.17). The same quantities, bias, variance and the Cramér-Rao lower bound, are used to judge direction-finding accuracy in Section 2.4.4.

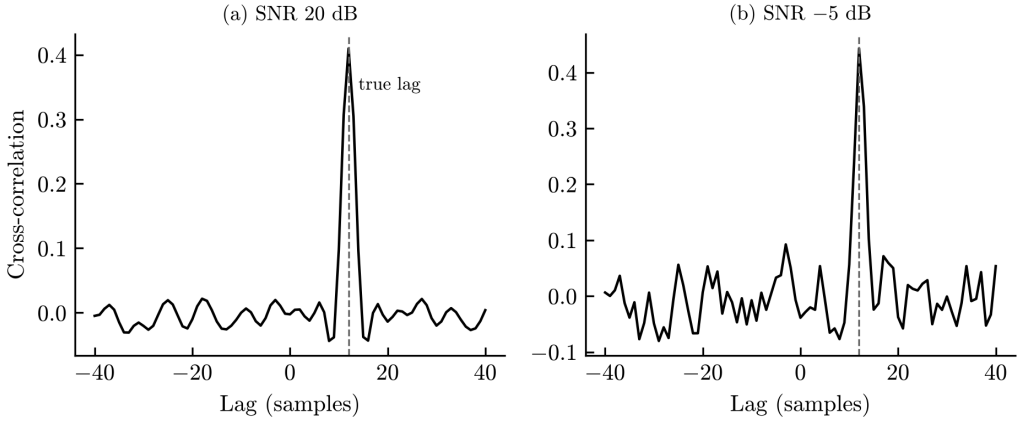


Figure 2.17: Cross-correlation of two simulated channels with a known delay at high and low SNR, with the true lag marked.

Link to the frequency domain

By the correlation theorem, the DFT of the cross-correlation is the cross-spectrum of the two signals, so correlation can be computed with the FFT, and its phase carries the same delay information as Equation 2.35 [3]. This equivalence is what lets the generalised cross-correlation methods in Section 2.6 reweight the spectrum before returning to the lag domain.

2.2.6 Noise models

Every microphone channel contains the wanted signal plus unwanted components, and the form of those components determines how well a delay or direction can be estimated. The usual model is additive:

$$x_i[n] = s_i[n] + w_i[n] \tag{2.48}$$

where w_i is the noise on channel i. The signal-to-noise ratio is the ratio of signal power to noise power:

$$\mathrm{SNR} = 10 \log_{10} \frac{P_s}{P_w} \ \mathrm{dB} \tag{2.49}$$

White Gaussian noise

The standard model is zero-mean noise with independent samples and variance σ_w^2. Its autocorrelation is an impulse at zero lag, and its power spectral density is flat across all frequencies [3, 15]:

$$R_{ww}[n] = \sigma_w^2 \delta[n], \qquad S_{ww}(f) = \sigma_w^2 \tag{2.50}$$

This model suits the noise inside the acquisition chain. Thermal and electronic noise from the microphone and preamplifier are close to white over the audio band [16, 17, 18, 19], and quantisation noise is approximately white with power $\Delta^2/12$ (Equation 2.27 [3, 20]. The self-noise of a MEMS microphone is specified through its signal-to-noise ratio [21]. The Gaussian assumption is what makes the estimator bounds in Section 2.2.7 tractable.

Coloured noise

Many real noise sources have unequal power across frequency. Ambient acoustic noise, such as ventilation, traffic and wind, is typically concentrated at low frequencies [22, 23, 24, 25], and electronic noise often has a $1/f$ component at low frequency [16, 26, 27]. A coloured noise has a non-flat spectrum and a correlation that extends over several lags, which broadens and distorts a cross-correlation peak [13, 28, 29]. This is the motivation for the spectral weighting in generalised cross-correlation (Section 2.6).

Spatial correlation of noise

For direction finding, the relationship between the noise at different microphones matters as much as its spectrum. Sensor noise is independent between channels, so $E[w_1[n]w_2[n+m]] = 0$ for all m, and, as shown in Section 2.2.5, it adds fluctuation around a correlation peak without moving it. Ambient acoustic noise is different, because the same field reaches every microphone. In an ideal diffuse field, the coherence between two omnidirectional microphones spaced d apart is [30, 31, 32]

$$\Gamma_{12}(f) = \frac{\sin(2\pi f d/c)}{2\pi f d/c} \tag{2.51}$$

At low frequencies, where $d \ll \lambda$, this approaches 1, so the noise is nearly identical at both microphones and cannot be averaged out. It falls towards zero as d becomes comparable to the wavelength, and the noise becomes effectively uncorrelated. Directional noise, such as a fan at a fixed position, behaves like a second source with its own delay and can pull the estimate towards it.

Reverberation

Reflections (Section 2.1.3) are not additive noise in the same sense. They are delayed copies of the source, so they are correlated with the wanted signal and produce extra correlation peaks at the wrong lags [33, 34, 35]. Increasing the SNR does not remove them, which is why reverberation is treated separately as a model error in Section 2.9.

Table 2.4 summarises the models. In practice the noise of the actual hardware should be measured rather than assumed, for example from a recording of silence.

Table 2.4: Noise models and their effect on delay estimation

Noise type	**Spectrum**	**Inter-channel correlation**	**Effect on the correlation peak**
Sensor and electronic	White, with a $1/f$ rise at low frequency	None	Adds random fluctuation, peak stays in place
Quantisation	White	None	Sets the noise floor of weak signals
Ambient (diffuse)	Concentrated at low frequency	High at low frequency, falling with fd/c	Biases the peak at low frequencies, where wavelengths exceed the spacing
Directional interferer	Source dependent	Coherent, with its own delay	Creates a competing peak
Reverberation	Copy of source spectrum	Coherent with the source	Creates additional peaks at wrong lags

2.2.7 Estimator bias and variance

An estimator $\hat{\theta}$ is a rule that turns noisy data into an estimate of an unknown parameter θ, such as a delay or an angle. Because the data are random, so is the estimate, and it is judged by its mean and its spread [15]:

$$\text{bias}(\hat{\theta}) = E[\hat{\theta}] - \theta, \qquad \text{var}(\hat{\theta}) = E\left[(\hat{\theta} - E[\hat{\theta}])^2\right] \tag{2.52}$$

$$\text{MSE}(\hat{\theta}) = \text{var}(\hat{\theta}) + \text{bias}(\hat{\theta})^2, \qquad \text{RMSE} = \sqrt{\text{MSE}} \tag{2.53}$$

Bias is a systematic error that would remain with unlimited averaging, such as a wrong value of c (Section 2.1.2) or a mismatch in microphone positions. Variance is the random scatter from noise and finite data. An estimator is consistent if its error tends to zero as the amount of data grows. Figure 2.18 illustrates the four combinations of low and high bias and variance.

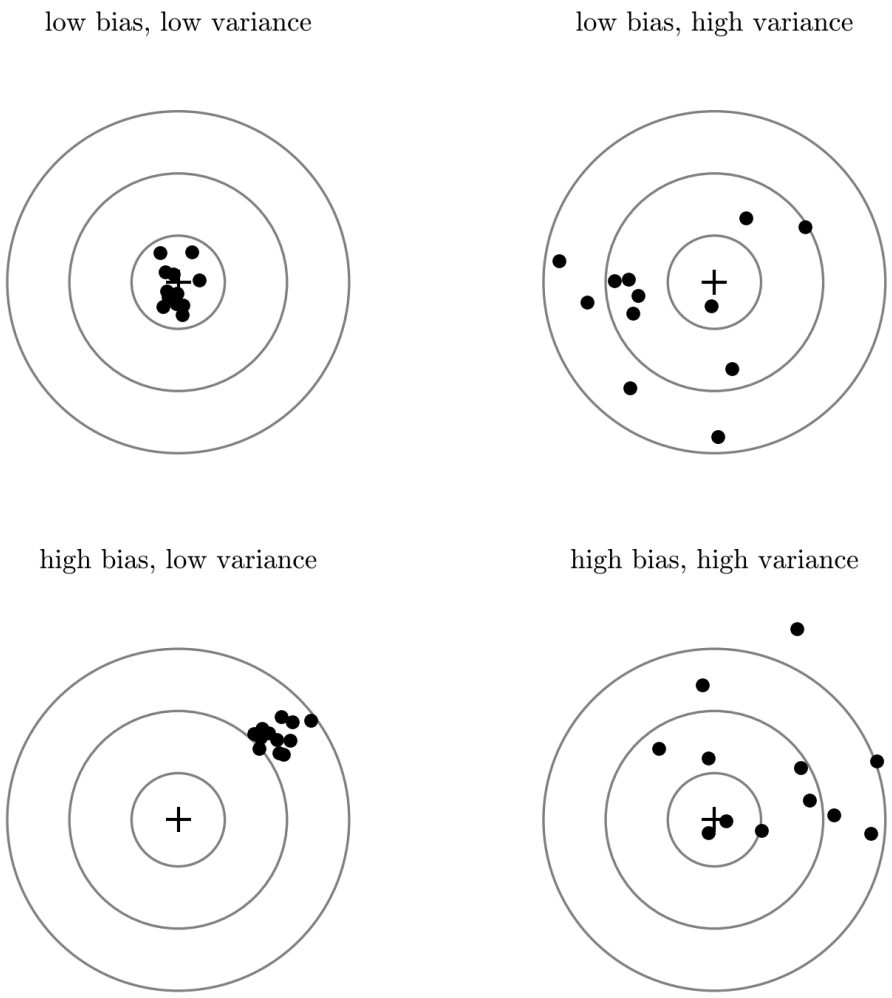


Figure 2.18: Bias and variance: estimates scattered around a true value for low and high bias against low and high variance.

Bias and variance of the correlation estimator

For the estimator in Equation 2.47, which divides by N, the expected value is [12]

$$E\left[\hat{R}_{xx}[n]\right] = \left(1 - \frac{|n|}{N}\right) R_{xx}[n] \tag{2.54}$$

so the estimate is biased towards zero at large lags, and it is unbiased only as $N \to \infty$. Dividing by $N - |n|$ removes this bias but gives larger variance at large lags, where fewer products are averaged. The variance of a correlation estimate for Gaussian signals falls roughly as $1/N$, and rises as the noise grows [12]. The variance therefore falls with more samples and a higher SNR. For delay estimation, the lags near the peak are the ones that matter, where the bias is small, so the variance is the dominant concern.

Variance of a delay estimate

The limit on what any unbiased estimator can achieve is the Cramér-Rao lower bound (CRLB), which states that $\mathrm{var}(\hat{\theta}) \geq 1/I(\theta)$, where $I(\theta)$ is the Fisher information [15]. For the delay of a broadband signal in noise, the bound depends on the coherence between the channels and on how far the signal's spectrum extends in frequency [28, 13]. The practical conclusions are that a wider bandwidth and a longer observation give a sharper delay estimate. The tone case can be written in a simple form. For a tone of frequency f_0 in white noise, observed for N samples at a per-channel SNR (as a power ratio), the delay estimated from the phase difference between two channels has variance approximately

$$\mathrm{var}(\hat{\tau}) \approx \frac{2}{(2\pi f_0)^2 N \cdot \mathrm{SNR}} \tag{2.55}$$

where the factor of 2 arises because both channels carry noise [36]. A higher frequency, more samples and a higher SNR all reduce the error.

Propagation to angle

For a pair spaced d apart, the angle follows from the delay (derived in Section 2.3.2) as $\theta = \cos^{-1}(c\tau/d)$. To first order, a delay error σ_τ becomes an angle error :

$$\sigma_\theta \approx \frac{c\,\sigma_\tau}{d\,|\sin\theta|} \tag{2.56}$$

This has two consequences. First, a larger spacing d reduces the angle error for the same timing error. Second, the error grows without limit towards $\theta = 0^\circ$ and 180° (endfire), where the delay barely changes with angle, so accuracy is best broadside.

Example

Take $d = 0.1$ m, $c = 346.47$ m/s, a 1 kHz tone, $N = 4800$ samples (0.1 s at 48 kHz) and an SNR of 10 dB (a ratio of 10). Equation 2.55 gives $\sigma_\tau \approx 1.0\ \mu$s, and at broadside Equation 2.56 gives $\sigma_\theta \approx 0.2^\circ$. These are lower-bound figures for ideal conditions, and real results will be worse because of reverberation, microphone mismatch and sampling jitter (Section 2.9).

Systematic and random error

Distinguishing bias from variance affects how results are reported. Repeating a measurement and averaging reduces variance, but not bias, so a consistent offset across repeated trials points to bias such as the wrong c, a mismatched microphone or a misaligned array, and scatter points to noise. Reporting both the mean error and the standard deviation (or RMSE) over many trials separates the two (Figure 2.19), and this is the method used in the evaluation chapter and in Section 2.9.

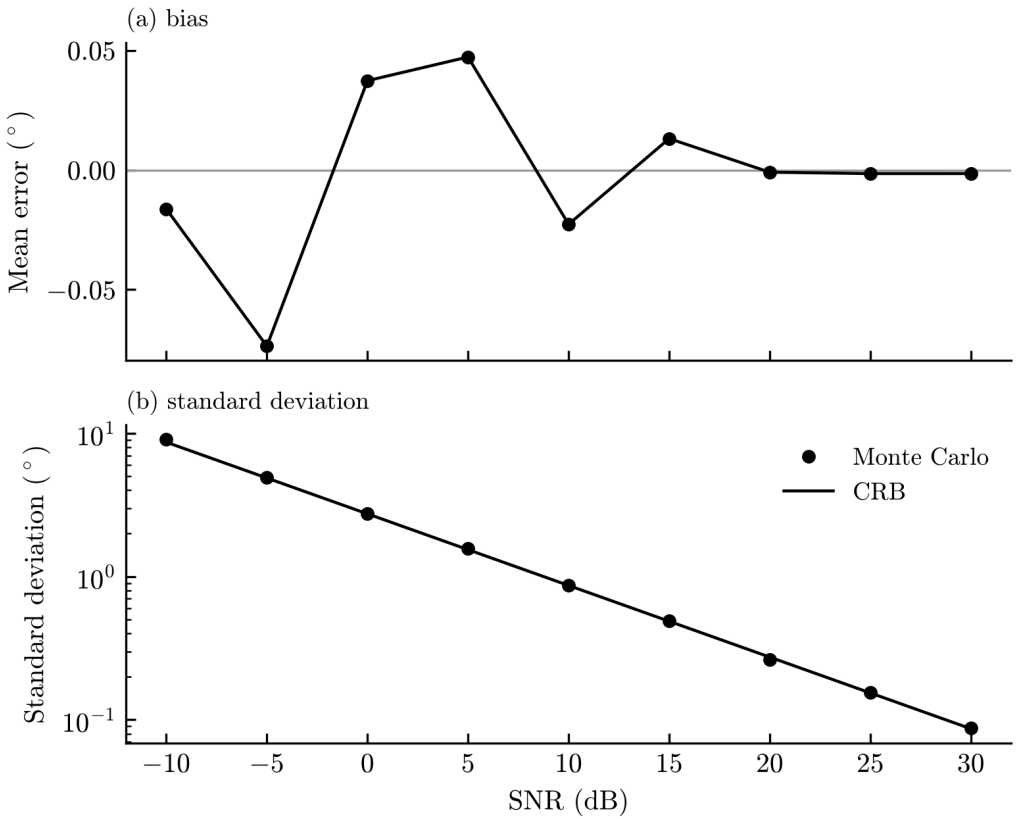


Figure 2.19: Angle estimates over many Monte Carlo trials at several SNRs, showing the mean error (bias) and standard deviation, with the theoretical prediction overlaid.

2.2.8 Narrowband and wideband signals

A signal is narrowband for an array when its delay across the array can be replaced by a single phase shift. If $\tilde{s}(t)$ is the complex envelope with bandwidth B and carrier f_0, then

$$s(t-\tau) \approx \tilde{s}(t)\, e^{-j2\pi f_0 \tau} \quad \text{when} \quad B\,\tau_{max} \ll 1 \tag{2.57}$$

[37], with $\tau_{max} = d/c$ (Equation 2.75. For $d = 0.1$ m, $\tau_{max} = 289$ μs, so the condition holds for a bandwidth well below about 3.5 kHz. A tone satisfies it, a mosquito's tonal sound nearly does, and speech and impulses do not, since a pulse of duration T_p has $B \approx 1/T_p$ (Equation 2.17. For a wideband source the delay changes the envelope as well as the phase, so the single-phase model fails and either true delays are used (GCC, SRP) or the signal is split into narrow bins.

The DFT does the second. Each bin has bandwidth f_s/N, and a frame with $N = 2048$ at 48 kHz gives 23 Hz, so $B\tau_{max} \approx 0.007 \ll 1$ and each bin is narrowband to a very good approximation (Section 2.2.3). The wideband problem then becomes a set of narrowband problems whose results are combined across bins, which is the basis of the incoherent and coherent subspace methods (Section 2.6.5). This divides the algorithms of Section 2.6 into those that use true delays, which suit broadband sources, and those that use per-frequency phase, which suit narrowband sources and need the spacing limit of Equation 2.91.

2.3 Sound source localisation

This section places the project in the field of sound source localisation: its applications, the mathematical formulation of the problem, the ways in which localisation systems are classified, and the families of methods that exist.

2.3.1 Overview and applications

Acoustic detection and acoustic localisation are related but distinct tasks. Detection is the identification of sound signals in the environment, while localisation is the determination of where the source producing them is [38]. The earliest ideas for locating sound sources came from animals such as bats and whales, which use echoes to find obstacles and prey [38].

Sound source localisation (SSL) is the technique of determining the position of one or more sources relative to a chosen reference, usually the microphone location, by analysing the signals the source produces at the microphones [39]. In many practical systems the problem is reduced to estimating the direction of arrival (DOA), meaning the azimuth and elevation angles, and the distance to the array is left out [39]. At its simplest, SSL needs two or more microphones and an algorithm that analyses their signals to find the direction of the source [39]. Knowing both direction and distance gives a full position, but many applications need only the direction, and dropping the distance allows cheaper and less complex systems [40]. Gunfire location and intrusion detection are examples of this [40].

SSL has a biological counterpart. It is one of the main functions of the human ear, which can also suppress noise and reverberation and extract information from several simultaneous sounds [41]. Attempts to build an artificial hearing system therefore aim to reproduce three functions: sound event detection, sound type classification and SSL [41]. Of these, SSL finds the direction of the source by analysing how the amplitude, frequency and phase of the sound differ between the signals received at each microphone [41]. In human hearing, these differences appear as interaural time and level differences [42].

SSL is used across a wide range of fields (Table 2.5). Sound ranging, the use of gunfire sound to find a hostile position without direct sight, was one of its first applications and remains important in force protection [40]. The same principle now serves civilian purposes.

Table 2.5: Application areas of SSL

Field	**Use**	**Source**
Bioacoustics	Locating animal calls for wildlife monitoring, biodiversity assessment, population density and tracking behaviour and migration	[39]
Wind turbines	Identifying noise sources so that mitigation can be targeted	[39]
Medicine	Locating abnormal heart or lung sounds for early diagnosis	[39]
Speech systems	Isolating a target speaker from ambient noise to improve recognition	[39]
Robotics, rescue, marine detection	Detecting and locating events or objects by sound	[38]
Defence and security	Gunfire location, intrusion detection, force protection	[40]

The task is difficult in practice. Real environments introduce reverberation and background noise, and a method must be fast enough to update the estimate quickly as conditions change [38]. Reflections from several surfaces combine with the direct sound and with sound from other places to form a field so complex that even careful analysis cannot describe it fully [38]. The factors that most control

performance are the number of microphones in the array and the signal-processing technique applied to the captured sound [40]. The sound itself also matters, because the physical properties and spectral content of impulsive noise such as gunfire differ between weapons, and the array must be designed so that these frequencies are detected [40].

This project addresses the direction-only form of the problem. A small microphone array estimates the angle of arrival of a source, the case where cost and complexity are lowest [40]. The test emitters named in the brief, tones, impulses such as hand-clap and speech, are also the sources behind several of the applications above, particularly gunfire location and bioacoustic monitoring. The formal problem definition follows in Section 2.3.2, and the angle-of-arrival principle is developed in Section 2.4.

2.3.2 Problem formulation

The localisation problem is to estimate where a sound source is from the signals measured at an array of microphones. This section states it mathematically, so that the methods in Section 2.6 can be described as different ways of solving the same problem.

Setting

An array of M omnidirectional microphones is placed at known positions $\mathbf{r}_m \in \mathbb{R}^3$, $m = 1, \ldots, M$, with microphone 1 taken as the reference. A source radiates the signal $s(t)$ from an unknown position. The direction of the source from the array is described by a unit vector $\mathbf{u}$ pointing from the array towards the source, written in terms of an azimuth angle θ in the horizontal plane and an elevation angle ϕ above it (Figure 2.20):

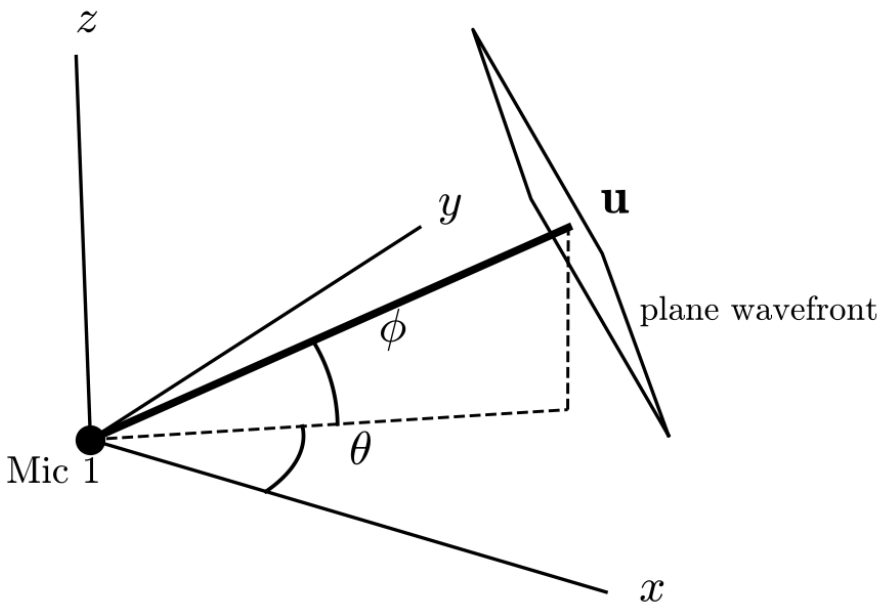


Figure 2.20: Geometry of the array model: coordinate axes, reference microphone, unit vector **u**, angles θ and ϕ, and a plane wavefront reaching the microphones at different times.

$$\mathbf{u}(\theta,\phi) = \begin{bmatrix} \cos\phi\cos\theta \\ \cos\phi\sin\theta \\ \sin\phi \end{bmatrix} \tag{2.58}$$

Propagation model

If the source is far enough away for the wavefront to be treated as a plane (Section 2.1.3), the wave reaches each microphone at a time that depends only on the microphone's position along **u**. A microphone displaced towards the source hears the wave earlier. Relative to the reference microphone, the delay at microphone m is

$$\tau_m = -\frac{\mathbf{u}^T(\mathbf{r}_m - \mathbf{r}_1)}{c}, \qquad \tau_1 = 0 \tag{2.59}$$

where c is the speed of sound (Equation 2.4. The delay of microphone j relative to microphone i is the time difference of arrival (TDOA), $\tau_{ij} = \tau_j - \tau_i$. Only $M-1$ of these are independent, since all the others follow from differences.

Signal model

Each microphone records a delayed copy of the source signal plus noise (Figure 2.21). Taking s as the signal received at the reference microphone,

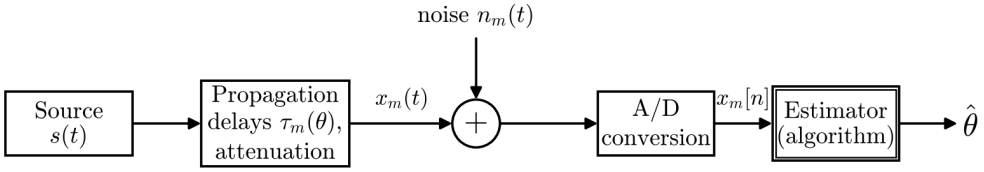


Figure 2.21: Block diagram of the signal model: source, propagation with delays, additive noise, A/D conversion and the estimator producing $\hat{\theta}$.

$$x_m(t) = s(t-\tau_m) + w_m(t), \qquad m = 1,\ldots,M \tag{2.60}$$

After sampling at f_s this becomes the discrete model already used in Section 2.2.5 (Equation 2.46, with the delay in samples $D_m = f_s\tau_m$, which is generally not an integer:

$$x_m[n] = s[n-D_m] + w_m[n] \tag{2.61}$$

By the time-shift property (Equation 2.37, delay becomes a phase factor in the frequency domain, so that for the DFT of a frame

$$X_m[k] = S[k]\, e^{-j2\pi f_k \tau_m} + W_m[k] \tag{2.62}$$

and stacking the M channels gives the vector form

$$\mathbf{x}[k] = \mathbf{a}_k(\theta,\phi)\, S[k] + \mathbf{w}[k], \qquad [\mathbf{a}_k]_m = e^{-j2\pi f_k \tau_m} \tag{2.63}$$

The vector $\mathbf{a}_k$ carries all the directional information. It is called the steering vector, and its properties are treated in Section 2.5.1. Equation 2.62 treats the delay as a circular shift of the frame, which is a good approximation when the largest delay is small compared with the frame length, $\tau_{max} \ll N/f_s$. This holds for small arrays and the frame lengths used here.

If the source is not in the far field, the wavefront curvature matters. With the source at position $\mathbf{r}_s$, the delays depend on the range as well as the direction [22]:

$$\tau_m = \frac{\|\mathbf{r}_s - \mathbf{r}_m\| - \|\mathbf{r}_s - \mathbf{r}_1\|}{c} \tag{2.64}$$

and the problem has a third unknown, the distance. Equation 2.59 is the limit of Equation 2.64 as $\|\mathbf{r}_s\| \to \infty$.

Knowns, unknowns and objective

The known quantities are the sampled signals $x_m[n]$, the microphone positions $\mathbf{r}_m$, the sampling rate f_s and the speed of sound c. The wanted quantities are the angles (θ,ϕ). The source signal $s[n]$ and the noise statistics are also unknown, but they are not of interest and are called nuisance parameters. The

task is therefore to find an estimator $(\hat{\theta}, \hat{\phi})$ that is a function of the recorded data alone, and whose error $\hat{\theta} - \theta$ is small in the sense of the bias, variance and RMSE defined in Section 2.2.7.

Localisation methods differ in how they get from the data to a direction, and Section 2.3.4 groups them by principle. Structurally, most fall into one of two families [41, 37]. Indirect methods first reduce the data to intermediate quantities, typically the TDOAs (Section 2.2.5), and then solve the geometry of Equation 2.59 for the direction. Direct methods instead define a cost function J that measures how well the data agree with a candidate direction, and search over all directions for the best:

$$(\hat{\theta}, \hat{\phi}) = \arg\max_{\theta,\phi} J\big(\theta, \phi; \mathbf{x}\big) \tag{2.65}$$

Beamforming, subspace and maximum-likelihood methods are different choices of J. Data-driven methods, which learn a mapping from features to direction, do not fit this split and are outside the scope of this project.

How many microphones are needed

Writing the $M-1$ independent delays of Equation 2.59 as one equation,

$$\tau = -\frac{1}{c}\,\mathbf{G}\,\mathbf{u}, \qquad \mathbf{G} = \begin{bmatrix} (\mathbf{r}_2 - \mathbf{r}_1)^T \\ \vdots \\ (\mathbf{r}_M - \mathbf{r}_1)^T \end{bmatrix} \tag{2.66}$$

shows that the number of independent components of $\mathbf{u}$ that the delays can reveal equals the rank of the $(M-1) \times 3$ matrix $\mathbf{G}$. This gives three cases [43, 44]:

- **Collinear microphones (rank 1).** Only the angle to the array axis is determined. All sources on a cone around the axis give the same delays, so a linear array cannot distinguish them (the cone of confusion, Section 2.4.3).
- **Coplanar, non-collinear microphones (rank 2).** The components of $\mathbf{u}$ in the array plane are determined, so the azimuth is fully determined, but the elevation is ambiguous in sign, since a source above the plane and its mirror image below give the same delays. This needs at least three microphones.
- **Non-coplanar microphones (rank 3).** The full direction is determined, which needs at least four microphones.

If the range is also unknown, as in Equation 2.64, one more independent delay is needed in each case.

Assumptions

The model above rests on a set of assumptions, summarised in Table 2.6. Real systems violate each of them to some degree, and the later sections of the review deal with how.

Table 2.6: Assumptions of the basic localisation model

Assumption	Statement	Treated in
Single point source	One source, and the source is small	Multiple sources: Section 2.6.7
Far field	Plane wavefront across the array	Section 2.1.3, 2.5.1
Free field	Direct path only, with no reflections	Section 2.1.3, 2.9
Known sound speed	c is known and constant	Section 2.1.2, 2.9
Known geometry	Microphone positions are exact	Section 2.9
Matched sensors	Identical response, with no gain or phase mismatch	Section 2.2.4, 2.7.1
Synchronous sampling	All channels share one clock	Section 2.2.1, 2.7.4
Noise	Additive, zero-mean, uncorrelated between channels	Section 2.2.6
Stationarity	Source and direction constant over a frame	Section 2.2.3

Formulation for this project

The brief and the terms of reference restrict the general problem to a single source in the far field, with $M = 3$ microphones in a plane. The elevation is then fixed at $\phi = 0$, the single unknown is the azimuth $\theta \in [0^\circ, 360^\circ)$, and the two independent delays τ_{12} and τ_{13} give a unique solution if the microphones are not collinear. The system must produce an estimate $\hat{\theta}$ for every processed frame, and its performance is judged by the error $\hat{\theta} - \theta$ against a known loudspeaker position, over the emitter types in Table 2.1.

2.3.3 Classes of localisation

Localisation systems can be classified in several ways, and each criterion corresponds to a design decision made before any algorithm is chosen [41, 38]. This section sets out the main criteria so that the position of this project among them is clear. The criteria are summarised in Table 2.7.

Number of microphones

Every method needs an acoustic sensor to capture the signal, and most use more than one microphone, although some solutions use a single microphone [38]. Systems are commonly named by their microphone count: binaural for two, tri-aural for three, tetra-aural for four and multi-aural for more [41]. Two microphones imitate human hearing, and the number can rise to 16 or beyond [41]. As Section 2.3.2 showed, the count is bounded below by the number of unknowns, since a given number of independent delays can fix only a limited number of direction components (Equation 2.66.

Dimensions of the estimate

Systems differ in which quantities they estimate. Some estimate only the azimuth in a single plane, some add the elevation, and some also estimate the distance to give a complete three-dimensional position [41, 38]. The terms 1D, 2D and 3D are used inconsistently across the literature, so this report uses the following terms:

- **Azimuth-only** (a single angle),
- **Azimuth and elevation** (a full direction), and

- **Range and bearing** (a full position).

Range estimation needs the near-field delays of Equation 2.64, which depend on distance as well as direction, and it needs at least one more independent delay. A bearing-only system omits it, which is why such systems are cheaper and simpler [40].

Number of sources

A system may be designed for a single source at a time, for two, or for many at once [41, 38]. The single-source case is the simplest, but methods that handle several sources are generally more realistic and practical [38]. Multiple sources raise a limit on resolvability. For narrowband subspace methods, the number of sources K that can be separated by M microphones satisfies [43, 44]

$$K \leq M - 1 \tag{2.67}$$

so a three-microphone array can resolve at most two sources, and this is an upper limit before noise and reverberation are considered.

Array geometry

The microphones may be arranged in many geometrical forms. Surrounding the source with the microphones is one option, and clustering them together to localise sources at a distance is another [41]. The common array types are [38]:

- **Circular arrays** place microphones around a circle and give omnidirectional azimuth coverage, but elevation is harder to determine.
- **Linear arrays** place them in a line and estimate direction accurately within the horizontal plane.
- **Hexagonal arrays** balance azimuth and elevation precision, and are used in immersive audio and robotics.
- **Ad-hoc arrays** are irregular configurations chosen to fit a particular experimental need.

Non-coplanar arrangements, such as tetrahedral and non-coplanar hexagonal configurations, are used to obtain elevation as well as azimuth [41]. Geometry also decides what can be recovered, as the rank argument of Section 2.3.2 showed, and it is developed in detail in Section 2.5 (Figure 2.22).

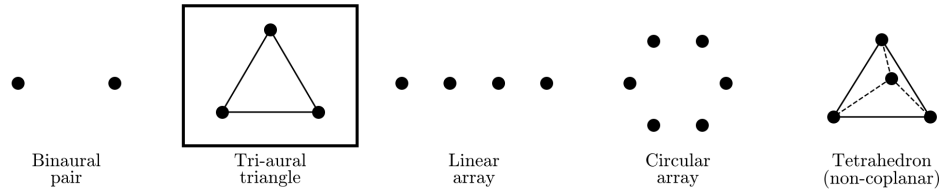


Figure 2.22: Common array layouts: binaural pair, tri-aural triangle, linear array, circular array and non-coplanar tetrahedron, with the three-microphone array highlighted.

Active and passive localisation

In passive localisation, the system listens to a sound that the source produces itself, and infers the position from it [38]. In active localisation, the system emits a sound and uses the returned echo to find an object [38]. Sonar and the echolocation of bats and whales (Section 2.3.1) are active, and in these cases the transmitted waveform is known to the receiver. A passive system must work with an unknown source signal, which is why the model of Equation 2.60 treats $s(t)$ as a nuisance parameter.

Stationary and moving sources

The direction of a stationary source is constant, so many frames can be averaged to reduce the error (Section 2.2.7). A moving source has a time-varying direction that must be tracked, and the estimate must be updated faster than the direction changes [38]. Motion also shifts the received frequency by the Doppler effect. For a source moving towards a stationary receiver at radial speed v,

$$f' = \frac{f\,c}{c - v} \tag{2.68}$$

which matters for fast sources such as drones and aircraft [38] but is small for a loudspeaker or a person at walking pace, where $v \ll c$.

Method of localisation

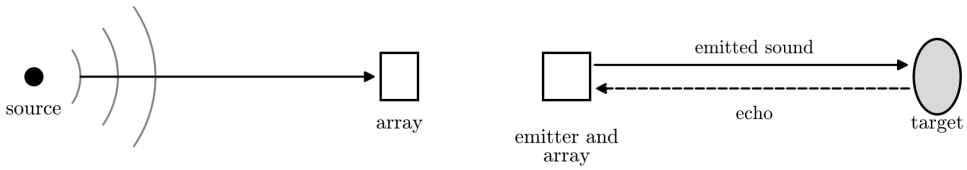
Finally, methods differ in the principle used to obtain the source position. Classical methods are built on a propagation model and treat localisation as a signal-processing problem, and in recent years methods based on artificial intelligence, such as neural networks, have become popular [38]. Among the classical algorithms, the generalised cross-correlation (GCC), time difference of arrival (TDOA), multiple signal classification (MUSIC), approximate maximum likelihood and multichannel cross-correlation methods all analyse the changes in amplitude, frequency and phase between the microphone signals [41]. Convolutional neural networks work differently, by extracting features automatically from the raw signal [41]. This criterion is the basis of the taxonomy in Section 2.3.4, and the algorithms are covered in Section 2.6.

Table 2.7: Classes of localisation and the position of this project

Criterion	**Classes**	**This project**
Number of microphones	Single, binaural (2), tri-aural (3), tetra-aural (4), multi-aural	Tri-aural, with three microphones
Dimensions	Azimuth-only, azimuth and elevation, range and bearing	Azimuth-only (bearing)
Number of sources	Single, dual, multiple	Single source
Array geometry	Linear, circular, hexagonal, tetrahedral, ad-hoc	Planar, non-collinear (Section 2.5)
Active or passive	Active (echo), passive (source emits)	Passive
Motion	Stationary, moving	Stationary source during each test
Method	Classical (model-based), learning-based	Classical, with TDOA and beamforming methods

The project is therefore a passive, bearing-only localisation system with a three-microphone planar array, designed for a single stationary source, and it uses classical model-based methods. These choices match the terms of reference. The planar array with three microphones is the smallest arrangement that gives a unique azimuth over the full circle (Section 2.3.2), and Equation 2.67 shows that multiple sources are beyond what three microphones can robustly resolve, which supports treating them as future

work. The learning-based class is not pursued, because it needs large training sets and the brief calls for a simulator and analysis built on a propagation model.



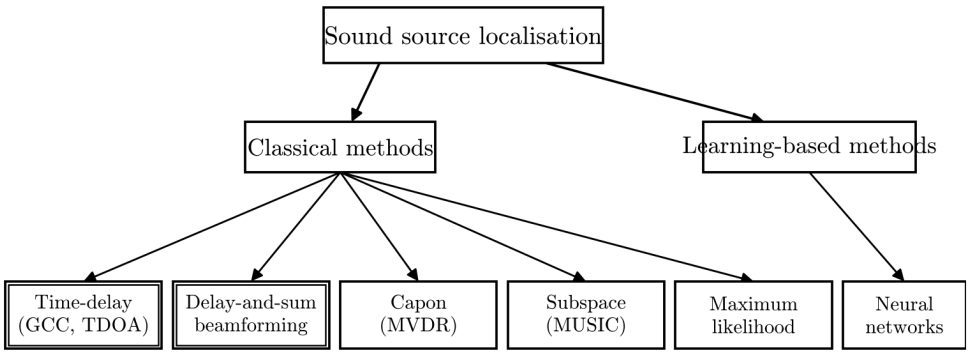
(a) passive: the source emits, the array listens

(b) active: the system emits, the echo returns

Figure 2.23: Passive localisation (the source emits and the array listens) contrasted with active localisation (the system emits and the echo returns).

2.3.4 SSL method taxonomy

Localisation methods can be grouped in several ways, and the grouping used in the literature depends on what is being compared. One review divides them into four types, according to whether the signals come from a microphone array or from a binaural pair, and whether they are processed by conventional algorithms or by convolutional neural networks (CNNs) [41]. A second separates classical methods, built on a propagation model, from the artificial-intelligence methods that have grown in popularity in recent years [38]. This review adopts the second split as its top level, because it distinguishes methods by how the direction is obtained and the array question is already covered in Section 2.3.3. Figure 2.24 shows the resulting map.



double border: methods used or compared in this project

Figure 2.24: Map of localisation methods: classical (time-delay, beamforming, Capon, subspace, maximum likelihood) and learning-based, with the time-delay and delay-and-sum branches highlighted.

Classical methods treat localisation as a signal-processing problem built on a propagation model, and analyse how amplitude, frequency and phase differ between microphones [41, 38]. They fall into four families, distinguished by the quantity they use:

- **Time-delay methods** estimate the delay between microphone pairs, usually by GCC, and convert the delays to an angle by geometry. Simplicity, low computational complexity and short execution time are why TOA and TDOA approaches are favoured for real-time direction-finding equipment [45].

- **Beamforming methods** steer the array across candidate directions and measure the output power. Conventional delay-and-sum beamforming is simple, robust to model mismatch and cheap, but its resolution is limited by the array length and it suffers from sidelobe interference [37]. Adaptive (Capon/MVDR) beamforming improves resolution by nulling interferers, but needs a matrix inversion and is sensitive to steering-vector errors and limited data [37].
- **Subspace methods**, such as MUSIC and ESPRIT, use the eigenstructure of the signal covariance matrix. They achieve super-resolution beyond what classical beamforming can, but they need the number of sources to be known, assume a calibrated array, and ESPRIT additionally needs a shift-invariant geometry [37].
- **Maximum-likelihood (ML) methods** fit a model of the data directly. They are asymptotically optimal and can approach the Cramér-Rao bound, at the cost of heavy iterative computation [37].

Sparse-reconstruction methods exist as a further modern family [37], but they are grid-based and computationally costly, and they are outside the scope of this project.

Learning-based methods use CNNs, which extract features from the raw signal automatically [41]. Their drawbacks for this project are the need for training data and the fact that most reported systems estimate only azimuth [41].

Table 2.8: Families of localisation methods

Family	**Quantity used**	**Structure**	**Advantages**	**Limitations**	**Sect.**
Time-delay (GCC, TDOA)	Inter-microphone delays	Indirect	Simple, low cost, fast [45]	Single source; degraded by reverberation [33]	2.6.1
Delay-and-sum beamforming	Steered output power	Direct	Simple, robust, low cost [37]	Poor resolution, sidelobes [37]	2.6.3
Capon (MVDR)	Inverse covariance	Direct	Better resolution, interference nulling [37]	Matrix inversion, mismatch sensitive [37]	2.6.4
Subspace (MUSIC, ESPRIT)	Covariance eigenstructure	Direct	Super-resolution, multiple sources [37]	Needs source count and calibration [37]	2.6.5
Maximum likelihood	Likelihood of the data	Direct	Asymptotically optimal [37]	Computationally intensive [37]	2.6.6
Learning-based (CNN)	Raw signal or features	Learned mapping	Little preprocessing [41]	Needs training data; mostly azimuth-only [41]	2.6.8

Reported performance

Direct comparison across the literature is hard, because studies use different metrics, sources and environments. Two general findings do recur. In Salama's simulations, subspace methods resolve closely spaced sources better than classical beamforming, ML methods approach the Cramér-Rao bound at high SNR, and classical methods are less accurate but more robust [37]. The SSL review observes that most systems estimate only azimuth, that performance falls in real environments with noise, echo and reverberation, and that multiple-source localisation lags [41]. These simulation results assume 8 to 64 sensors, narrowband signals and a calibrated array, which is far from this project's three microphones and wideband sources. The subspace advantage also shrinks here, because three microphones can separate at most two sources (Equation 2.67 and MUSIC needs a wideband extension to handle speech and impulses (Section 2.6.5).

Many existing methods rely on complex hardware, intensive processing and high computational demand, which gives high accuracy but makes them impractical for lightweight, low-cost applications [46]. This project falls in that category, so simple, low-cost methods are preferred. The three methods implemented are a time-delay method on the FFT phase (Trig + FFT), a subspace method (MUSIC) and a model-matching method (LUT + cost function). The other families are reviewed for completeness but are not implemented.

2.4 Direction finding and angle of arrival

This section narrows the field to direction finding. It defines the angle of arrival, derives how the delay between a pair of microphones maps to an angle, and sets out the ambiguities and performance measures that govern what a given array can achieve.

2.4.1 Definition of angle of arrival

Direction of arrival (DOA) estimation is the process of finding the spatial directions from which signals reach an array of sensors [37]. In the sound localisation literature, the term angle of arrival (AoA) is used for the same quantity, and this report uses the two interchangeably. When localisation is reduced to estimating direction alone, the output is the azimuth and elevation of the source, and the distance to the array is omitted [39]. A full position would add the range [41], which this project does not estimate (Section 2.3.3).

Coordinate convention

Angles are defined in a right-handed Cartesian frame fixed to the array, with its origin at the reference microphone (Figure 2.20). The *x*--*y* plane is the horizontal plane, and the *z*-axis points up. The direction of the source is the unit vector **u** of Equation 2.58, described by two angles:

- the **azimuth** $\theta \in [0^\circ, 360^\circ)$, measured in the horizontal plane from the *x*-axis towards the *y*-axis, and
- the **elevation** $\phi \in [-90^\circ, 90^\circ]$, measured upward from the horizontal plane.

For a planar array lying in the *x*--*y* plane, as in this project, the elevation is fixed at $\phi = 0$ and the azimuth is the only unknown (Section 2.3.2).

Other conventions in the literature

Sources do not use a single convention, so quoted formulas need translating. Spherical coordinates are often written with a polar angle ϑ measured from the *z*-axis, which is related to the elevation by

$$\vartheta = 90^\circ - \phi \tag{2.69}$$

Salama [37] measures the angle θ_B of a linear array from the broadside normal, whereas the delay expressions in this report measure the angle θ_A from the array axis. The two are related by

$$\theta_A = 90^\circ - \theta_B, \qquad \cos\theta_A = \sin\theta_B \tag{2.70}$$

so a delay written as $\tau = d\sin\theta_B/c$ in [37] is the same as $\tau = d\cos\theta_A/c$ in Equation 2.40. Both forms describe one physical situation, and this report uses the axis form for pairs and the azimuth--elevation form for the array as a whole.

What the angle means for a linear pair or array

A pair of microphones, or any collinear array, measures only the angle between the source direction and the array axis, because every source on a cone about the axis produces the same delay (Section 2.3.2). This is the cone of confusion, and it is why the azimuth over the full circle needs three non-collinear microphones.

Array frame and world frame

The angles above are relative to the array. To report a compass bearing, the array orientation must be known, and the estimate is rotated into the world frame by adding the array's heading. This project reports angles in the array frame, so no such rotation is needed.

2.4.2 Geometric principle: from delay to angle

A common signal that reaches two spatially separated receivers arrives at different times, and this time difference of arrival (TDOA) can be used to estimate the direction of the source [47]. Consider two perfectly synchronised receivers. Following the model in [47], with the signal at the second receiver delayed, attenuated and corrupted by independent noise,

$$r_1[k] = s[k] + n_1[k], \qquad r_2[k] = a\,s[k-\Delta] + n_2[k] \tag{2.71}$$

where a is an attenuation factor and Δ is the unknown delay between the channels [47]. This is the two-microphone case of the model in Equation 2.60, and estimating Δ is the job of the correlation methods of Section 2.2.5. The remaining step, from delay to angle, is purely geometric.

Derivation

Place two microphones a distance d apart, with microphone 1 at the origin and microphone 2 on the negative x-axis (Figure 2.25). Let θ be the angle between the positive x-axis (the direction from microphone 2 to microphone 1) and the direction to the source, as in Equation 2.70. If the source is far from the pair, the wavefront is effectively a plane (Section 2.1.3), and the wave direction is the same at both microphones [47]. The two ray paths are then parallel.

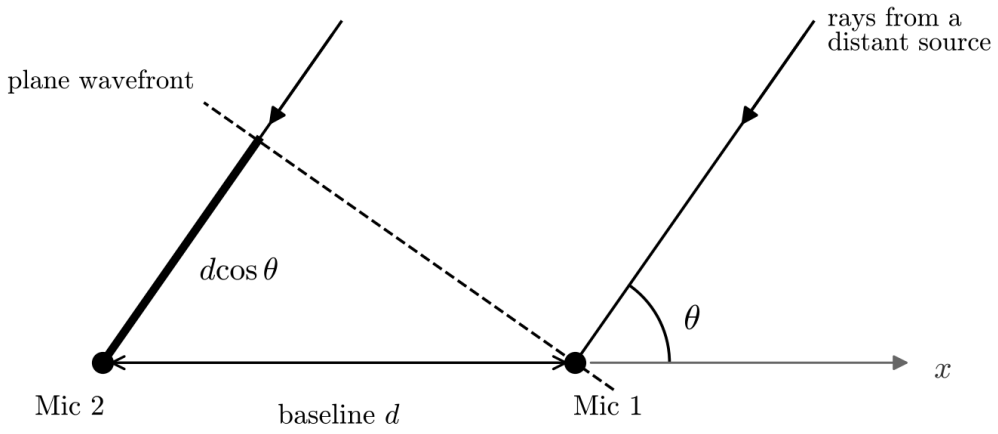


Figure 2.25: Geometry of a microphone pair: baseline d, parallel rays arriving at the microphones, plane wavefront through microphone 1, extra path $d\cos\theta$ and angle θ measured from the axis.

Draw the plane wavefront through microphone 1. By the time the wave reaches microphone 2, it has travelled an extra distance equal to the projection of the baseline onto the arrival direction. In the right triangle formed by the baseline, the wavefront and the ray to microphone 2, the baseline is the hypotenuse and the angle between it and the ray is θ, so the extra path is

$$\Delta r = d\cos\theta \tag{2.72}$$

Dividing by the speed of sound gives the delay, defined here as the arrival time at microphone 2 minus the arrival time at microphone 1:

$$\tau = t_2 - t_1 = \frac{d\cos\theta}{c} \tag{2.73}$$

The same result follows from the vector form of Equation 2.59, since the delay is the projection of the microphone separation onto the unit vector $\mathbf{u}$ divided by c. With the sign defined this way, a source on the microphone-1 side ($\theta < 90°$) gives $\tau > 0$, because microphone 1 hears it first. This is the same sign as τ_2 in Equation 2.59, which measures delay relative to microphone 1.

Inverting Equation 2.73 gives the estimator

$$\hat{\theta} = \cos^{-1}\left(\frac{c\,\hat{\tau}}{d}\right) \tag{2.74}$$

where $\hat{\tau}$ is the delay estimated from the data. The source measures the angle from the normal to the baseline, so its equivalent form is $\sin\theta_B = c\Delta/d$ and $\hat{\theta}_B = \sin^{-1}(c\hat{\Delta}/d)$ [47]. By Equation 2.70 this is the same relation as Equations 2.73 and 2.74. The source also normalises the delay by setting $f_s = 1$ Hz, so that its Δ is in samples, and the actual delay is Δ/f_s [47].

Range of delays and mapping

Because $|\cos\theta| \leq 1$, the delay is bounded by

$$|\tau| \le \tau_{max} = \frac{d}{c} \tag{2.75}$$

with the extremes at the two endfire directions ($\theta = 0^\circ, 180^\circ$). At broadside ($\theta = 90^\circ$) the delay is zero. An estimated delay with $|c\hat{\tau}/d| > 1$ cannot come from any angle, so it signals noise or a model error, and in practice it is clipped to ± 1 before Equation 2.74 is applied.

Hyperbola and plane-wave limit

In general, a fixed delay places the source on one branch of a hyperbola with the two microphones as its foci, because the difference of the two distances to the source is constant. Far from the pair, the hyperbola approaches a cone about the array axis with half-angle θ, which is the plane-wave result above [22]. This also explains why a single pair determines the angle to the axis but not the position of the source, as noted in Section 2.3.2.

Ambiguity

Equation 2.74 returns one angle in $[0^\circ, 180^\circ]$. Any source on the cone around the axis at that angle gives the same delay, including the mirror image on the other side of the axis, so the sign of the perpendicular offset is lost (Section 2.4.3). A third, non-collinear microphone resolves it. With microphone 1 at the origin and microphones 2 and 3 at $\mathbf{r}_2, \mathbf{r}_3$ in the horizontal plane, the two delays satisfy

$$\begin{bmatrix} \tau_{12} \\ \tau_{13} \end{bmatrix} = \frac{1}{c} \begin{bmatrix} \mathbf{r}_2^T \\ \mathbf{r}_3^T \end{bmatrix} \begin{bmatrix} \cos\theta \\ \sin\theta \end{bmatrix} \tag{2.76}$$

where $\tau_{1j} = t_1 - t_j$ as above. When the 2×2 matrix is invertible, which means the microphones are not collinear, the unit vector follows directly, and $\hat{\theta} = \operatorname{atan2}(\sin\theta, \cos\theta)$ gives the azimuth over the full circle. With noisy delays the vector is not exactly of unit length, and the least-squares form is used (Section 2.6.2).

Sampling and resolution

The data are sampled, so the delay is measured in units of $1/f_s$. The delay in samples is $D = f_s\tau$, and the largest value is

$$D_{max} = \frac{f_s d}{c} \tag{2.77}$$

so a pair can distinguish only about $2D_{max} + 1$ integer-sample delays. The angle step that corresponds to one sample is found by differentiating Equation 2.73:

$$\delta\theta \approx \frac{c}{f_s d\,|\sin\theta|} \tag{2.78}$$

Two properties follow (Figure 2.26). First, the step is smallest at broadside and grows without limit towards endfire, which matches the error behaviour in Equation 2.56 (Section 2.2.7). Second, larger spacing and a higher sampling rate give finer resolution. For $d = 0.1$ m, $f_s = 48$ kHz and $c = 346.47$ m/s, $D_{max} \approx 13.9$, so only 27 integer delays are available, and one sample at broadside corresponds to about 4.1°. Sub-sample interpolation (Section 2.6.1) is therefore needed to reach finer angles.

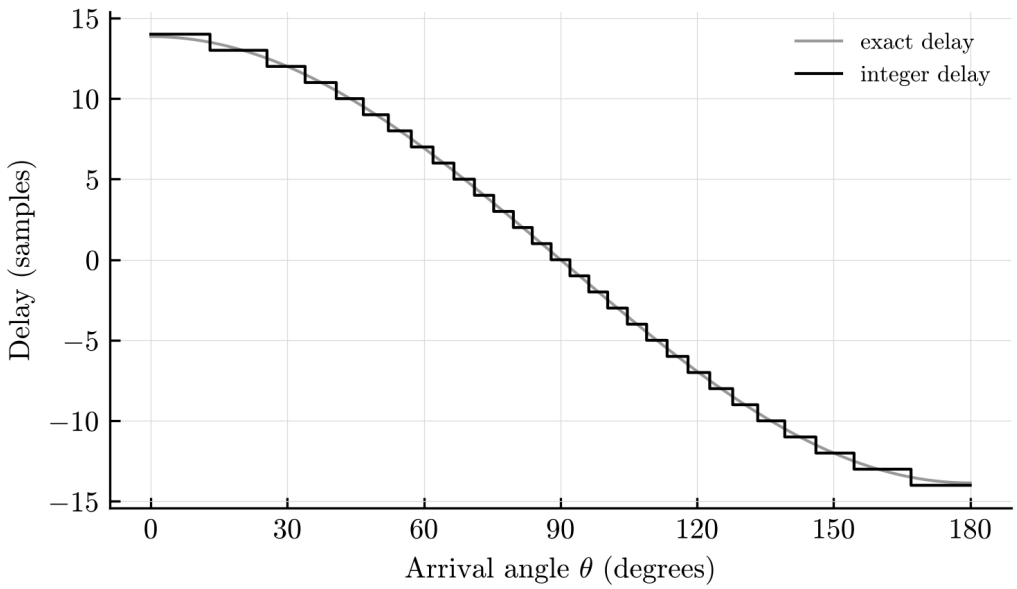


Figure 2.26: Delay against arrival angle for the array, with the integer-sample delays marked, showing the angle steps that widen towards endfire.

Assumptions

The relations above rest on the plane-wave assumption, which holds when the source distance is large compared with the baseline [47], with the conditions quantified in Section 2.5.1. They also need a known sound speed (Equation 2.4, exact microphone positions and synchronised sampling. Errors in each of these become angle errors, and Section 2.9 treats them.

2.4.3 Ambiguities

A localisation system is ambiguous when different directions produce the same measurements. The delay measured by a microphone pair depends only on the angle to the baseline (Equation 2.73, so

$$\tau(\theta) = \frac{d\cos\theta}{c} = \tau(-\theta) \tag{2.79}$$

Every direction at the same angle to the baseline gives the same delay, and together they form a cone about the baseline, the cone of confusion [42, 43]. In the horizontal plane this leaves two solutions, mirrored across the baseline. It is called the front-back ambiguity, after hearing, where the baseline joins the ears. Humans resolve it with spectral cues from the pinnae and with head movement [42]. A pair of omnidirectional microphones has no such cue, so two microphones cannot give a unique azimuth over the full circle.

Resolving it with a third microphone

Put microphone 1 at the origin, microphone 2 at $(d,0)$ and microphone 3 at (x_3,y_3) with $y_3 \neq 0$, so that the three are not collinear. Writing the two delays as in Equation 2.76 and solving in turn,

$$\cos\theta = \frac{c\,\tau_{12}}{d}, \qquad \sin\theta = \frac{c\,\tau_{13} - x_3\cos\theta}{y_3} \tag{2.80}$$

The second microphone fixes $\cos\theta$, and the third fixes the sign of $\sin\theta$, which removes the mirror solution. The azimuth is then $\hat{\theta} = \text{atan2}(\sin\theta, \cos\theta)$ over the full circle. Geometrically, the second baseline cuts the first cone along a different axis, and the two cones meet in a single direction in the plane.

Elevation

For a source at elevation ϕ, the planar array measures the horizontal component of the direction vector, $\mathbf{v} = \cos\phi\,[\cos\theta,\ \sin\theta]^T$, through $\mathbf{G}\mathbf{v} = c\tau$ (Equation 2.76 generalised). The azimuth still follows from the direction of $\mathbf{v}$, so a modest elevation does not bias it. The length of $\mathbf{v}$ gives only the magnitude of the elevation:

$$\|\mathbf{v}\| = \cos\phi \ \Rightarrow\ |\phi| = \cos^{-1}\|\mathbf{v}\| \tag{2.81}$$

A source above the array plane and its mirror image below produce identical delays, so the sign of ϕ is lost. Near the horizon, $\cos\phi \approx 1 - \phi^2/2$ barely changes with ϕ, so the elevation magnitude is also poorly conditioned. Only a non-coplanar array gives a unique full direction. Table 2.9 summarises what each array can determine, in line with the rank argument of Section 2.3.2.

Table 2.9: What each microphone arrangement determines

Arrangement	**Minimum mics**	**Determined**	**Remaining ambiguity**
Collinear (pair or line)	2	Angle to the axis	Cone of confusion, including front-back
Planar, non-collinear	3	Full azimuth, and $\|\phi\|$ (poorly)	Sign of elevation (above or below the plane)
Non-coplanar	4	Azimuth and elevation	None in the far field

Three further ambiguities do not come from the geometry. Spatial aliasing gives multiple directions per phase measurement when the spacing is too large (Section 2.5.3). A tone gives a delay only modulo its period (Section 2.2.4). Reflections appear as additional apparent sources in other directions (Section 2.1.3).

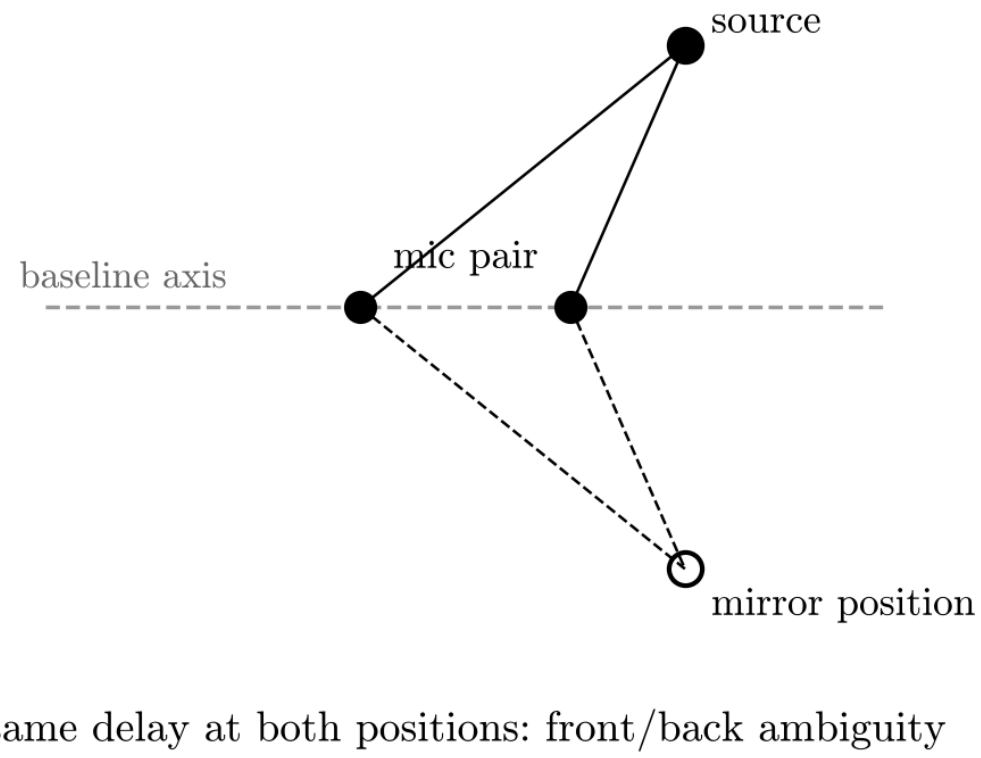


Figure 2.27: Two mirrored source positions across the baseline of a microphone pair that give the same delay (front–back ambiguity).

2.4.4 Performance measures

Direction-finding performance is described by several measures, each answering a different question (Table 2.10). Studies use different measures and conditions, so results can be compared only when they are defined (Section 2.3.4).

Accuracy

For trial i with true angle θ_i, the error must be wrapped, since 359° and 1° are 2° apart, not 358°:

$$e_i = \big((\hat{\theta}_i - \theta_i + 180^\circ) \bmod 360^\circ\big) - 180^\circ \tag{2.82}$$

The bias $\bar{e}$, the standard deviation s_e and the RMSE, which combines them, are

$$\bar{e} = \frac{1}{N}\sum_i e_i, \qquad \mathrm{RMSE} = \sqrt{\frac{1}{N}\sum_i e_i^2} = \sqrt{\bar{e}^2 + s_e^2} \tag{2.83}$$

This is the decomposition of Equation 2.53 applied to angles. Bias reflects systematic error such as a wrong c or misplaced microphone, and the standard deviation reflects noise (Section 2.2.7).

Resolution

For several sources, resolution is the smallest angular separation at which they can still be distinguished, and a common criterion is a probability of resolution above 90 % [37]. A conventional beamformer resolves roughly λ/L for array length L [37], while subspace and maximum-likelihood methods do better [37]. For a single source, the relevant quantity is the smallest detectable change in angle, set by the delay quantisation of Equation 2.78.

Robustness

Mean error hides rare large errors. Robustness is therefore reported as an outlier rate, $P(|e| > \varepsilon)$ for a chosen tolerance ε, and as the SNR or reverberation level at which this rate rises sharply (the threshold effect). Real environments with noise, echo and reverberation lower the performance of reported systems [41]. For a real-time system, the update rate and latency are also relevant.

Table 2.10: Performance measures and their use in this project

Measure	**Question answered**	**Used here**
Bias, standard deviation, RMSE	How close and how repeatable is the estimate	Per angle and per emitter type
Outlier rate	How often is the estimate wrong	Reverberant and low-SNR tests
Resolution	Smallest separation or change detected	Single source: Equation 2.78
Latency, update rate	Is it real-time	Display chain

The Cramér-Rao lower bound

No unbiased estimator can have a variance below the CRLB, $\mathrm{var}(\hat{\theta}) \geq 1/I(\theta)$, where $I(\theta)$ is the Fisher information [15]:

$$\mathrm{var}(\hat{\theta}) \;\geq\; \frac{1}{I(\theta)} \tag{2.84}$$

It gives the theoretical limit against which an algorithm is judged, so the gap between measured RMSE and the bound shows how much improvement is possible. For a single narrowband source in white noise, observed by a uniform linear array of M microphones with spacing d for N snapshots at per-microphone SNR ρ (complex signal, power ratio), the bound in radians^2 is [48]

$$\mathrm{CRB}(\theta) = \frac{6}{N\rho\, M(M^2-1)\,(2\pi d/\lambda)^2 \cos^2\theta_B} \tag{2.85}$$

with θ_B measured from broadside. It shows how accuracy improves with snapshots and SNR, with spacing relative to wavelength, and strongly with the number of microphones, roughly as M^3. It also shows that accuracy is best at broadside and diverges at endfire, as in Equation 2.56. For $M = 2$ it equals $1/(N\rho(2\pi d/\lambda)^2\cos^2\theta_B)$, which is half the real-tone estimate of Equation 2.55 after converting to angle with Equation 2.56. The factor 2 arises because Equation 2.55 uses a real sinusoid whereas Equation 2.85 assumes a complex signal. There is no simple closed form for a three-microphone planar array, so the bound is computed numerically from the Fisher information and compared with Monte Carlo trials from the simulator (Figure 2.28).

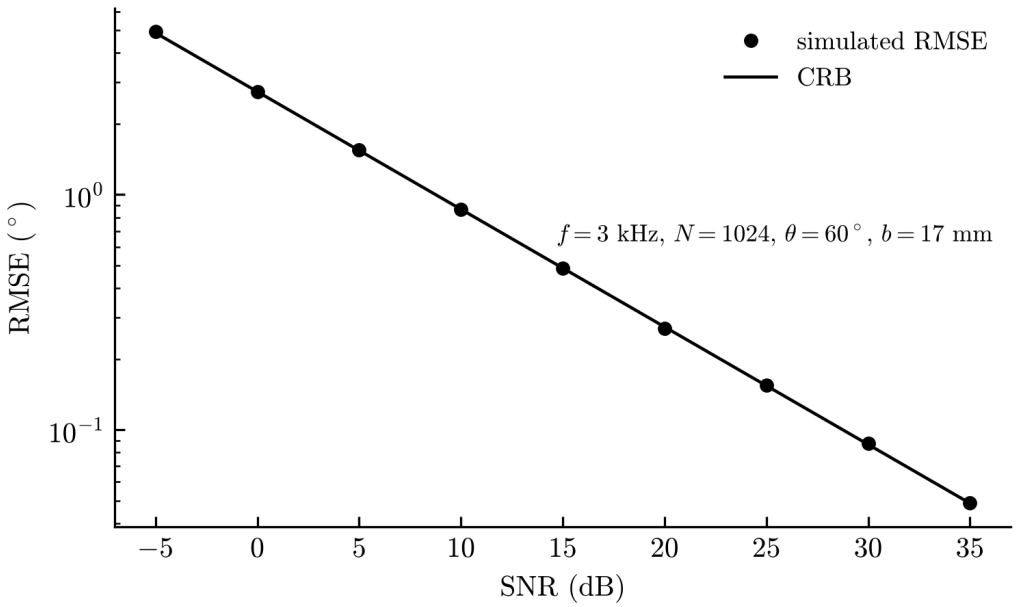


Figure 2.28: Simulated RMSE against SNR for the array, with the numerically computed Cramér–Rao bound overlaid.

2.5 Microphone array geometry

2.5.1 Array manifold and steering vectors

A plane wave arriving from direction $\mathbf{u}$ reaches microphone m earlier or later than microphone 1 by an amount set by its position (Equation 2.59. At frequency f this is a phase factor, and stacking the M factors gives the steering vector [43, 44]:

$$[\mathbf{a}(\mathbf{u}, f)]_m = e^{j\mathbf{k}^T(\mathbf{r}_m - \mathbf{r}_1)}, \qquad \mathbf{k} = \frac{2\pi f}{c}\mathbf{u} \tag{2.86}$$

This is the vector in Equation 2.63. For a uniform linear array on the x-axis with spacing d and angle θ from the axis,

$$[\mathbf{a}(\theta, f)]_m = e^{j2\pi f(m-1)d\cos\theta/c} \tag{2.87}$$

which matches the form in [37] with $\sin\theta_B = \cos\theta$. Each element has unit magnitude, so $\|\mathbf{a}\|^2 = M$. The array manifold is the set of steering vectors over all directions of interest,

$$\mathscr{A} = \{\mathbf{a}(\theta, \phi, f) \;:\; \theta \in [0^\circ, 360^\circ),\ \phi \in [-90^\circ, 90^\circ]\} \tag{2.88}$$

a curve or surface in $\mathbb{C}^M$ determined entirely by the geometry [43]. Its properties decide what an array can do. If two directions give the same vector, they are indistinguishable, which is the ambiguity of Section 2.4.3 and the spatial aliasing of Section 2.5.3. If any K steering vectors are linearly independent, K sources can be separated, which gives the $K \leq M - 1$ limit of Equation 2.67 [44]. The steering vector also makes the beamformer and subspace cost functions of Section 2.6 simple to write, because the array model enters only through $\mathbf{a}$.

When the plane-wave model holds

The steering vector assumes the plane-wave model. For a source at range r and an array of aperture D, the spherical wavefront at the edge of the array lags the plane approximation by about

$$\delta \approx \frac{D^2}{8r} \tag{2.89}$$

since $\sqrt{r^2 + (D/2)^2} - r \approx D^2/(8r)$ (Figure 2.29). Requiring this to stay below $\lambda/16$, which corresponds to a phase error of $\pi/8$, gives the far-field criterion [49, 22]:

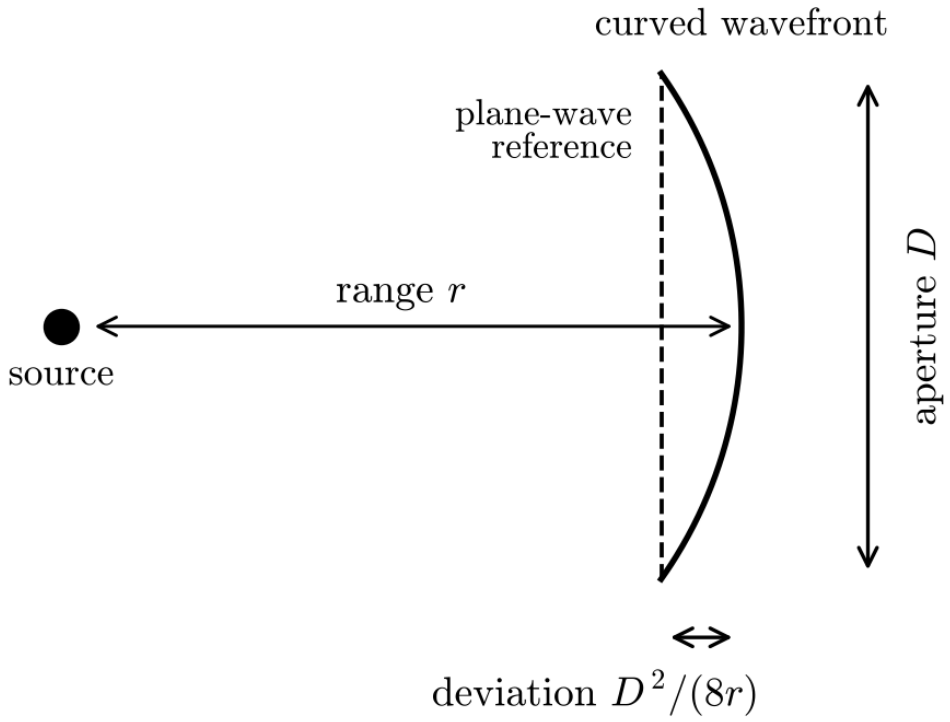


Figure 2.29: Wavefront deviation $D^2/(8r)$ across the aperture for a source at range r.

$$r > \frac{2D^2}{\lambda} \tag{2.90}$$

The strictest case is the highest frequency, where λ is smallest. For $D = 0.1$ m this requires $r > 0.29$ m at 5 kHz and $r > 1.15$ m at 20 kHz (at $c = 346.47$ m/s). Closer than this, the delays depend on range as well as direction (Equation 2.64, and a bearing-only estimate degrades. This is a different condition from Toole's rule on source size (Section 2.1.3), which concerns the source and not the array.

2.5.2 Common geometries

Array geometry decides coverage, the quantities that can be estimated and how well (Table 2.11). Linear arrays estimate direction accurately in the horizontal plane [38], and circular arrays give omnidirectional azimuth coverage but find elevation harder [38]. Hexagonal and other non-coplanar arrays balance azimuth and elevation precision [41, 38].

Table 2.11: Common array geometries and trade-offs

Geometry	**Min mics**	**Strengths**	**Weaknesses**
Linear (ULA)	2	Simple steering vector, well-understood beampattern, suits Root-MUSIC and ESPRIT [37]	Covers only a half-plane, with front-back ambiguity. Accuracy falls towards endfire
Circular (UCA)	3	Uniform azimuth coverage over 360° [50]	No shift invariance, so ESPRIT does not apply. Poorer elevation
Equilateral triangle	3	Smallest array with unique azimuth over 360°. Equal accuracy at all azimuths (Section 2.5.4)	Only two sources at most (Equation 2.67. No elevation sign
Planar (L-shaped, rectangular)	3 to 4	Simple orthogonal baselines	An L-shaped three-microphone array is anisotropic (Section 2.5.4)
Non-coplanar (tetrahedron, hexagonal)	4 or more	Full 3D direction [41]	More channels, harder to build
Ad hoc	any	Fits the application [38]	No standard methods, so calibration is essential

The equilateral triangle is the three-microphone case of the circular array, and the one this project uses (Section 2.5.5).

2.5.3 Spacing and spatial aliasing

Phase measures the delay (Equation 2.40, and a measured phase is known only modulo 2π (Section 2.2.4). Two microphones can therefore distinguish all directions only when the largest phase difference, at endfire, stays within π. This is Equation 2.41:

$$d \leq \frac{\lambda}{2} \quad \Longleftrightarrow \quad f \leq \frac{c}{2d} \tag{2.91}$$

For a uniform linear array steered to angle θ_0, the delay-and-sum beampattern is [37, 43]

$$B(\theta) = \left| \frac{\sin(M\psi/2)}{M\sin(\psi/2)} \right|, \qquad \psi = \frac{2\pi d}{\lambda}(\cos\theta - \cos\theta_0) \tag{2.92}$$

which has its main lobe at $\theta = \theta_0$. The pattern repeats whenever $\psi = 2\pi n$, at directions

$$\cos\theta = \cos\theta_0 + n\frac{\lambda}{d}, \qquad n = \pm 1, \pm 2, \ldots \tag{2.93}$$

If any of these lies within $|\cos\theta| \leq 1$, a second full-strength lobe, called a grating lobe, appears and the array cannot tell it from the true direction [50]. For every steering angle to be free of grating lobes, $\lambda/d \geq 2$, which gives Equation 2.91. This is the spatial counterpart of temporal aliasing (Section 2.2.1), with the half-wavelength spacing playing the role of the Nyquist rate.

Aperture and resolution

The width of the main lobe is of order λ/L, where L is the array length [37], so a larger aperture gives finer resolution. The aliasing rule limits the spacing, so for a fixed aperture, more microphones are needed to avoid grating lobes. The lobe also widens at low frequency (Figure 2.30), so a small array has poor resolution on low-frequency sources.

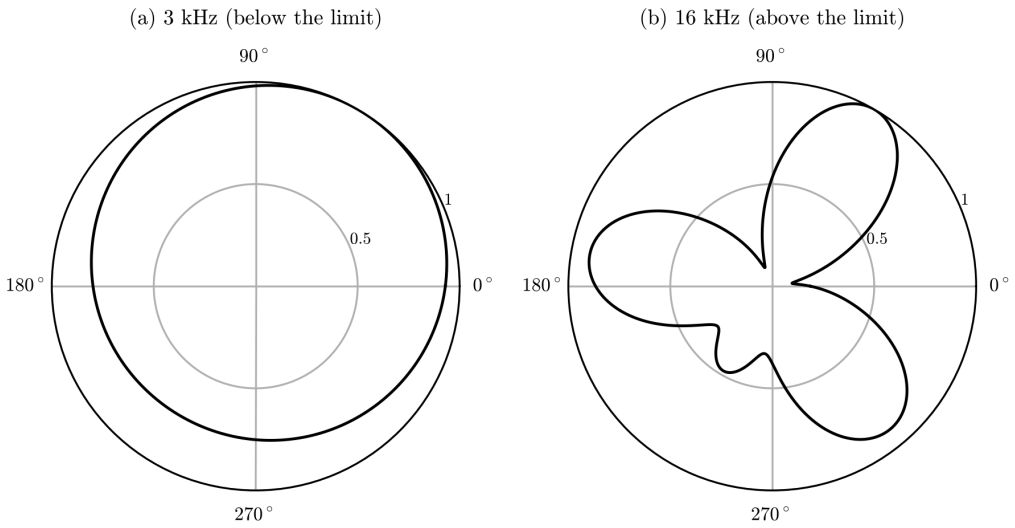


Figure 2.30: Polar beampatterns for the array at a frequency below and one above the aliasing limit.

Broadband sources

The rule applies at the highest frequency of interest, $d \leq c/(2f_{max})$. For $f_{max} = 20$ kHz this requires $d \leq 8.7$ mm. A spacing of 0.1 m is alias-free only up to 1.73 kHz (Section 2.2.4). The rule governs methods based on the phase at single frequencies, meaning tones and narrowband beamforming and subspace methods. A delay-based method acting on a broadband signal measures the time delay from the correlation peak and not from a wrapped phase (Section 2.2.4), so it can use larger spacings. The risk is then concentrated on tonal and narrowband sources, where the correlation has repeated peaks (Section 2.2.5). A lower limit also exists, since a small spacing gives a small delay range and poor resolution (Equations 2.77 and 2.78, and the physical size of the microphones limits how close they can be placed.

2.5.4 Geometry-dependent effects

Accuracy and geometry

Suppose the arrival time at each microphone has independent timing error of standard deviation σ_t. Fitting $\mathbf{v}$ by least squares to the measured times, with positions $\tilde{\mathbf{r}}_m$ taken relative to the array centroid, gives the covariance [51]

$$\operatorname{cov}(\hat{\mathbf{v}}) = c^2\sigma_t^2\left(\sum_m \tilde{\mathbf{r}}_m\tilde{\mathbf{r}}_m^T\right)^{-1} \tag{2.94}$$

The matrix in brackets depends only on geometry, so it acts as a dilution-of-precision factor, in the sense used for satellite positioning [52]. It converts timing error to direction error. For an equilateral triangle of side d, $\sum\tilde{\mathbf{r}}\tilde{\mathbf{r}}^T = (d^2/2)\mathbf{I}$, which is isotropic, and the azimuth error is the same at every angle:

$$\sigma_\theta = \frac{\sqrt{2}\,c\,\sigma_t}{d} \tag{2.95}$$

For a linear array of M microphones, $\sum\tilde{x}^2 = d^2M(M^2-1)/12$, and the error depends on the azimuth:

$$\sigma_\theta = \frac{\sqrt{12}\, c\, \sigma_t}{d\sqrt{M(M^2-1)}\,|\sin\theta|} \tag{2.96}$$

For $M = 2$ this reduces to $\sqrt{2}c\sigma_t/(d|\sin\theta|)$, the pair result of Equation 2.56 with $\sigma_\tau^2 = 2\sigma_t^2$. So the triangle matches the best case of a pair, at broadside, in all directions, while a line is better at broadside but fails towards endfire and cannot cover the full circle (Figure 2.31). A three-microphone L-shape with the same shortest baseline has $\sum \tilde{\mathbf{r}}\tilde{\mathbf{r}}^T$ with eigenvalues d^2 and $d^2/3$, so its worst direction is worse than the triangle's $d^2/2$ (Table 2.11). This is my own working, which assumes small errors, independent timing noise and a free field.

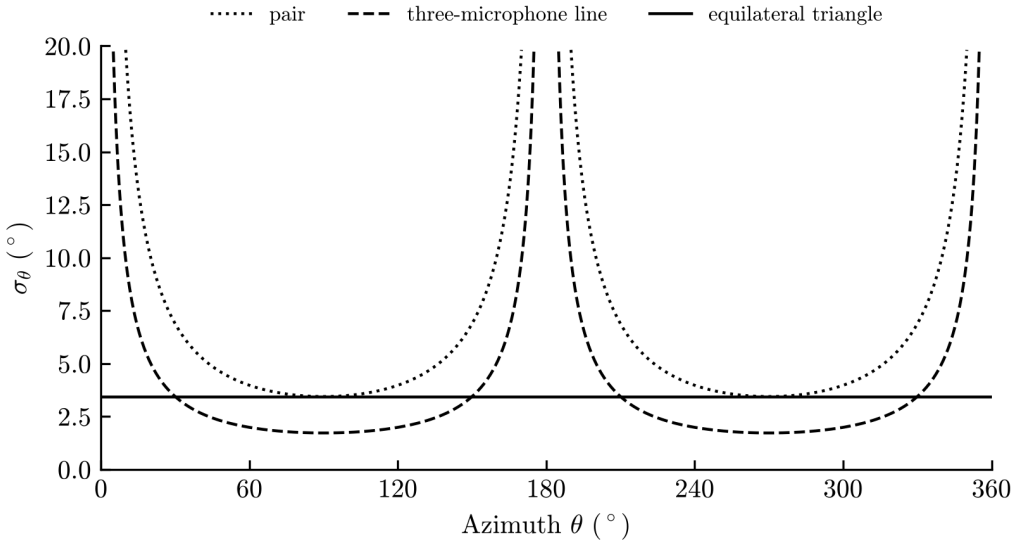


Figure 2.31: Angular accuracy σ_θ against azimuth for a pair, a three-microphone line and the equilateral triangle.

Position errors

A microphone displaced by $\delta\mathbf{r}_m$ gives a delay error $\mathbf{u}^T\delta\mathbf{r}_m/c$ (Equation 2.59. For a pair, an error δd in the spacing changes the angle by

$$\delta\theta \approx \cot\theta\, \frac{\delta d}{d} \tag{2.97}$$

and a transverse displacement δy of one microphone tilts the baseline, adding a bias of about $\delta y/d$ radians. For $d = 100$ mm, a 1 mm error in either gives up to 0.57°, and a 1 % scale error at $\theta = 45°$ gives the same. An error δc in the speed of sound has the same form as Equation 2.97 with $\delta c/c$, so the 1.8 % temperature error of Section 2.1.2 gives about 1.0° at 45°. All of these vanish at broadside and grow towards endfire. Unlike noise, they do not average out (Section 2.2.7), so they are removed by measuring the geometry carefully and calibrating (Section 2.9).

2.5.5 Justification of the three-microphone case

The brief sets a minimum of three microphones. The rank argument of Section 2.3.2 and the ambiguity analysis of Section 2.4.3 show that this is the smallest array giving a unique azimuth over the full circle.

An equilateral triangle is the natural choice, because it has isotropic accuracy (Equation 2.95 and every pair has the same baseline, which keeps the delay range and aliasing limit equal for all three pairs.

Three microphones also give three pair delays but only two independent ones, since

$$\tau_{12} + \tau_{23} + \tau_{31} = 0 \tag{2.98}$$

for exact delays. With estimated delays, the sum is a residual that measures how consistent the measurements are, and a large value flags a bad estimate caused by noise or reverberation. This is a useful quality check that two microphones cannot provide.

The limits follow from the earlier sections. There is no elevation sign (Table 2.9), at most two sources can be separated and only with narrowband subspace methods (Equation 2.67, and the redundancy is a single consistency check, so no channel can fail. Extensions are straightforward. A fourth non-coplanar microphone gives elevation, and more microphones improve accuracy sharply (the M^3 dependence in Equation 2.85, allow multiple sources and add redundancy. They also add acquisition channels, data rate and calibration effort (Section 2.7). Three microphones are a deliberate minimum that fits the low-cost, low-complexity aim [46] and the terms of reference, and the limits above are the reasons future work would add microphones.

2.6 Direction-of-arrival algorithms

Each subsection gives the principle, the mathematics, the assumptions, strengths and weaknesses, and suitability for this project (three microphones, wideband and tonal sources, real time). All use the model of Section 2.3.2 and the convention $\tau_{ij} = t_i - t_j$ of Section 2.4.2.

2.6.1 Time delay estimation

Principle

The delay between two channels is the lag at which their cross-correlation peaks (Section 2.2.5). Generalised cross-correlation (GCC) applies a frequency weighting before returning to the lag domain, to sharpen the peak and suppress noise and reverberation [13].

Derivation

With $G_{12}[k] = X_1[k]X_2^*[k]$ the cross-spectrum of a frame (Equation 2.35, and a weighting $\Psi[k]$,

$$R_{12}[n] = \mathrm{IDFT}\big\{\Psi[k]\, G_{12}[k]\big\}, \qquad \hat{\tau} = \frac{1}{f_s} \arg \max_{|n| \le D_{max}} R_{12}[n] \tag{2.99}$$

For $x_1 = s[n - D_1]$ and $x_2 = s[n - D_2]$, $G_{12} = |S|^2 e^{-j2\pi f(D_1 - D_2)/f_s}$, so the peak falls at $D_1 - D_2$, which is $\tau = t_1 - t_2$ as in Equation 2.73. The search is limited to $|n| \le D_{max}$ (Equation 2.77, because delays beyond it are physically impossible. The common weightings are [13, 28]

$$\Psi_{\mathrm{PHAT}} = \frac{1}{|G_{12}|}, \qquad \Psi_{\mathrm{SCOT}} = \frac{1}{\sqrt{G_{11}G_{22}}}, \qquad \Psi_{\mathrm{ML}} = \frac{|\gamma_{12}|^2}{|G_{12}|\big(1 - |\gamma_{12}|^2\big)} \tag{2.100}$$

where $\gamma_{12} = G_{12}/\sqrt{G_{11}G_{22}}$ is the coherence. Unweighted GCC ($\Psi = 1$) is the plain cross-correlation. SCOT whitens each channel by its own spectrum. The ML weight is optimal for a Gaussian signal in uncorrelated noise, but it needs the coherence, which must be estimated, and it is sensitive to that estimate. PHAT keeps only the phase of the cross-spectrum. For a single free-field delay, $\Psi_{\text{PHAT}} G_{12} = e^{-j2\pi f\tau}$ has unit magnitude at every frequency, and its inverse transform is an ideal impulse at the true delay. Discarding magnitude also makes PHAT insensitive to gain mismatch between microphones (Section 2.9). The spectra G_{ij} are estimated by averaging over frames (Section 2.2.3), and the DFT length must be at least twice the frame length to avoid circular wrap-around.

Sub-sample interpolation

The peak index is an integer, which gives the coarse angle step of Equation 2.78. Two refinements are used:

- *Parabolic interpolation* fits a parabola through the peak sample y_0 and its neighbours y_{-1}, y_{+1} and takes the vertex [53]:

$$\hat{\delta} = \frac{1}{2}\,\frac{y_{-1} - y_{+1}}{y_{-1} - 2y_0 + y_{+1}}, \qquad \hat{\tau} = \frac{n_0 + \hat{\delta}}{f_s} \tag{2.101}$$

  with $|\hat{\delta}| \le \frac{1}{2}$. It is cheap, but the true PHAT peak is much sharper than a parabola, so the estimate is biased towards the nearest integer lag (peak locking) [53].

- *Upsampling* zero-pads the spectrum G_{12} by a factor L before the inverse transform, which interpolates the lag axis to a spacing of $1/(Lf_s)$ with no peak-shape assumption. It costs a longer inverse FFT. A common compromise is to upsample by $L = 4$ to 16 and then apply the parabolic step.

Assumptions

One source, a direct path that dominates, noise uncorrelated between channels, and a stationary source over the frame.

Strengths and weaknesses

GCC-PHAT is cheap, has a closed-form delay, and its sharp peak resists moderate reverberation, which is why it is widely used for speech [54]. It breaks down for several simultaneous sources, because each pair then has several peaks, and for strong reverberation, which adds peaks at wrong lags. PHAT also amplifies noise in frequency bins where the signal is weak, so the weighting is applied only inside the signal band.

Suitability

An impulse is the ideal case, with a sharp peak, although a short event gives few frames to average. Speech works frame by frame with PHAT, and silent frames must be detected and skipped. A single tone is poorly suited, since PHAT leaves only one non-zero bin and the correlation becomes $\cos(2\pi f_0(\tau - \tau_0))$, with peaks every $1/f_0$ (Section 2.2.5). The delay is then found modulo the period unless $d \le \lambda/2$ (Section 2.5.3).

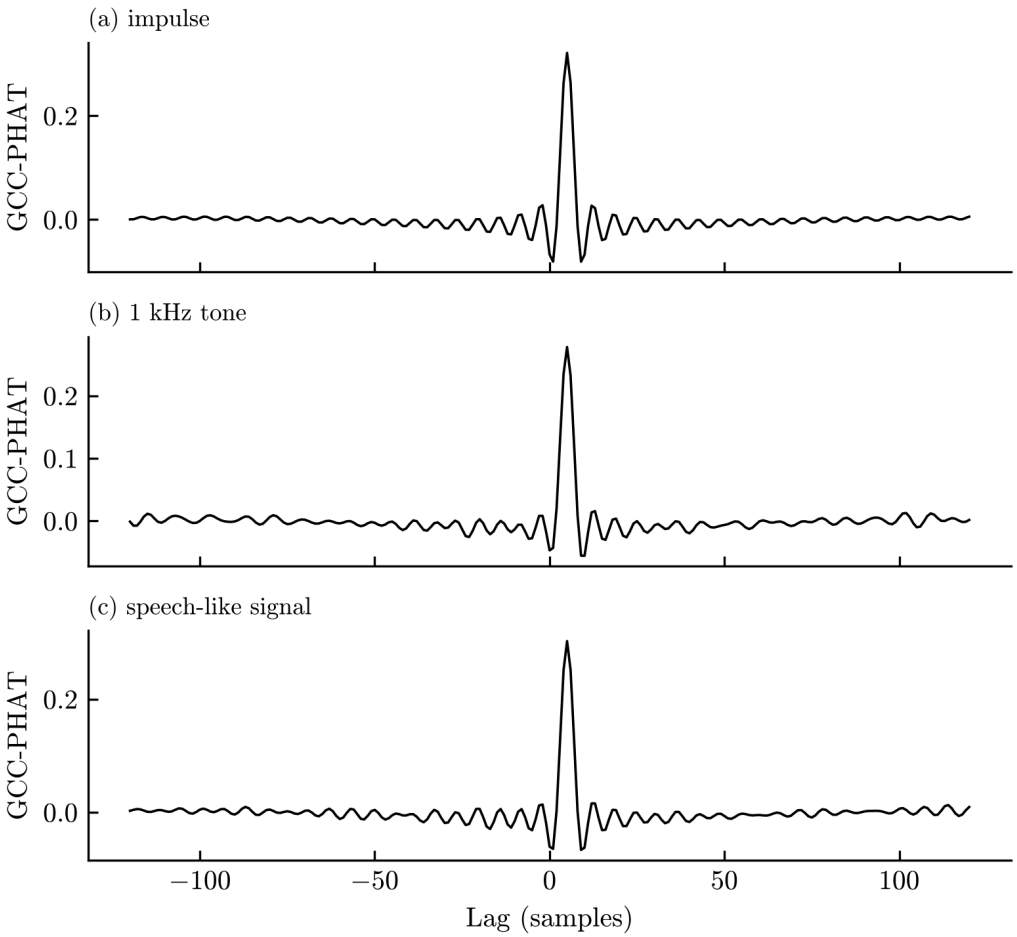


Figure 2.32: GCC-PHAT correlation functions for an impulse, a tone and a speech-like signal, showing sharp and repeated peaks (the speech-like signal is synthetic, amplitude-modulated band-limited noise).

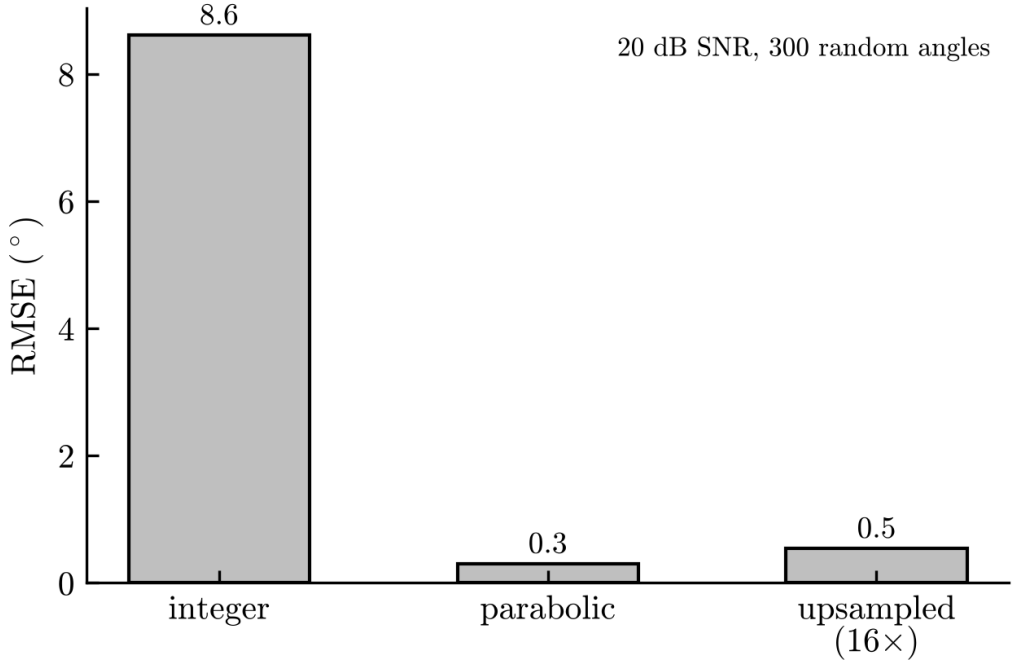


Figure 2.33: Angle error against delay-interpolation method (integer, parabolic and upsampled).

2.6.2 TDOA-to-DOA conversion

Principle

The pair delays are linear in the unknown direction vector (Equation 2.76, so the direction is found by linear least squares.

Derivation

Stack the baselines $\mathbf{b}_p = \mathbf{r}_j - \mathbf{r}_i$ of the P microphone pairs as the rows of $\mathbf{G}$. With $\mathbf{v} = [\cos\theta,\ \sin\theta]^T$ in the plane,

$$c\,\tau = \mathbf{G}\,\mathbf{v} + \varepsilon \tag{2.102}$$

where ε is the delay error, and the least-squares solution (with optional weights $\mathbf{W}$ equal to the inverse delay variances) is

$$\hat{\mathbf{v}} = \left(\mathbf{G}^T\mathbf{W}\,\mathbf{G}\right)^{-1}\mathbf{G}^T\mathbf{W}\,c\,\tau, \qquad \hat{\theta} = \operatorname{atan2}(\hat{v}_y, \hat{v}_x) \tag{2.103}$$

Taking atan2 of the vector imposes $\|\mathbf{v}\| = 1$ after the fit. With only the two pairs 12 and 13, $\mathbf{G}$ is 2×2 and the solution is exact, as in Equation 2.80. Using all three pairs gives one redundant equation, which averages the noise.

Three-microphone worked case

For an equilateral triangle with side d, put microphone 1 at the origin, microphone 2 at $(d, 0)$ and microphone 3 at $(d/2,\ \sqrt{3}d/2)$. The three baselines have length d and angles $\beta = 0°, 60°, 120°$, so the rows of $\mathbf{G}$ are $d[\cos\beta,\ \sin\beta]$ for pairs 12, 13 and 23, with $\tau_{23} = t_2 - t_3$. The matrix $\mathbf{G}^T\mathbf{G} = \frac{3}{2}d^2\mathbf{I}$ is a multiple of the identity (the isotropy behind Equation 2.95, so the inverse is trivial and, with $\mathbf{W} = \mathbf{I}$,

$$\begin{aligned} \hat{v}_x &= \frac{2c}{3d}\left(\tau_{12} + \tfrac{1}{2}\tau_{13} - \tfrac{1}{2}\tau_{23}\right) \\ \hat{v}_y &= \frac{2c}{3d}\cdot\frac{\sqrt{3}}{2}\left(\tau_{13} + \tau_{23}\right) \end{aligned} \tag{2.104}$$

Substituting $\tau_{12} = \frac{d}{c}\cos\theta$, $\tau_{13} = \frac{d}{c}\cos(\theta - 60°)$ and $\tau_{23} = \frac{d}{c}\cos(\theta - 120°)$ returns $\hat{v}_x = \cos\theta$ and $\hat{v}_y = \sin\theta$ exactly. The estimate is a fixed linear combination of three delays followed by one atan2, so it costs a handful of multiplications.

Consistency check

Exact delays satisfy $\tau_{13} = \tau_{12} + \tau_{23}$ (Equation 2.98, so the residual

$$\varepsilon = \tau_{12} + \tau_{23} - \tau_{13} \tag{2.105}$$

is zero for a clean measurement and measures the estimation error when it is not. A frame with $|\varepsilon|$ above a threshold (a few samples) can be rejected, which suppresses reverberation outliers.

Assumptions

Far field, free field, known c and positions, and one source (Table 2.6). Near-field hyperbolic or nonlinear least-squares forms exist [22] but are not needed here.

Strengths and weaknesses

It is closed form, fast, and gives the full 360° azimuth. Its accuracy equals that of the delays (Equation 2.95, and it has no mechanism to detect a second source.

Suitability

It is the natural back end for GCC-PHAT, giving the two-step (indirect) method that the taxonomy in Section 2.3.4 identifies as the common choice for low-cost systems.

2.6.3 Beamforming

Principle

Steer the array towards each candidate direction by compensating the expected delays, sum the channels, and take the direction of maximum output power [37].

Derivation

At bin k with steering vector $\mathbf{a}_k(\theta)$ (Equation 2.86, the delay-and-sum output and the steered response power (SRP) summed over the analysis band are

$$y_k(\theta) = \frac{1}{M}\mathbf{a}_k^H(\theta)\,\mathbf{x}[k], \qquad P_{\mathrm{SRP}}(\theta) = \sum_k \left|y_k(\theta)\right|^2 = \frac{1}{M^2}\sum_k \mathbf{a}_k^H \hat{\mathbf{R}}_k\,\mathbf{a}_k \tag{2.106}$$

with $\hat{\mathbf{R}}_k$ the covariance estimate at bin k [37]. Expanding the squared sum shows that SRP is a sum of pair correlations evaluated at the steered lags [54]:

$$P_{\mathrm{SRP}}(\theta) = \frac{1}{M^2}\Big[\sum_m E_m + 2\sum_{i<j} R_{ij}\big(\tau_{ij}(\theta)\big)\Big] \tag{2.107}$$

where E_m is the energy of channel m and R_{ij} is the unweighted cross-correlation. This links beamforming to Section 2.6.1. Replacing R_{ij} by its PHAT-weighted version gives SRP-PHAT [54]:

$$P_{\text{SRP-PHAT}}(\theta) = \sum_{i<j} R_{ij}^{\mathrm{PHAT}}\big(\tau_{ij}(\theta)\big), \qquad \hat{\theta} = \arg\max_\theta P_{\text{SRP-PHAT}}(\theta) \tag{2.108}$$

The search is over a grid of angles. At 1° spacing this is 360 points, each needing a lookup in three correlation functions, so the cost is negligible once the GCCs exist.

Assumptions

One dominant source, plane wave, known geometry. Resolution is limited to about λ/L [37], so with a 0.1 m aperture the main lobe is very wide at low frequencies (Section 2.5.3).

Strengths and weaknesses

Delay-and-sum is simple, robust to model mismatch and cheap, but has limited resolution, poor performance against strong interferers and sidelobe interference [37]. SRP-PHAT inherits the robustness of PHAT to reverberation [54], and it combines all pairs into one function, which avoids the pair-by-pair decision of Section 2.6.2. In exchange, it needs the grid search, and its resolution does not exceed that of GCC for a single source.

Suitability

SRP-PHAT is a sound second method for broadband sources and a useful alternative to the methods implemented in this project. For tones the wide beam gives a weak, broad peak.

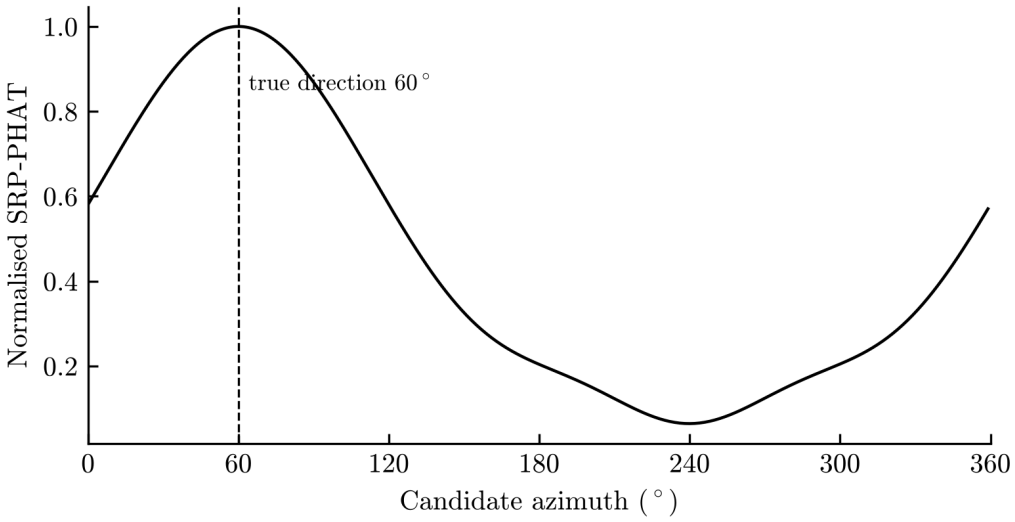


Figure 2.34: SRP-PHAT spatial spectrum for the triangular array.

2.6.4 Adaptive beamforming (MVDR/Capon)

Principle

Choose the weights that minimise output power while passing the look direction undistorted, so that interference from other directions is suppressed [55, 37].

Derivation

$$\min_{\mathbf{w}} \mathbf{w}^H \mathbf{R}\, \mathbf{w} \quad \text{subject to} \quad \mathbf{w}^H \mathbf{a}(\theta) = 1 \tag{2.109}$$

A Lagrange multiplier gives the weights and spatial spectrum [37]:

$$\mathbf{w}(\theta) = \frac{\mathbf{R}^{-1}\mathbf{a}(\theta)}{\mathbf{a}^H(\theta)\mathbf{R}^{-1}\mathbf{a}(\theta)}, \qquad P_{\text{Capon}}(\theta) = \frac{1}{\mathbf{a}^H(\theta)\mathbf{R}^{-1}\mathbf{a}(\theta)} \tag{2.110}$$

With few snapshots the sample covariance is ill-conditioned, so diagonal loading $\hat{\mathbf{R}} + \varepsilon \mathbf{I}$ is used [37]. For broadband data the spectrum is computed per frequency bin and the results are combined across bins.

Assumptions

A calibrated array and a covariance estimated from enough independent snapshots, in the narrowband (per-bin) sense.

Strengths and weaknesses

It resolves better than conventional beamforming and nulls interferers, but needs a matrix inversion, is sensitive to steering-vector errors and to a limited number of snapshots, and can cancel the signal itself [37].

Suitability

For a three-microphone array with uncalibrated, mismatched channels the advantage is small and the sensitivity to mismatch is a real risk (Section 2.9). It is reviewed for completeness.

2.6.5 Subspace methods

Principle

Split the covariance matrix into a signal subspace and a noise subspace, and find directions whose steering vectors are orthogonal to the noise subspace [56, 37].

Derivation

For K sources, $\mathbf{R} = \mathbf{A}\mathbf{R}_s\mathbf{A}^H + \sigma^2\mathbf{I}$, estimated as $\hat{\mathbf{R}} = \frac{1}{N}\sum_n \mathbf{x}[n]\mathbf{x}^H[n]$ [37]. Its eigendecomposition has K eigenvalues above σ^2 and $M-K$ equal to σ^2:

$$\mathbf{R} = \mathbf{U}\Lambda\mathbf{U}^H, \qquad \mathbf{U}_n = [\mathbf{u}_{K+1}, \ldots, \mathbf{u}_M], \qquad \mathbf{U}_n^H \mathbf{a}(\theta_k) = \mathbf{0} \tag{2.111}$$

The MUSIC spectrum peaks where this orthogonality holds [37]:

$$P_{\mathrm{MUSIC}}(\theta) = \frac{1}{\mathbf{a}^H(\theta)\,\mathbf{U}_n\mathbf{U}_n^H\,\mathbf{a}(\theta)} \tag{2.112}$$

With $M = 3$ and $K = 1$, the noise subspace has dimension two. ESPRIT instead uses the shift invariance between two subarrays to get the angles from the eigenvalues of a small matrix, with no spectral search [57, 37]. It needs a geometry that splits into two identical, shifted subarrays, such as a uniform linear array [37]. An equilateral triangle has no such structure, so ESPRIT does not apply directly.

Wideband extensions

The model above is narrowband (Section 2.2.8). Two extensions handle wideband sources [37]:

- *Incoherent* methods apply MUSIC independently in each frequency bin and combine the spectra,

$$P_{\mathrm{ISSM}}(\theta) = \sum_k \frac{1}{\mathbf{a}_k^H(\theta)\,\mathbf{U}_{n,k}\mathbf{U}_{n,k}^H\,\mathbf{a}_k(\theta)} \tag{2.113}$$

  which is simple but does not exploit phase coherence across bins.

- *Coherent* methods first transform all bins to a reference frequency with focusing matrices and then average the covariances [58, 59, 37]. They perform better at low SNR, but they need an initial angle estimate that, if wrong, biases the result.

Assumptions

Known number of sources $K < M$, uncorrelated sources, white noise, a calibrated array and enough snapshots [37]. Coherent sources, such as a direct path and its reflection, collapse the rank of the source covariance, and the standard remedy (spatial smoothing or forward-backward averaging) needs a uniform linear array [37].

Strengths and weaknesses

Super-resolution, and several sources resolved, at the cost of more computation, the need to know K and sensitivity to calibration errors [37]. The eigendecomposition itself is trivial at $M = 3$, and the cost is the grid search over bins.

Suitability

For one source and three microphones MUSIC adds little over SRP-PHAT, and reverberation produces coherent arrivals, which is the case it handles worst. Source counting (AIC and MDL [60, 37]) cannot work well with only three eigenvalues. It is included as a reference method.

2.6.6 Maximum likelihood methods

Principle

Choose the angles that make the observed data most probable under the signal model [48, 37].

Derivation

In the deterministic form, the source signals are treated as unknown constants and the noise as white Gaussian. Concentrating out the signals leaves the cost [37]

$$\hat{\theta}_{\mathrm{DML}} = \arg\min_{\theta} \operatorname{tr}\left[\mathbf{P}^{\perp}(\theta)\,\hat{\mathbf{R}}\right], \qquad \mathbf{P}^{\perp} = \mathbf{I} - \mathbf{A}(\mathbf{A}^{H}\mathbf{A})^{-1}\mathbf{A}^{H} \tag{2.114}$$

The stochastic form models the sources as Gaussian with unknown covariance and leads to a different, asymptotically more efficient cost [37]. For one source, $\mathbf{A} = \mathbf{a}(\theta)$ with $\mathbf{a}^{H}\mathbf{a} = M$, so $\operatorname{tr}[\mathbf{P}^{\perp}\hat{\mathbf{R}}] = \operatorname{tr}\hat{\mathbf{R}} - \mathbf{a}^{H}\hat{\mathbf{R}}\mathbf{a}/M$, and minimising it is the same as maximising $\mathbf{a}^{H}\hat{\mathbf{R}}\mathbf{a}$:

$$\hat{\theta}_{\mathrm{DML}} = \arg\max_{\theta} \mathbf{a}^{H}(\theta)\,\hat{\mathbf{R}}\,\mathbf{a}(\theta) \tag{2.115}$$

This is the delay-and-sum spectrum of Equation 2.106. For a single source in white noise, the deterministic maximum-likelihood estimator and the conventional beamformer therefore coincide, and the GCC with the ML weight of Equation 2.100 is the corresponding time-delay estimator.

Assumptions

The model must be correct, including white noise and a known K, and the search is global and, for several sources, multi-dimensional.

Strengths and weaknesses

It is asymptotically optimal and approaches the Cramér-Rao bound at high SNR [37], but it is computationally intensive and sensitive to model mismatch.

Suitability

For one source the search is one-dimensional and cheap, but it adds nothing beyond SRP in the free-field white-noise case, and in the reverberant case the model is wrong. It is reviewed for completeness, and its main use here is as the benchmark that the Cramér-Rao bound represents (Section 2.4.4).

2.6.7 Multi-source localisation

Several simultaneous sources bring three difficulties. The number of sources must be known or estimated [60, 37]. The sources must be separable, since a narrowband subspace method resolves at most $K \leq M - 1$ sources (Equation 2.67, so three microphones give two. The correlation pair-by-pair approach produces several peaks per pair, and matching them across pairs creates false "ghost" directions [54].

The main methods are the subspace and maximum-likelihood methods above, sparse reconstruction on an angular grid, which can resolve more sources than sensors but is computationally costly [37], and time-frequency masking. Speech sources rarely occupy the same time-frequency bin, so each bin can be assigned a direction from its inter-channel phase, and a histogram of these directions shows one peak per source, even with more sources than microphones [61]. This project is limited to one source, so multi-source work is future work (Section 2.3.3).

2.6.8 Learning-based approaches

Convolutional networks learn a mapping from features to direction. Typical inputs are the phase of the STFT [41], GCC-PHAT vectors [41], or the raw waveform [41], and the architectures include CNNs, CNNs with LSTM layers and convolutional recurrent networks that detect and localise several events [41]. The review [41] reports the lowest errors for CNN-based systems, but studies use different data and metrics, so the figures cannot be compared directly (Section 2.4.4).

They are outside the scope of this project for four reasons: they need labelled training data, whose collection and generalisation across rooms and array geometries is the main difficulty [41, 38]; they are harder to interpret and to relate to the physical error sources; they are heavier to run on a small system; and the brief asks for a propagation-model simulator and analysis. A hybrid, in which a network takes GCC-PHAT vectors as input, is a natural extension.

2.6.9 Comparative analysis

Table 2.12: Comparison of direction-of-arrival methods for this project's conditions (qualitative, from [37, 54] and the analysis above) (a)

Method	**Complexity (per frame)**	**Resolution**	**Noise and reverberation**	**Wideband handling**
GCC-PHAT + LS	$O(PN\log N)$ for P pairs	Delay limited by f_s, refined by interpolation	Good against moderate reverberation, noisy in weak bins	Native, uses delay
SRP-PHAT	GCCs plus $O(PI)$ grid search	Beamwidth $\sim \lambda/L$	Good, robust	Native
Delay-and-sum	$O(M^2N+MI)$ [37]	$\sim \lambda/L$ [37]	Robust, sidelobes	Per bin, summed
Capon (MVDR)	$O(M^2N+M^3+M^2I)$ [37]	Better than DAS	Sensitive to mismatch	Per bin
MUSIC	$O(M^2N+M^3+MI)$ [37]	Super-resolution (many mics)	Fails for coherent arrivals	Needs ISSM or CSSM
ESPRIT	$O(M^2N+M^3+K^3)$ [37]	Super-resolution	Needs calibration	Needs extension
Maximum likelihood	Iterative, $O(I_{iter}M^3)$ [37]	Near the CRB	Model-dependent	Per bin
CNN	Heavy to train, moderate to run	Learned	Robust if trained on similar conditions	Learned

Table 2.12: Comparison of direction-of-arrival methods for this project's conditions (qualitative, from [37, 54] and the analysis above) (b)

Method	**Real-time (3 mics, laptop)**	**Tones**	**Impulses**	**Speech**
GCC-PHAT + LS	Excellent	Poor (periodic peaks)	Excellent	Good
SRP-PHAT	Excellent	Fair	Good	Good
Delay-and-sum	Excellent	Fair	Fair	Fair
Capon (MVDR)	Good	Fair	Poor (few snapshots)	Fair
MUSIC	Good	Good (narrowband)	Poor	Fair
ESPRIT	Good	Good	Poor	Fair
Maximum likelihood	Fair	Good	Poor	Fair
CNN	Needs GPU or optimised code	Data-dependent	Data-dependent	Data-dependent

ESPRIT is not applicable to the triangular array.

Conclusion

Published comparisons use arrays of 8 to 64 narrowband elements [37], which do not transfer to three microphones. At $M = 3$ the advantages of subspace and adaptive methods shrink (Section 2.4.4), while the time-delay and SRP-PHAT methods match wideband sources and a real-time laptop display. The project therefore implements three methods that are simple to execute and share the same front end of phase differences between the channels: a trigonometric method on the FFT phase (Trig + FFT), MUSIC, and a look-up table with a cost function (LUT + cost function). They were chosen because they are easy to execute, of limited complexity, and reported in the literature to perform well. GCC-PHAT with least squares and SRP-PHAT remain the main alternatives and are not implemented.

2.7 Hardware subsystems

2.7.1 Microphones

The array needs small omnidirectional microphones with matched response. Table 2.13 compares the main types. The values are typical and must be checked against the datasheet of the chosen part.

Table 2.13: Microphone types

Type	**Output**	**Typical SNR and sensitivity tolerance**	**Advantages**	**Disadvantages**
Electret condenser	Analogue, needs bias and preamplifier	SNR to about 60–70 dB; tolerance often ± 3 dB	Low cost, good acoustic performance, wide choice	Larger, matching needs selection, external front end (Section 2.7.2)
Analogue MEMS	Analogue, low impedance	SNR about 60–67 dB; tolerance about ± 1–2 dB	Small, consistent unit to unit	Still needs an anti-alias filter and ADC
Digital MEMS, PDM	1-bit oversampled stream	SNR about 60–65 dB; ± 1 dB	No analogue front end, shared clock, small	Needs decimation filter in the host (Section 2.7.4)
Digital MEMS, I2S	Multi-bit words at f_s	SNR about 65 dB; ± 1 dB	ADC and decimation inside, shared clock gives synchronous channels, simple wiring	Fixed internal filtering, restricted clock range, less control

The criteria that matter for direction finding are:

- **Frequency response:** flat across the band of interest, with a resonance peak near the top of the range in many MEMS parts, and the port and housing alter the response above a few kilohertz.
- **Directivity:** omnidirectional, as the steering vector assumes (Equation 2.86, and a port on one side of the board slightly shadows the other side at high frequency (Section 2.1.3).
- **Sensitivity and SNR:** the signal level and the noise floor limit the usable range (Section 2.2.2), and the SNR of a MEMS part sets how far a weak source can be heard.
- **Inter-channel matching:** gain and phase mismatch becomes angle error (Equation 2.42, and a MEMS lot from one reel is generally better matched than electrets [62].

An I2S MEMS microphone such as the SPH0645LM4H [62] puts the converter in the microphone, which removes the analogue front end and its mismatch, and any two microphones on a shared bit clock and word-select line are sampled by the same clocks. Read its datasheet for the bit alignment and clock range, because some hosts need care with its data timing.

2.7.2 Analogue front end

An analogue microphone needs a chain of bias, preamplifier, anti-alias filter and converter. Its purposes follow from the earlier theory:

- **Preamplification and gain staging.** The gain should bring the largest expected signal to just below the converter's full scale, because a signal that uses fewer bits loses 6 dB of SNR per bit

(Equation 2.28, while too much gain clips. The noise should be dominated by the microphone and not the preamplifier.

- **Noise.** The independent noise sources add in power, so the input-referred noise is

$$v_{n,\text{tot}}^2 = v_{n,\text{mic}}^2 + v_{n,\text{amp}}^2 + \frac{v_q^2}{G^2} \tag{2.116}$$

with v_q the quantisation noise from Equation 2.27 and G the gain. A good design makes the quantisation term negligible.

- **Anti-alias filter.** It must pass up to f_{max} and attenuate by roughly the converter's dynamic range at $f_s - f_{max}$ (Section 2.2.1), or the converter oversamples to relax it. Its phase response matters here. Two channels with different filter components have different group delays, and a delay difference acts as a timing error (Section 2.9). All channels should use identical topology and tight-tolerance parts.

A digital MEMS microphone avoids most of this, because the equivalent filtering is built into the part.

2.7.3 Acquisition options

The captured signals must be sampled at a common clock and transferred to a laptop (brief, Section 1). Table 2.14 compares the three options.

Table 2.14: Acquisition options

Option	Synchronisation	Channels	Cost	Flexibility	Latency
Multichannel audio interface (USB)	Good, one clock	Fixed, commonly 2–8	Higher	Low, hardware fixed, but drivers and software are ready	Set by driver buffers
Microcontroller	Good with shared clock lines	Set by interfaces and pins	Low	High, firmware is user-defined	Set by buffer sizes
FPGA	Excellent, deterministic	Very high	Highest	Highest, but longest development	Lowest

The brief allows a sound card, a microcontroller or an FPGA, according to which direction the student takes. A microcontroller with digital microphones is the lowest-cost route and gives full control of the sampling chain and timing, which suits a hardware-focused project. The data rate is

$$R = M f_s B \tag{2.117}$$

with B bits per sample, so three channels at 48 kHz and 24 bits need 3.456 Mbit/s, which is well within USB.

2.7.4 Digital audio interfaces

I2S

It uses a bit clock (BCLK), a word-select or frame clock (WS or LRCLK) and one serial data line, which carries two channels (left and right) per frame [63]. The bit clock rate is

$$f_{\text{BCLK}} = f_s \times N_{\text{slots}} \times B_{\text{slot}} \tag{2.118}$$

for example 3.072 MHz for $f_s = 48$ kHz, two slots and 32 bits per slot. Time-division multiplexing (TDM) extends the same idea to more slots per frame on one line. A three-microphone array therefore needs a second data line or a second port, which depends on the microcontroller (Section 2.7.5).

PDM

A PDM microphone outputs a 1-bit oversampled stream at a clock typically of about 1 to 3 MHz. A decimation filter converts it to PCM at f_s, so the clock is $f_{\mathrm{PDM}} = \mathrm{OSR} \cdot f_s$, with an oversampling ratio OSR of typically 64. This is the oversampling and noise-shaping trade-off of Section 2.2.1 implemented in the digital domain.

Synchronisation

Microphones that share the same bit clock and word-select signals are sampled by the same clocks, and this is the principal reason to prefer digital microphones for an array. The remaining issues are:

- *Offset between the two channels of one data line.* Check in the datasheet whether the left and right channels are sampled at the same instant or half a frame apart. A half-frame offset is $1/(2f_s) = 10.4\ \mu$s at 48 kHz, which is 3.6 mm of path and a systematic angle bias (Equation 2.126, but it is fixed, so it can be measured and corrected.
- *Jitter.* Random timing error in the sampling clock adds noise, with an SNR limit for a signal at frequency f and rms jitter σ_j of [64]

$$\mathrm{SNR}_{\mathrm{jitter}} = -20\log_{10}\left(2\pi f\,\sigma_j\right)\ \mathrm{dB} \tag{2.119}$$

For instance, $\sigma_j = 1$ ns at 10 kHz gives about 84 dB, which is negligible against the microphone noise. Jitter matters more for timing, since a delay error of σ_j gives an angle error through Equation 2.56, but it averages out.

- *Drift between separate clocks.* Channels from separate devices have a relative frequency offset of a few ppm, which grows into a delay error of about 1 μs per second per ppm. This is why one clock must drive all channels.

2.7.5 Microcontroller platforms

A suitable platform needs a high-rate digital audio port with several data lines, DMA to move samples without the CPU, enough RAM for buffers, and fast host connectivity. Table 2.15 compares typical candidates [65].

Table 2.15: Microcontroller platforms (typical values; see the manufacturers' documentation)

Board	Core	Audio interface	USB to host	Comment
Teensy 3.2 / 3.6	Cortex-M4, 72 / 180 MHz	One I2S port	Full or high speed	Mature audio library, limited RAM
Teensy 4.0 / 4.1	Cortex-M7, 600 MHz, about 1 MB RAM	Several serial audio ports with DMA	High speed, 480 Mbit/s	Fast, audio library support, a good fit for several I2S microphones
STM32 (F4, H7)	Cortex-M4 / M7	I2S and SAI with multiple data lines	Full or high speed	Flexible, more setup effort
ESP32	Dual-core Xtensa	I2S, PDM	Serial or wireless	Wireless adds latency and variability
RP2040	Dual Cortex-M0+	Programmable I/O can implement I2S	Full speed	Low cost, timing is custom

DMA moves each block of samples from the audio port into RAM while the CPU does other work. Double buffering (a "ping-pong" pair) lets one buffer fill while the other is read. The block length adds acquisition latency:

$$T_{\text{acq}} = \frac{B_{\text{blk}}}{f_s} \tag{2.120}$$

so for example a 128-sample block at 44.1 kHz adds about 2.9 ms. Dropped blocks appear as discontinuities in the signal, so each transferred block should carry a counter.

2.7.6 Data transfer to a host

USB serial

On boards with a native USB port the serial interface runs at the USB rate, not the nominal baud rate, so the stream of Equation 2.117 is comfortably carried. The firmware sends framed packets, each with a header, a sequence number and the interleaved samples, and the host checks the sequence to detect loss. Any number of channels can be sent.

USB audio

A class-compliant audio device needs no custom software on the host, but the channel count is limited by the device class implementation, and the standard microcontroller libraries often provide only stereo. For three or more channels, custom serial framing is the simpler route.

Buffering and latency

The delay from sound to display is the sum of the stages:

$$T_{\text{total}} = T_{\text{acq}} + T_{\text{USB}} + T_{\text{host buffer}} + T_{\text{proc}} + T_{\text{display}} \tag{2.121}$$

The frame length used for the DFT (Section 2.2.3) adds to this, because a result is only available once a frame has been collected. A "real-time" display means the update rate and T_{total} are short compared with changes in the scene, so a few frames per second with a latency under a few hundred milliseconds is adequate for a stationary source. In Julia, serial reading libraries (such as LibSerialPort.jl) can supply the host side, and reading should run in a separate task from the processing so that buffers never overflow.

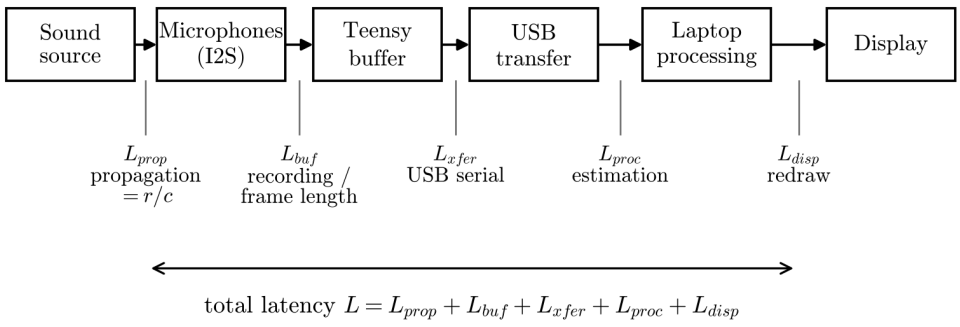


Figure 2.35: Data path from the microphones to the display, with the latency terms marked.

2.7.7 Existing commercial and research arrays

Commercial arrays provide benchmarks for geometry, channel count and interface (Table 2.16).

Table 2.16: Representative arrays (check current specifications)

Array	**Microphones and layout**	**Interface**	**Relevance**
ReSpeaker USB mic array v2.0	4 MEMS, circular	USB audio, onboard DSP with built-in DOA	Reference for a small circular array and a ready-made DOA to compare against
ReSpeaker 4-mic and 6-mic circular arrays	4 or 6 MEMS, circular	Analogue to codec, or I2S, with a host board	Low-cost multichannel capture with open hardware
miniDSP UMA-8	7 MEMS, circular plus centre	USB audio	Compact array with a well-defined geometry
MATRIX Voice	8 MEMS (PDM), circular	FPGA plus ESP32	FPGA-based acquisition example
Eigenmike (32 mics)	Spherical	Professional interface	Research benchmark for high-order spatial processing

Many commercial arrays run direction finding in their own firmware, which limits control of the algorithm, the sampling chain and the data. A custom capture system, as here, gives access to the raw channels, which the analysis of Section 2.9 needs. A commercial array can still serve as an independent reference during evaluation.

2.8 Simulation of acoustic arrays

A simulator generates the microphone signals for a known geometry, source and noise, so that algorithms can be developed and tested against known ground truth before any hardware exists (brief, Section 1).

Free-field model

Each channel is a delayed and attenuated copy of the source plus noise (Equation 2.60. For a point source at range r_m from microphone m, pressure spreads as $1/r$ (Equation 2.7, so

$$x_m[n] = a_m\, s\big[n - D_m\big] + w_m[n], \qquad a_m = \frac{r_1}{r_m}, \quad D_m = \frac{f_s\, r_m}{c} - D_{\text{ref}} \tag{2.122}$$

Near-field range effects follow automatically if the true distances r_m are used instead of the plane-wave delay (Equation 2.64.

Fractional delay

The delay D_m is not an integer, and rounding it to a whole sample would limit the simulation to the same angle steps as Equation 2.78. Fractional delays are produced by a windowed-sinc interpolation filter [67]:

$$h_D[n] = \operatorname{sinc}(n - D)\, w(n - D), \qquad \operatorname{sinc}(x) = \frac{\sin \pi x}{\pi x} \tag{2.123}$$

or by applying the phase ramp of Equation 2.37 in the frequency domain, or with Lagrange or Thiran filters [67]. The windowed sinc is accurate if the filter is long enough and the signal is band-limited below $f_s/2$.

Noise

White noise at a chosen SNR (Section 2.2.6) has variance

$$\sigma_w^2 = \frac{P_s}{10^{\text{SNR}/10}} \tag{2.124}$$

with P_s the signal power at the microphone. It is drawn independently per channel for sensor noise, and coloured noise is made by filtering white noise with a shaping filter. Diffuse ambient noise needs correlated channels with the coherence of Equation 2.51.

Reverberation

The image-source method models a rectangular room by replacing each wall reflection with a mirrored source behind the wall [35]. The room impulse response from a source to a microphone is a sum over images i of delayed, attenuated impulses,

$$h(t) = \sum_i \frac{\beta_i}{4\pi r_i}\, \delta\Big(t - \frac{r_i}{c}\Big) \tag{2.125}$$

where r_i is the distance from image i to the microphone and β_i is the product of the wall reflection coefficients along the path (Equation 2.11. The reverberation time can be estimated from Sabine's formula $T_{60} = 0.161\, V/A$, with V the room volume in m^3 and A the total absorption in m^2 [34]. Existing packages implement the method [68], and the model is a free-field approximation, since it ignores diffraction and scattering.

Role of simulation

The simulator verifies each algorithm against known angles, runs Monte Carlo trials across SNR, geometry and reverberation, and allows direct comparison with the Cramér-Rao bound (Figure 2.28). It also

exposes the sampling effects of Sections 2.2.1 to 2.2.2 and the error sources of Section 2.9, one at a time. Its limit is that it contains only the effects built into it, so agreement with measurement must be checked (Section 2.9).

Julia

The brief requires the simulation and signal processing in Julia, a language designed for numerical computing that compiles to fast machine code while keeping a high-level syntax [69]. Its standard linear algebra, together with packages for FFTs and signal processing, covers all the methods above, and fixed random seeds make Monte Carlo experiments reproducible.

2.9 Sources of error and performance evaluation

2.9.1 Error sources

Each error source enters the angle estimate through the delay (Equation 2.56, and the pair relation $\delta\theta \approx c\,\delta\tau/(d|\sin\theta|)$ gives their size.

Table 2.17: Error sources, effect and mitigation

Source	**Effect on the estimate**	**Size (for $d = 0.1$ m)**	**Mitigation**
Fixed channel timing offset (e.g. half-frame offset)	Bias	10.4 μs gives about 2.1° at broadside	Measure and subtract (Section 2.9.2)
Clock jitter	Noise, averages out	Negligible at ns level (Equation 2.119	Average, use a stable clock
Separate clocks (drift)	Growing bias	About 1 μs per second per ppm	One clock for all channels
Microphone phase mismatch	Bias	2° at 1 kHz gives about 1.1° (Section 2.2.4)	Calibrate per frequency
Microphone gain mismatch	Hurts MVDR and beamformers, little effect on PHAT	—	Calibrate gain, use PHAT
Position error	Bias	1 mm gives up to 0.57° (Section 2.5.4)	Careful assembly, calibrate
Speed of sound	Bias	1.8 % gives about 1° at 45° (Section 2.5.4)	Measure temperature, use Equation 2.4
Reverberation	Extra correlation peaks, outliers	Depends on room	PHAT, SRP, outlier rejection (Equation 2.105
Low SNR	Variance and threshold failure	Equations 2.55 and 2.85	Longer frames, better microphones
Quantisation	Noise floor	$6.02N + 1.76$ dB (Equation 2.28	Adequate gain
Near-field source	Model error	Equation 2.90	Keep $r > 2D^2/\lambda$

A fixed timing offset b between channels gives an angle bias of

$$\delta\theta \approx \frac{c\,b}{d\,|\sin\theta|} \tag{2.126}$$

so the half-frame offset above gives $346.47 \times 10.4 \times 10^{-6}/0.1 \approx 0.036$ rad, which is about 2.1°. Systematic errors do not average out, so they are reduced by calibration and not by more data (Section 2.2.7).

2.9.2 Calibration

- **Geometry.** Measure the microphone positions as precisely as possible and use them in the steering vector. A coordinate-measuring jig or a printed mount holds them to a fraction of a millimetre.
- **Speed of sound.** Read the air temperature and use Equation 2.4, or find c from the measured travel time over a known distance.
- **Timing offsets.** Place a source on the perpendicular bisector of a pair, so that the true delay is zero, and record the measured delay as the offset b for that pair. More generally, measure at several known angles and fit $\tau_{\text{meas}} = \tau_{\text{true}}(\theta; d, c) + b$ by nonlinear least squares, which also refines d and c.
- **Phase and gain.** Record a broadband signal from a common equidistant source and use the per-bin transfer function ratio $H_m[k]/H_1[k]$ to correct the other channels.
- **Self-calibration.** Methods that estimate array errors jointly with the directions exist [37], but they need more channels and are outside this project.

2.9.3 Evaluation methodology

1. **Ground truth.** Place a loudspeaker at a measured angle and range, at or beyond the far-field distance of Equation 2.90, and use a turntable or marked arc so that the angle uncertainty is well below the expected error (a third or less).
2. **Test conditions.** Sweep the angle over the full circle in fixed steps with several repeats at each, over the emitters of Table 2.1 (tones below the aliasing limit of Equation 2.91, impulses, speech, noise), in a controlled room, and repeat with deliberate reverberation or added noise to find the failure threshold.
3. **Metrics.** Use the wrapped error, the bias, standard deviation, RMSE and outlier rate of Section 2.4.4, per angle and per emitter. The uncertainty of the mean error from N trials is

$$\bar{e} \pm t_{N-1} \frac{s_e}{\sqrt{N}} \tag{2.127}$$

(about $1.96\, s_e/\sqrt{N}$ for large N), and it should accompany each reported value. Also report update rate and latency (Equation 2.121.

4. **Simulation against measurement.** Run the same algorithm on simulated data configured to match the experiment (geometry, SNR, room) and compare error against angle. Agreement validates the simulator and the model. A systematic difference points to an unmodelled error source from Table 2.17, which can then be added to the simulator.

5. **Benchmark.** Compare the measured RMSE with the Cramér-Rao bound (Section 2.4.4) to see how much room is left, and with a commercial array (Section 2.7.7) if one is available.

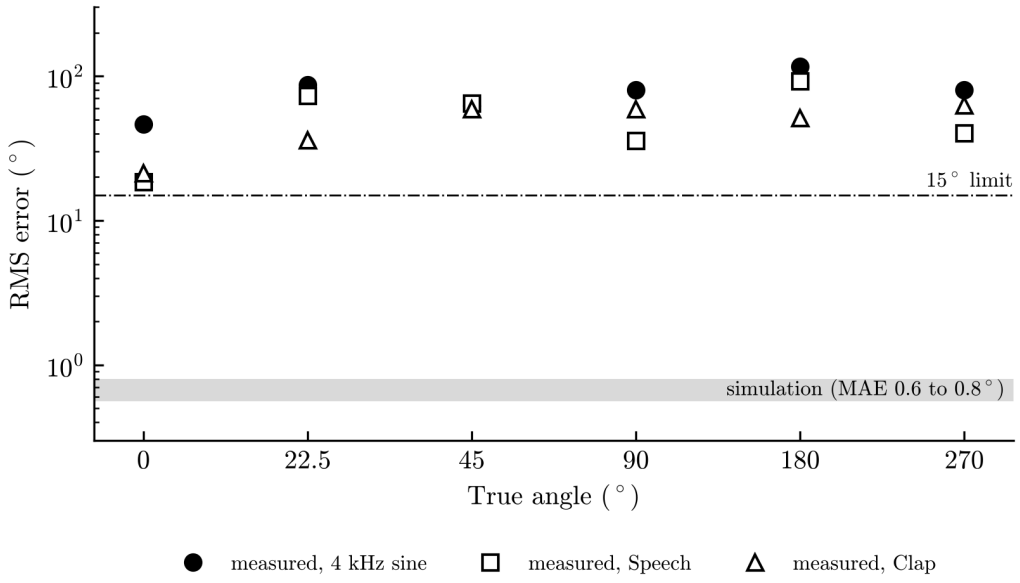


Figure 2.36: Measured error at each test angle on the real array (RMS over the three algorithms and both distances, from Table 5.3) against the simulated mean absolute error of 0.6 to 0.8 degrees (Table 5.1). The simulation is a single band because it is reported for all angles together.

2.10 Summary and implications for the design

What the literature shows.

- Sound localisation is commonly reduced to estimating direction (azimuth and elevation) without range, which allows cheaper and simpler systems [39, 40].
- The delay between microphones, together with the plane-wave and far-field assumptions, is what carries the direction, and sampling, quantisation and noise limit how precisely it can be measured (Sections 2.1 to 2.5).
- Time-delay methods are favoured for simple, low-cost, real-time systems [45]. Subspace and maximum-likelihood methods give better resolution and handle several sources, but need more computation and a calibrated array [37], and their published results are for far larger arrays than three microphones.
- Reported systems mostly estimate azimuth only, perform worse in real environments with noise and reverberation, and handle several sources poorly [41].
- Errors split into bias from systematic effects (geometry, timing, c, mismatch) and variance from noise, and need separate treatment (Sections 2.2.7 and 2.9).

What it points to.

Finding	Design decision
Unique azimuth over 360° needs three non-collinear microphones; the equilateral triangle gives isotropic accuracy	Three microphones in an equilateral triangle (Section 2.5.5)
Phase ambiguity limits spacing to $\lambda/2$ for tones; larger spacing reduces timing-to-angle error	Choose the spacing as a compromise, and state the tone limit (Section 2.5.3)
Synchronous digital MEMS microphones remove the analogue front end and channel clock skew	Digital I2S MEMS microphones on shared clocks (Section 2.7.4)
A microcontroller gives low cost and full control of sampling, and USB carries the data easily	Microcontroller acquisition and framed serial transfer to the laptop (Sections 2.7.5, 2.7.6)
Time-delay, subspace and model-matching methods are all established, and their simplest forms are easy to implement and suit three microphones	Three methods compared on the same data: Trig + FFT, MUSIC and LUT + cost function; GCC-PHAT and SRP-PHAT remain alternatives (Section 2.6.9)
Sub-sample interpolation is needed for fine angles at small spacing	Upsampling plus parabolic refinement (Section 2.6.1)
Systematic errors dominate and do not average out	Calibrate geometry, timing offsets and phase (Section 2.9.2)

Trade-off and gap

Published work mainly uses larger arrays, one emitter type or simulation alone, whereas this project builds a three-microphone, microcontroller-based system and evaluates one set of algorithms across tones, impulses and speech on both simulated and real data. The central trade-off is between cost and simplicity (three microphones, one source, closed-form estimation) and the accuracy and robustness that more microphones and heavier algorithms provide.

Bridge

Chapter 3 turns these findings into a design: the array geometry and spacing from Section 2.5, the hardware chain from Section 2.7, the algorithms from Section 2.6, and a simulator built as in Section 2.8, with the evaluation plan of Section 2.9.

Chapter 3

System Design

This chapter documents the design of the acoustic direction finding system. For each subsystem it lists the options that were available, states the selection criteria, records the decision and justifies it using the theory reviewed in Chapter 2. The individual choices are then combined into the complete system, and the chapter closes by tracing each requirement to the design element that meets it.

3.1 Requirements and Specifications Analysis

The project brief and the terms of reference, set by the supervisor, Associate Prof. A. J. Wilkinson, give the user requirements shown in Table 3.1. These requirements informed the derivation of the functional requirements (Table 3.2), the system specifications (Table 3.3) and the acceptance test procedures (Table 3.4). The specifications define the required operational and performance criteria of the system, while the acceptance test procedures verify compliance with these criteria.

Table 3.1: User requirements of the acoustic direction finding system

ID	Requirement
UREQ01	Capture sound using a minimum of three microphones arranged in a suitable geometry.
UREQ02	Amplify, sample and transfer the captured signals to a laptop using a microcontroller-based acquisition system.
UREQ03	Display the received signals and the estimated direction of arrival of the source in real time.
UREQ04	Estimate the direction of arrival across a range of audio frequencies.
UREQ05	Provide a simulator, written in Julia, that models the array geometry, the emitting source and the channel noise.
UREQ06	Perform all simulation and signal processing in Julia, using open-source tools where practicable.
UREQ07	Evaluate performance for continuous tones, impulsive sounds and human speech using both simulated and real recorded data.

Table 3.2: Functional requirements of the acoustic direction finding system

ID	Requirement
FREQ01	Acquire all microphone channels simultaneously from a common clock.
FREQ02	Digitise and buffer the samples, and transmit them to the host in framed packets so that lost data can be detected.
FREQ03	Estimate the azimuth of a single source for each processed frame from the inter-microphone signal differences.
FREQ04	Display the received signals and the estimated direction, updating continuously.
FREQ05	Generate simulated multichannel data for a given geometry, source type, direction and signal-to-noise ratio.
FREQ06	Process recorded and simulated data with the same algorithms.
FREQ07	Record raw multichannel data to file with identifying metadata (distance, angle and signal type).
FREQ08	Compute error statistics of the direction estimates for each type of emitter.

Table 3.3: Specifications for the acoustic direction finding system

ID	Specification	Target value
S01	Microphone array	Three or more microphones arranged in a suitable geometry.
S02	Operating frequency range	Sound in the audible frequency range.
S03	Channel synchronisation	All microphone channels sampled simultaneously.
S04	Sampling	Sampling rate high enough to capture the operating frequency range without aliasing.
S05	Data transfer	Captured data transferred to the laptop without loss.
S06	Direction estimate	Direction of arrival of a single source estimated and displayed in the horizontal plane.
S07	Accuracy	Direction error of at most 15 degrees: 10 degrees for the system and 5 degrees for the uncertainty of the known angle.
S08	Display update	Display updated at a rate suitable for real-time viewing, at least once every 1 min after receiving a trigger.
S09	Source types	Continuous tones, impulsive sounds and human speech supported.
S10	Simulator	Models the array geometry, source and channel noise, and produces multichannel data sets.
S11	Software	All simulation and signal processing done in Julia, using open-source tools where practicable.
S12	Data records	Recordings stored with the information needed to identify the test conditions.

Table 3.4: Acceptance test procedures for the acoustic direction finding system

ATP ID	Traceability	Test procedure	Pass criterion
ATP01	UREQ01, S01	Inspect the assembled array and count the microphones and their positions.	Three or more microphones, positions recorded.
ATP02	UREQ02, FREQ01, S03	Apply a common test signal to all microphones and compare the recorded channels.	Channels start together, with no relative offset beyond [1] sample.
ATP03	UREQ02, FREQ02, S05	Transfer a recording to the laptop and check it for lost or repeated data.	No data lost or repeated.
ATP04	UREQ04, S02, S04	Play tones across the audible range and examine the recorded spectrum.	Tones appear at the correct frequencies, with no aliasing.
ATP05	UREQ03, FREQ03, S06, S07	Place a source at known angles and compare the estimated direction with the true direction.	Error within 15 degrees (S07).
ATP06	UREQ07, FREQ08, S09	Repeat ATP05 with tones, impulsive sounds and speech.	Error statistics computed for each source type, within the S07 limit.
ATP07	UREQ03, FREQ04, S08	Observe the display while a source is active.	Updates at least as often as S08 requires.
ATP08	UREQ05, FREQ05, S10	Run the simulator for a chosen geometry, source and noise level.	Multichannel data produced that match the chosen settings.
ATP09	UREQ06, FREQ06, S11	Process a recorded and a simulated data set with the same code.	The same code runs on both without changes.
ATP10	FREQ07, S12	Inspect the saved recordings.	Each recording identifies its test conditions.

3.2 Design Approach and Selection Criteria

The acoustic direction-finding system is designed around the requirements, specifications and acceptance test procedures in the previous section, so that the finished system can be verified against them. These, together with the theory reviewed in Chapter 2, guide how system performance is tested.

To achieve optimum performance, the system is divided into individual subsystems. For each subsystem, the state-of-the-art options are analysed as design choices, and the best option is selected with a stated justification. The selection criteria are cost, microphone capability, channel synchronisation, timing resolution, real-time capability, algorithm accuracy and computational speed. Each criterion follows from the requirements and specifications.

Every subsystem selection follows the same pattern: options considered, criteria, decision and justification.

3.3 Array Geometry and Spacing

Chapter 2 identified linear, circular, equilateral-triangular, planar, non-coplanar (three-dimensional) and ad hoc arrays. Each accommodates a particular number of microphones, and each has its own trade-offs, summarised in Table 3.5.

Table 3.5: Array geometry options

Geometry	**Azimuth coverage**	**Uniformity of accuracy**	**Min. mics**	**Remark**
Linear	180 degrees only (front-back ambiguity)	Poor near end-fire	2	Simple, but ambiguous over the full circle
Equilateral triangle	Full 360 degrees	Fairly uniform	3	Smallest planar array that is unambiguous
Circular (4 or more)	Full 360 degrees	Uniform	4	Needs more channels than the brief requires
Three-dimensional	Azimuth and elevation	Depends on layout	4	Elevation is not required
Ad hoc	Depends on layout	Poor	3	Geometry is irregular or unknown

Criteria. A unique azimuth over the full circle, similar accuracy at all angles, the spatial-aliasing limit, physical size, and a minimum of three microphones (UREQ01, S01).

Decision. An equilateral triangle of three microphones with a side length of 17 mm, shown in Figure 3.1.

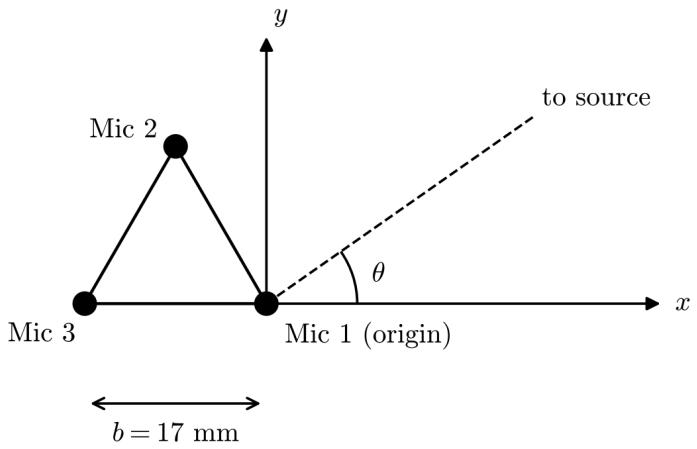


Figure 3.1: Geometry of the three-microphone array

Justification and trade-off. Only azimuth is measured and three microphones are available, so a planar array fits. A triangle gives three pair baselines at 60 degrees to each other, so a source at any bearing is seen by at least two non-collinear pairs. This removes the front-back ambiguity of a linear array and makes accuracy more uniform with angle.

The spacing sets the aliasing limit. For one pair, spatial aliasing begins at f = c/(2d) = 343/(2 x 0.017), about 10 088 Hz, so the usable band ends there. The maximum delay between a pair is d/c, about 49.6 microseconds, which is only about 2.2 samples at the sampling rate, so the delay must be estimated to a fraction of a sample for a useful angle.

The trade-off is low-frequency resolution. A small aperture gives small delays at low frequency, so low-frequency sources are localised less accurately. A larger spacing would help there but would lower the aliasing limit.

3.4 Microphone Selection

Three microphone types are commonly used in compact arrays, as reviewed in Chapter 2. Electret condenser microphones use a permanently charged diaphragm and give an analogue output. Analogue MEMS microphones give an analogue output from a silicon capsule. Digital MEMS microphones (I2S or PDM) integrate the capsule, amplifier and converter and give a digital output. Table 3.6 compares them.

Table 3.6: Microphone options

Option	**Interface and size**	**Sensitivity and noise**	**Channel matching and synchronisation**	**Cost**
Electret	Analogue; needs preamplifier and ADC; large (about 6–10 mm)	Good, varies between units	Poor, needs calibration; synchronisation depends on the ADC	Very low
Analogue MEMS	Analogue; needs ADC; very small	Consistent between units	Good; synchronisation depends on the ADC	Low
Digital MEMS (PDM/I2S)	Digital; needs PDM/I2S peripheral; very small	Consistent between units	Good; channels share one clock	Low

Criteria. Cost, size (which sets the array aperture), matching between channels, simultaneous sampling, and ease of integration with the microcontroller (S01 to S05).

Decision. Digital MEMS microphones with an I2S output, which have a built-in ADC and filtering. The SPH0645 is used; its datasheet is in Appendix D.

Justification and trade-off. Digital MEMS microphones are not the cheapest option, but they are easy to source and therefore easy to replace during testing. The ADC and filtering are built in, so no separate preamplifier, anti-aliasing filter or ADC has to be designed, which removes the noise and errors that a hand-built analogue front end can introduce. All channels can share one I2S clock, so they are sampled simultaneously (S03), and they connect directly to the microcontroller. Their sampling rate is sufficient for audible sound (S02, S04). Their small size makes them easy to mount and allows a small spacing, which raises the highest frequency the array can handle before spatial aliasing.

Consequence. The microcontroller must provide an I2S interface that can read all three channels from a common clock.

3.5 Acquisition Hardware and Digital Interface

Three kinds of acquisition platform were considered: a multichannel sound card, a microcontroller, and an FPGA (Table 3.7). The digital interface to the microphones can be I2S, PDM or TDM (Table 3.8).

Table 3.7: Acquisition platform options

Option	Raw channel access	Synchronisation	Cost and effort	Fit to brief
Multichannel sound card	Through the audio driver only	Good within the card	Moderate cost, little effort	Not microcontroller based
Microcontroller with I2S (Teensy 4.0)	Direct, in firmware	Shared I2S clock	Low cost, moderate effort	Meets UREQ02
FPGA	Direct, in hardware	Excellent	Higher cost and effort	Possible, but more than needed

Table 3.8: Digital interface options

Interface	How it works	Remark
I2S	Separate bit clock, word clock and data lines; two microphones can share a data line (left and right)	Widely supported by MEMS microphones and microcontrollers
PDM	One-bit stream at a high clock rate, decimated to audio samples	Decimation filtering must be done in the processor
TDM	Many channels time-multiplexed on one data line	More complexity than three channels need

Criteria. Simultaneous sampling, a common clock, channel count, cost, and the brief requirement for a microcontroller-based system (UREQ02, S03).

Decision. A Teensy 4.0 reading the three microphones over I2S. The datasheet and reference information are in Appendix D.

Justification and trade-off. The Teensy generates the I2S clocks itself, so all three microphones share one clock and are sampled simultaneously (FREQ01). The Audio library input object for four I2S channels reads the three channels directly: microphones 1 and 2 share one data pin (left and right) and microphone 3 uses a second pin. DMA moves the samples into memory in blocks without loading the processor. A sound card would hide the raw channels and clocks and conflicts with the brief, and an FPGA meets the requirement but with more development effort than the project needs.

The trade-off is memory. The recording is held in the Teensy's two RAM banks (about 794 kB in total), which limits one recording to 1034 blocks, or about 3 s.

3.6 Data Transfer to the Host

USB audio, USB serial, a wireless link and an SD card (Table 3.9).

Table 3.9: Data transfer options

Option	Raw channel access	Throughput	Reliability	Remark
USB audio	Through the audio driver; channels may be mixed or resampled	Adequate	Good	Raw data not guaranteed
USB serial	Full: raw samples of every channel	Adequate for about 265 kB/s	Good, with framing to detect loss	Supported natively by the Teensy 4.0
Wireless	Full	Limited	Packet loss possible	Adds latency and hardware
SD card	Full	Adequate	Good	Needs extra hardware and manual file handling

Criteria and decision. The criteria are raw access to every channel, enough throughput, and framing that allows dropped data to be detected (FREQ02, S05). USB serial was chosen, because the Teensy 4.0 supports it natively and it passes the raw samples of each channel to Julia. The raw data rate is three channels at about 44.1 kHz with two bytes per sample, about 265 kB/s, which USB serial carries easily.

Justification and trade-off. Each 128-sample block is sent as one 781-byte packet, shown in Figure 3.2: the marker START (5 bytes), a block counter (4 bytes), the total number of blocks (4 bytes) and the samples of the three microphones (768 bytes). Julia checks the marker and the counter of every packet and counts the blocks that are missing, so dropped data is detected. The trade-off is latency: the recording is sent only after it is complete, so a result appears after the recording, the transfer and the processing.

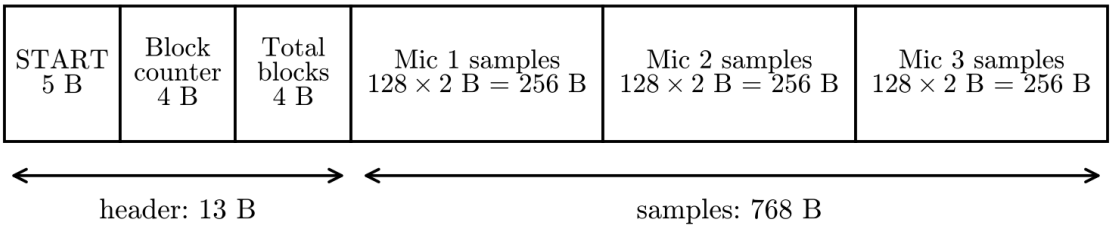


Figure 3.2: Format of one serial packet

3.7 Sampling Parameters

Table 3.10 lists the sampling parameters and the reasons for them. They link to the aliasing, quantisation, timing-resolution and time-frequency trade-off material in Chapter 2.

Table 3.10: Sampling parameters

Parameter	Value	Reason
Sampling rate	44 117.6 Hz	Fixed rate of the Teensy Audio library I2S and cannot be changed; its Nyquist limit of about 22 kHz is above the analysis band, so there is no temporal aliasing (S04)
Block length	128 samples (about 2.9 ms)	Audio library block; also the unit used to count dropped data
Bit depth	16 bits	Format delivered by the library; quantisation noise is well below the acoustic noise floor
Recording length	1034 blocks (about 3.0 s)	Limited by the available RAM
Trigger	200 counts from the block mean on any channel	Starts the recording at the onset of the sound; the triggering block is kept as the first block
Timing resolution	One sample, about 22.7 microseconds	Compared with a maximum pair delay of 49.6 microseconds, so sub-sample estimation is needed
Frame length	1024 samples (about 23 ms), 50% overlap	Compromise between frequency resolution and following a changing source; gives about 257 frames per recording

3.8 Signal Conditioning

The conditioning steps are listed in Table 3.11. They link to the noise and spatial-aliasing limits in Chapter 2.

Table 3.11: Signal conditioning steps

Step	Setting	Reason
Mean removal	Subtract the mean of each channel	Removes the offset of the microphone output
High-pass filter	2 kHz cut-off, applied in the frequency domain	Removes low-frequency noise, which dominates in a room and gives very small, unreliable delays for a small array
Analysis band	2 kHz to 10.088 kHz	The lower edge is the filter cut-off and the upper edge is the spatial-aliasing limit of the 17 mm spacing
Window	Hann, 1024 samples	Reduces spectral leakage
Overlap	50%	Recovers the energy that the window removes at the frame edges
Frame gate	Discard frames below 5% of the peak power	Rejects frames that contain no source

Trade-offs. The filter works on the whole recording, so it suits record-then-process but not streaming. The analysis band is also a subset of the audible range in S02, so S02 is met only between 2 kHz and 10.088 kHz.

3.9 Direction-of-Arrival Algorithm Selection

GCC-PHAT with least squares, SRP-PHAT, MVDR and MUSIC were reviewed in Chapter 2, together with delay-based closed-form and model-matching approaches. Table 3.12 compares them. Three methods were implemented: Trig + FFT, which estimates the time delay from the signal spectra and calculates the angle from the array geometry; MUSIC, a subspace method using the covariance of the

three channels; and LUT + cost function, which compares the measured phase differences with the values expected for each candidate angle and picks the angle with the lowest cost.

Table 3.12: Direction-of-arrival algorithm options

Method	**Principle**	**Strength reported in the literature**	**Limitation**	**Used**
GCC-PHAT with least squares	Delay from the phase-weighted cross-correlation of each pair; angle from least squares	Robust to reverberation	Delay must be resolved to a fraction of a sample at this spacing; needs interpolation	No
SRP-PHAT	Steered response power over a grid of angles	Robust, uses all pairs together	Higher computational cost	No
MVDR	Minimum-variance beamformer	Good resolution with many microphones	Needs a covariance inverse; loses degrees of freedom with three microphones	No
MUSIC	Orthogonality of the steering vector and the noise subspace	High resolution	Computationally heavier; needs a good covariance estimate	Yes
Trig + FFT	Delay from the phase of the spectra; angle by geometry	Simple and fast	Sensitive to noise and phase errors	Yes
LUT + cost function	Match measured phases to expected phases for each angle	Robust, resolves below one sample	Resolution limited by the angle grid	Yes

Criteria. Suitability for three microphones, wideband and tonal sources, simplicity, computational cost, real-time operation, and accuracy (S06, S07, S09).

Decision. Three algorithms are implemented and compared in the later chapters: Trig + FFT, MUSIC and LUT + cost function. The one that performs best is chosen in the conclusions, after the results (Chapters 5 and 6).

Justification and trade-off. The three methods were chosen for four reasons.

First, they are the simplest representatives of their kind. Most direction-finding methods use the same principle, the phase or delay differences between channels, and differ mainly in how those differences are turned into an angle. Trig + FFT does it in closed form, LUT + cost function by searching a model, and MUSIC by a subspace test. Comparing them therefore shows the effect of the estimator itself and not of a different front end, because all three share the same conditioning.

Second, they are easy to implement and run in real time. None needs the large covariance inversion of MVDR or the full steered-power search of SRP-PHAT, which cost more and gain little with only three microphones.

Third, they are well supported by the literature. MUSIC is reported to give high resolution, and model-matching of phase differences is closely related to maximum-likelihood estimation for a single source, which is reported to reach near-optimal accuracy at high signal-to-noise ratio. The closed-form delay method is the standard baseline against which others are compared. GCC-PHAT, the usual choice in reverberant rooms, is a small modification of the delay method, so its main benefit can be assessed through that method.

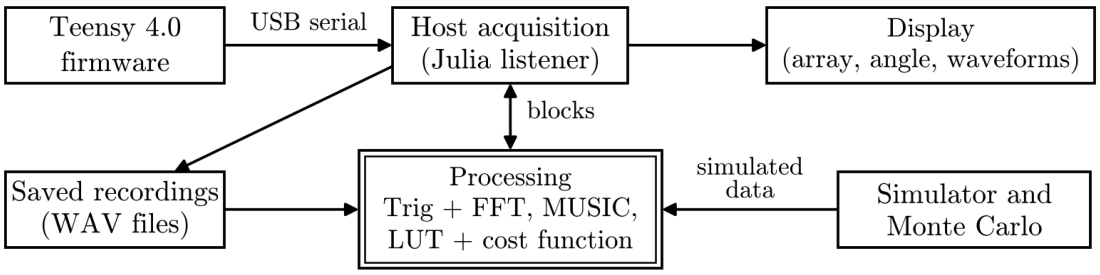
Fourth, they cover the spread of cost and complexity, from the cheapest to the heaviest, so the trade

between accuracy and speed can be shown. What is given up is the robustness to reverberation of GCC-PHAT and SRP-PHAT and the resolution of larger-array methods. The choice rests on the literature and on simplicity, not on the project's own results, which are reported in the results chapter.

3.10 Software Design

3.10.1 Software architecture

The software has four parts (pseudocode in Appendix A and Appendix B; full code in Appendix E): the Teensy firmware, the host acquisition and display program, the offline processing scripts and the simulator (Table 3.13). Data moves from the firmware to the host over USB, and the same processing code is used on recorded and simulated data (FREQ06). Figure 3.3 shows the architecture. Julia was chosen because the brief requires it (UREQ06), it is fast enough for numerical work, and it has open-source packages for serial communication, FFTs and plotting.



The same processing code runs on live, recorded and simulated data.

Figure 3.3: Software architecture

Table 3.13: Software components

Component	Language and libraries	Role
Teensy firmware	Arduino C++, Teensy Audio library	Reads the microphones, triggers, records to RAM and sends packets
Acquisition and display	Julia: LibSerialPort, GLMakie, FFTW, WAV	Receives packets, estimates the angle and draws the display
Offline processing	Julia: one script per algorithm	Runs Trig + FFT, MUSIC and LUT + cost function on saved recordings
Simulator and Monte Carlo	Julia	Generates multichannel data and runs repeated trials

3.10.2 Firmware

The firmware reads the three channels into three record queues. It waits until any channel moves more than 200 counts from the mean of its block, and keeps that block as the first block of the recording, so the onset of the sound is not lost. It then records 1034 blocks (about 3 s) into RAM, sends nothing

during the recording, and sends the finished recording as framed packets (Figure 3.4). A pause of 5 s lets the sound die away before the trigger is armed again.

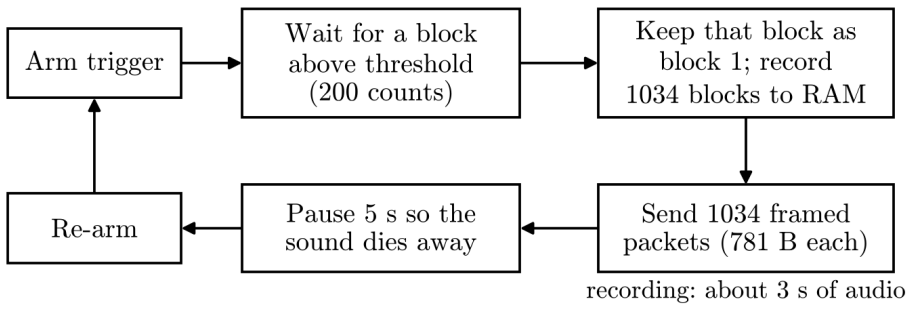


Figure 3.4: Firmware flow

3.10.3 Host acquisition

A listener in Julia reads the serial port in the background so the display stays responsive. It finds the START marker, reads the total number of blocks, and places every block in a matrix by its counter. Blocks that never arrive are counted as dropped and the number is shown on the display. Recordings can be saved as WAV files whose names encode the distance, angle, source type and frequency (FREQ07, S12).

3.10.4 Signal processing pipeline

Figure 3.5 shows the processing. The mean of each channel is removed and the recording is high-pass filtered at 2 kHz. It is cut into 1024-sample Hann-windowed frames with 50% overlap, and each frame is transformed with an FFT. For each frame the strongest bin of microphone 1 inside the analysis band is found, and frames below 5% of the peak power are discarded. The phase differences of microphones 2 and 3 relative to microphone 1 at that bin are passed to the algorithm, and the per-frame angles are combined with a circular mean, so angles near 0 and 360 degrees average correctly.

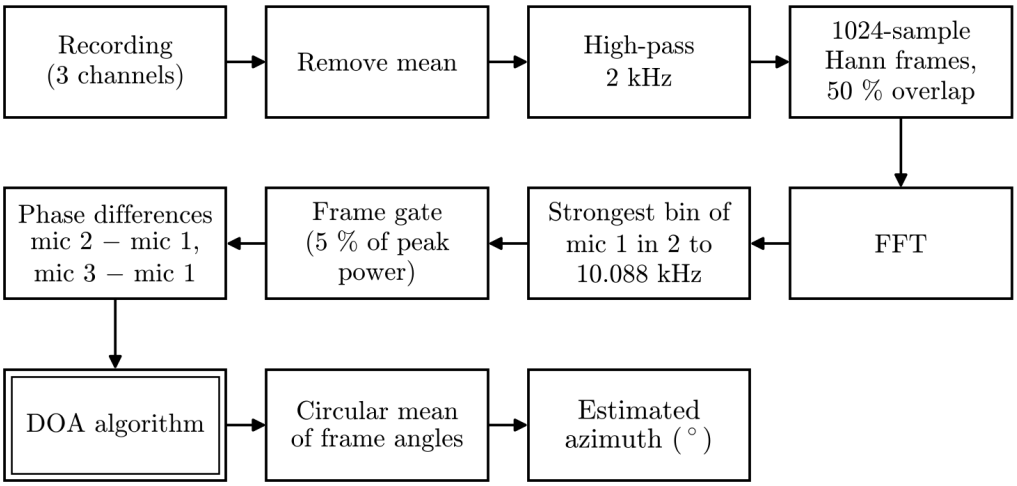


Figure 3.5: Signal processing pipeline

For LUT + cost function, the expected phase differences are computed for every candidate angle on a fine grid, and the angle with the lowest cost is selected, with phase wrapping handled in the cost. Trig + FFT converts the measured delays to an angle by geometry, and MUSIC uses the covariance of the three channels. Their flow diagrams are in Figure 3.6. Pseudocode for the three algorithms and the simulations is in Appendix A, pseudocode for the firmware and real-time program is in Appendix B, and the full code is in the repository listed in Appendix E.

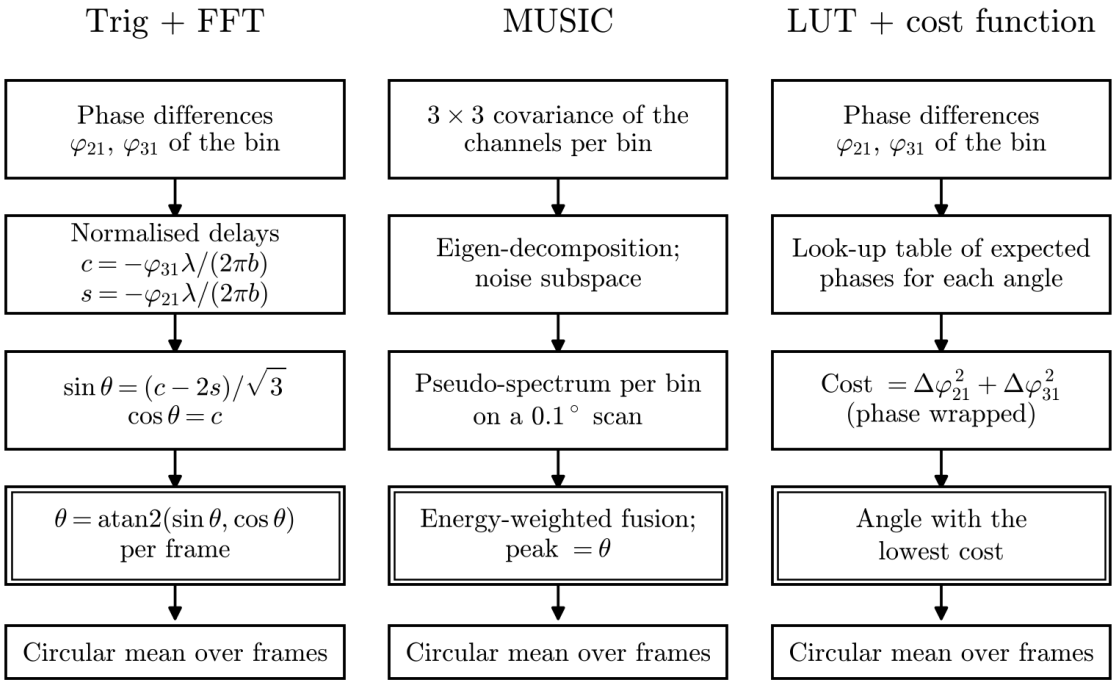


Figure 3.6: Flow diagrams of the three algorithms

3.10.5 Simulators and Monte Carlo framework

Two simulation tools support the work: a signal simulator that generates the microphone signals for a known source angle, and a Monte Carlo framework that perturbs real recordings. Pseudocode for both is in Appendix A.

Signal simulator

The source is a sum of tones at 4, 6 and 9.5 kHz, generated at a sampling rate of 44.1 kHz for 3 s. For a chosen azimuth theta, the delays follow from the array geometry: microphone 2 lags microphone 1 by b sin(30 degrees - theta)/c and microphone 3 by b cos(theta)/c, where b is the 17 mm side and c is 343 m/s. The delayed signals are formed directly from the tone, so fractional delays are exact. White Gaussian noise with a standard deviation of 0.01 (the tones have unit amplitude) is added independently to each channel to model the channel noise. The signals are then processed by the same algorithm as a recording, and the microphone layout and the estimated direction are plotted. One simulation script exists for each of the three algorithms.

Monte Carlo framework

The framework is written in Julia and run in a notebook. It is built around the three algorithm scripts and measures how each source of error changes the estimate. It starts from real recordings made at known angles and perturbs either the recording or the geometry assumed by the algorithm, so the effect of each error source is measured on the real signal and room. Figure 3.7 shows the structure.

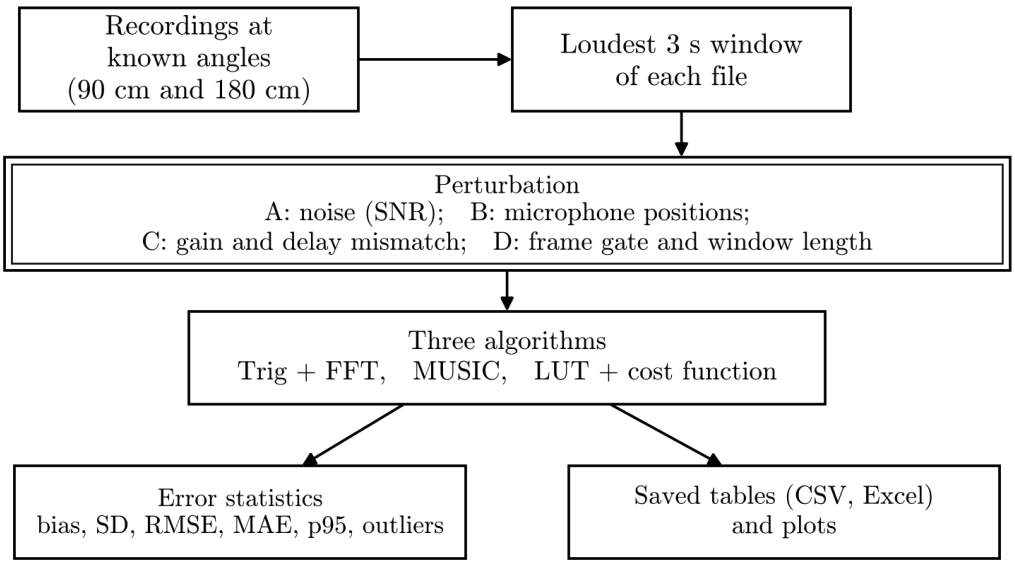


Figure 3.7: Structure of the Monte Carlo framework

All three algorithms are written in a general-geometry form that takes the microphone coordinates as an input, so the geometry can be perturbed. They share the front end of the real-time program (mean removal, 2 kHz high-pass filter, 1024-sample Hann frames with 50% overlap, analysis band of 2 to 10.088 kHz) and the same conventions: azimuth is counter-clockwise from the positive x axis, the delay

of microphone k relative to microphone 1 is (p1 - pk) . u / c, and the steering vector is built from those delays. The offline scripts use an oversampling factor of 1 for MUSIC and Trig + FFT and 2 for LUT + cost function, and the angle grid in the framework is 0.1 degrees.

The true angle of each recording is read from its file name, and the loudest 3 s window of each file is analysed. Table 3.14 lists the experiments.

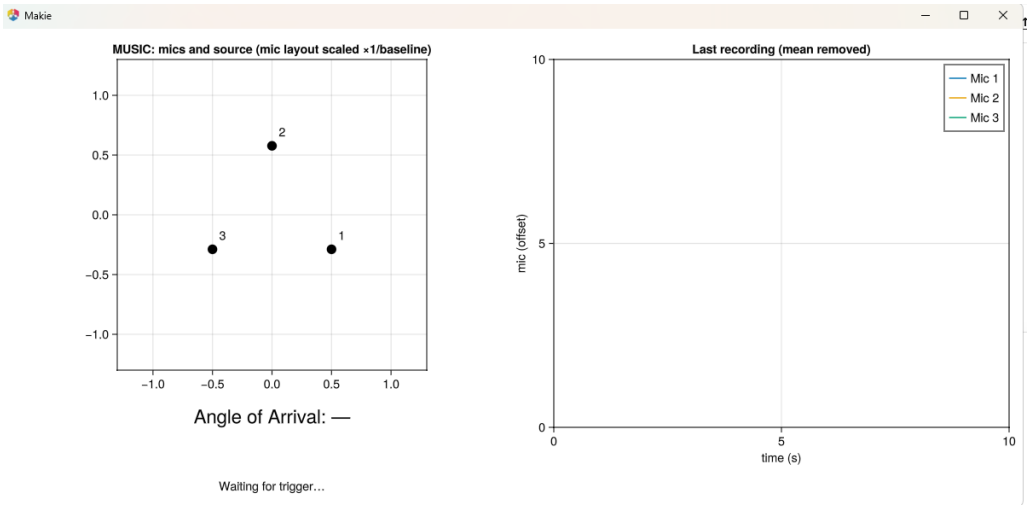
Table 3.14: Monte Carlo experiments

Exp.	Error source	How it is applied	Levels
A	Noise	White Gaussian noise added so that the in-band signal-to-noise ratio takes each level	40, 30, 20, 10, 5, 0 and -5 dB
B	Geometry	Each coordinate of the assumed microphone positions is perturbed with Gaussian noise; the recordings are untouched	Standard deviation 0.25, 0.5, 1 and 2 mm
C	Channel mismatch	A random gain error and a random timing error per channel, the delay applied as a phase ramp in the frequency domain so that it can be fractional; measured systematic offsets can be added	Gain 0.25 to 2 dB; delay 0.5 to 10 microseconds
D1	Frame-to-frame variation	The per-frame estimates that Trig + FFT and LUT + cost function average are collected, and the effect of discarding low-energy frames is measured	Energy gates of all, 30, 20 and 10 dB
D2	Observation length	Random energetic windows are cut from each recording and processed by all three algorithms	4096, 8192, 16384 and 32768 samples; 200 windows each

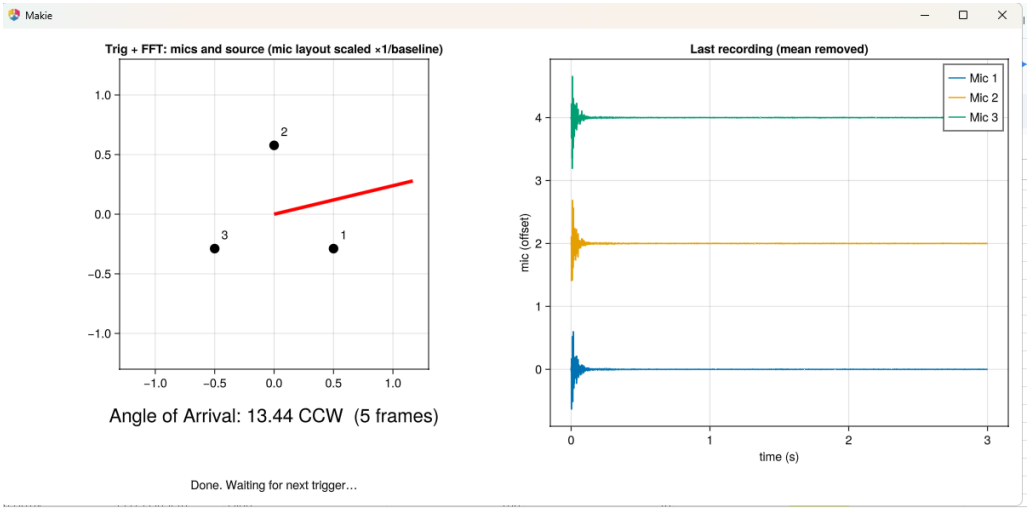
Each trial gives an estimate, the error against the true angle (wrapped to plus or minus 180 degrees) and the shift from the algorithm's own unperturbed estimate. The shift shows the sensitivity to the error source and the error shows the accuracy. For each level the program reports the bias, standard deviation, RMSE, mean absolute error, the 95th percentile of the absolute error, and the percentage of outliers beyond 15 degrees. An error budget compares the sources at one typical level each. The random generator is seeded, so a run can be repeated exactly. The program writes the raw trials and the summaries as CSV files, the summaries as one Excel workbook, and the plots as images, and it is first run with a small number of trials to check the run time.

3.10.6 Real-time display

The display shows the microphone layout with a line pointing in the estimated direction, the angle in text, the waveforms of the last recording and a status line (Figure 3.8). It updates once per recording, after the processing. The time from sound to displayed angle is the recording, transfer and processing time, budgeted in the next section.



(a)



(b)

Figure 3.8: Real-time display: (a) waiting for a trigger; (b) after a hand clap at 0 degrees, with the array layout and estimated direction, the angle readout, the three recorded waveforms and the status line.

3.10.7 Software verification

The software is verified against a known angle. The array and its electronics are held in an enclosure with angle markings and mounted on a turntable, so a rotation of the turntable sets the angle between the array and the source to a value that is known independently of the software (the fixture is described in the Final System Design section, Figure 3.12). Each recording is saved with its known angle in the file name, and the estimate of each algorithm is compared with that angle.

The markings of the turntable are in steps of 22.5 degrees, and any angle between the markings can be set by reading the position against the scale, so the test angle is not limited to the marked values. The uncertainty of the known angle is taken as plus or minus 5 degrees, from reading the scale and from aligning the zeros of the enclosure and the turntable. This uncertainty is part of the accuracy requirement (S07): the error of the system is allowed to be 10 degrees, and with the 5 degrees of uncertainty in the reference, a measured error of up to 15 degrees is accepted.

Table 3.15 lists the checks. They separate errors in the software from errors in the hardware and the room, because the same recordings and the same known angles are used for all three algorithms.

Table 3.15: Software verification checks

Check	**Method**	**Expected result**
Angle convention	Record at angles in each quadrant and check the sign and the direction of rotation of the estimate	Estimate follows the turntable counter-clockwise from the positive x axis
Known-angle accuracy	Compare the unperturbed estimate of each algorithm with the angle read from the markings, at each recorded angle	Error within the limit of S07
Agreement between algorithms	Compare the three estimates for the same recording	Estimates agree to within their expected error
Synthetic signal	Run the signal simulator with a known angle and compare the estimate with it	Estimate equals the simulated angle to within the noise; for 22.5 degrees the Trig + FFT simulation returned 22.499 degrees
Perturbation baseline	Run the Monte Carlo framework with no perturbation	Reproduces the unperturbed estimates
Repeatability	Repeat a run with the same seed, and repeat the recording at the same angle	Identical results for the same seed; scatter of the recordings quantified
Data integrity	Check the block counter of every recording	No dropped blocks

3.11 Final System Design

This section brings the individual design choices of the previous sections together into the complete system. Figure 3.9 shows the system from the sound source to the display, and Table 3.16 lists the final design parameters.

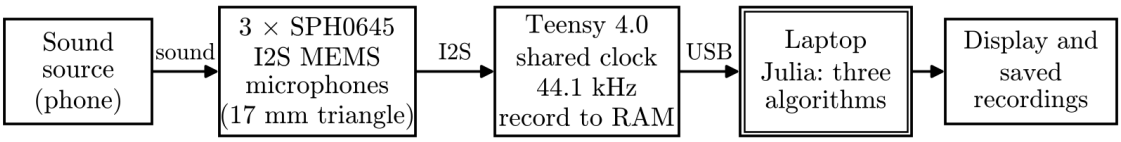


Figure 3.9: Block diagram of the complete system

3.11.1 Physical design

The three microphones, the Teensy 4.0 and the wiring are held in a single enclosure. The microphones are fixed at the corners of a 17 mm equilateral triangle, and the enclosure carries angle markings in steps of 45 degrees so that the orientation of the array can be read directly. Figure 3.10 shows the CAD model of the enclosure and Figure 3.11 shows the finished system. Further views of the model (front, back, top, side and isometric) are in Appendix C.

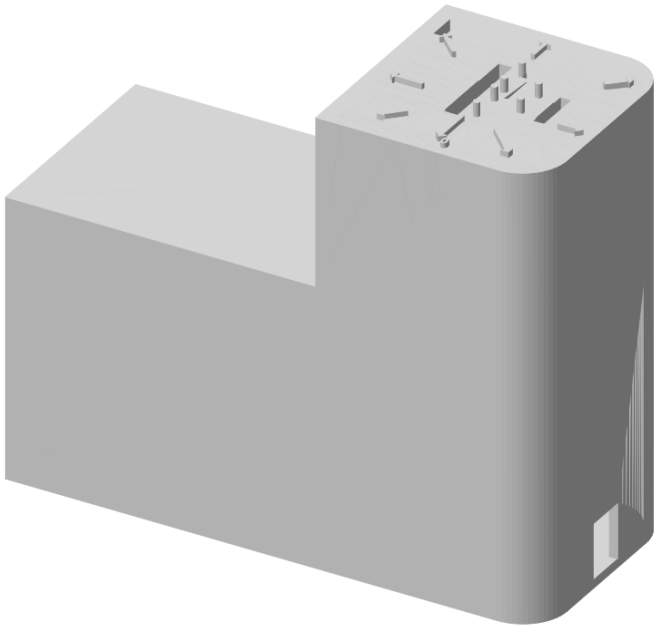
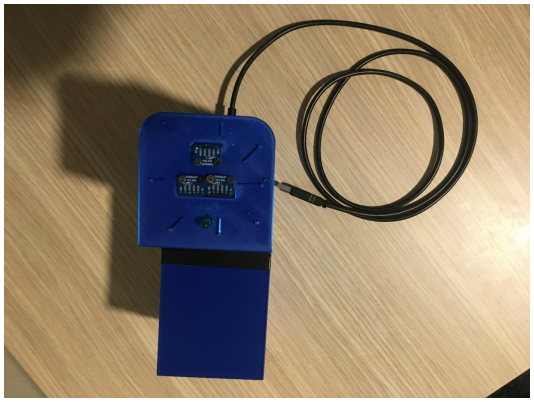
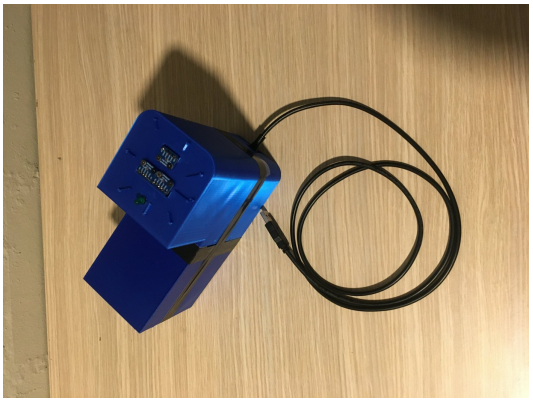


Figure 3.10: CAD model of the enclosure



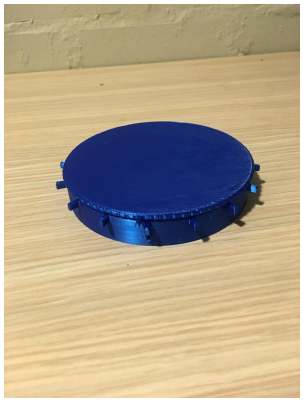
(a)



(b)



(c)



(d)

Figure 3.11: The completed system: (a) enclosure with the three microphones, angle markings, status LED and USB cable, seen from above; (b) the same enclosure from an angle; (c) the two parts of the turntable; (d) the turntable assembled.

3.11.2 Test fixture

The known angle is set with a turntable on which the enclosure is mounted (Figure 3.12). The turntable model was obtained from an online repository of 3D-printable models[70] and adapted for this project: angle markings in steps of 22.5 degrees were added to it in the CAD software. Any angle can be set, because the position is read against the scale between the markings. The enclosure carries its own markings in steps of 45 degrees. Before a test, the zero of the enclosure is aligned with the zero of the turntable. The top of the turntable then carries the enclosure, so the system and the top rotate together as one body, and the source stays fixed. The array is turned to each test angle and the measurement is made at that angle. More detailed drawings of the turntable, in several views, are in Appendix C.

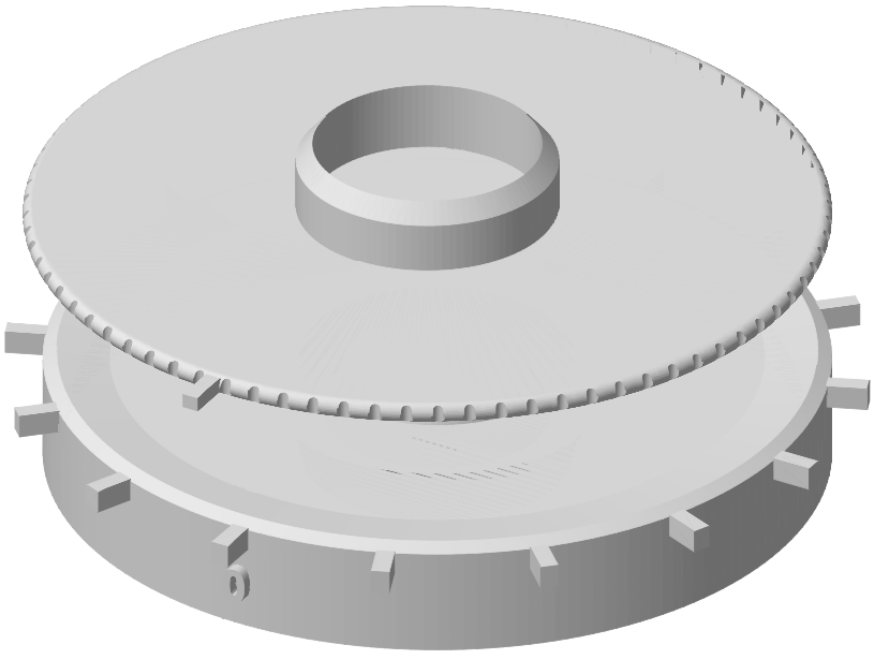


Figure 3.12: CAD model of the turntable, with the top plate lifted above the base

The source is placed at a fixed distance from the array, and the two distances used were 90 cm and 180 cm. The distance was measured with a measuring tape with 1 mm graduations, from the centre of the array to the speaker of the source, and its uncertainty is about plus or minus 5 mm, mostly from locating the two centres. The uncertainty of the zero alignment is included in the plus or minus 5 degrees of the known angle.

3.11.3 Operation of the complete system

A sound reaches the three microphones, and the Teensy samples them at 44 117.6 Hz from one I2S clock. When any channel exceeds the trigger level, it records about 3 s into RAM and then sends the recording to the laptop over USB serial in framed packets. The laptop checks the packets, conditions the signals, estimates the azimuth in each frame with the selected algorithm, combines the frames and shows the angle with the waveforms. The same processing is applied to recordings saved as WAV files and to the Monte Carlo experiments.

Table 3.16: Final design parameters

Item	**Final design**
Microphones	Three SPH0645 digital MEMS microphones with an I2S output (datasheet in Appendix D)
Array geometry	Equilateral triangle, 17 mm side, planar
Acquisition	Teensy 4.0, three channels from one I2S clock, 16-bit samples
Sampling	44 117.6 Hz; 128-sample blocks; 1034 blocks (about 3.0 s) per recording
Trigger	200 counts from the block mean on any channel; the triggering block is kept
Data transfer	USB serial, 781-byte packets with a block counter
Host software	Julia: LibSerialPort, FFTW, GLMakie, WAV
Conditioning	Mean removal, 2 kHz high-pass filter, 1024-sample Hann frames with 50% overlap, analysis band 2 to 10.088 kHz
Algorithms	Trig + FFT, MUSIC and LUT + cost function, compared; the best is chosen in the conclusions
Display	Array layout with a direction line, angle readout, waveforms and status line, updated once per recording
Enclosure	Material is PLA, with angle markings (CAD views in Appendix C)
Test fixture	Turntable with markings every 22.5 degrees; enclosure markings every 45 degrees; test distances 90 cm and 180 cm

3.12 System Integration

3.12.1 Interaction between design choices

The decisions constrain one another. The 17 mm spacing sets the 10.088 kHz aliasing limit, which sets the upper edge of the analysis band, and the high-pass filter sets the lower edge. The sampling rate is fixed by the Teensy and is well above this band. The available RAM limits the recording to about 3 s, which fixes the number of frames used for each estimate, and so the averaging that reduces the effect of noise.

3.12.2 Latency and data budget

The data rate through the chain is about 265 kB/s, from three channels at about 44.1 kHz with two bytes per sample, or about 269 kB/s with the packet framing. A recording is about 0.8 MB. Table 3.17 gives the delay from sound to displayed angle.

Table 3.17: Latency budget and measured time for each stage

Stage	**Budget (s)**	**Measured (s)**	**Note**
Recording	3.0	3.0	1034 blocks of 128 samples at 44 117.6 Hz
Transfer	17	10	USB serial transfer to the laptop
Processing	10	1.5	Estimation on the laptop
Display update	30	25	Depends on the laptop processor speed and RAM
Total	**60**	**39.5**	Budget remainig 20.5 s

3.13 Design Summary

Table 3.18 traces each specification to the design decision that meets it.

Table 3.18: Design decision for each specification

ID	Design decision	Section
S01	Three SPH0645 digital MEMS microphones in an equilateral triangle, 17 mm side	Array geometry; Microphone selection
S02	Analysis band of 2 kHz to 10.088 kHz, with a 2 kHz high-pass filter	Signal conditioning
S03	One shared I2S clock from the Teensy 4.0	Acquisition hardware
S04	Fixed 44 117.6 Hz sampling rate, above twice the analysis band	Sampling parameters
S05	USB serial with framed packets and a block counter	Data transfer
S06	Azimuth from the phase differences in each 1024-sample frame, combined over the recording	DOA algorithms; Software design
S07	Three algorithms implemented and compared; accuracy judged in the results	DOA algorithms; Results
S08	One display update per recording, after processing	Real-time display; Latency budget
S09	Strongest in-band component per frame, with a frame gate	Signal conditioning; Software design
S10	Julia simulator with a Monte Carlo framework	Simulator
S11	Julia with LibSerialPort, GLMakie, FFTW and WAV	Software architecture
S12	WAV files named by distance, angle, source type and frequency	Host acquisition

Chapter 4

Experimental Setup and Analysis Methodology

This chapter describes how the system was tested: the environment, the equipment, the procedure, the test signals, the processing parameters and the performance metrics. It also describes the design of the simulation experiments. The description is detailed enough for the tests to be repeated. The results are given in the next chapter.

4.1 Objectives and Overview of the Experiments

The experiments answer the following questions, which come from the acceptance test procedures in Chapter 3. First, how accurately does each of the three algorithms (Trig + FFT, MUSIC, and LUT + cost function) estimate a known angle, in simulation and on the real array? Second, does the accuracy depend on the angle, the source distance or the type of signal? Third, how repeatable is the estimate when the same test is repeated? Fourth, how sensitive are the algorithms to noise, to errors in the microphone positions, to mismatch between the channels and to the choice of frames and the observation length? The first three questions are answered with known-angle tests, and the fourth with Monte Carlo simulation. Table 4.1 gives the overview.

Table 4.1: Overview of the experiments

Exp.	Question	Conditions	Metrics
K1	Accuracy of each algorithm on a known angle in simulation	3 algorithms; 4 kHz sine and broadband signals; angles 0, 22.5, 45, 90, 180 and 270 degrees; 6 runs each	Error, MAE, bias, SD, RMSE, 95% CI
K2	Accuracy of each algorithm on the real array	3 algorithms; 4 kHz sine, speech and clap; distances 90 cm and 180 cm; 6 angles; 6 repeats each	Error, MAE, bias, SD, RMSE, 95% CI, share of runs within 15 degrees
MC-A	Sensitivity to noise	Recorded speech at 90 cm and 180 cm, 6 angles; SNR from 40 dB to -5 dB	Shift of the estimate (bias, SD, RMSE, MAE, p95, outliers)
MC-B	Sensitivity to microphone position errors	As MC-A; position perturbation of 0.25 to 2 mm	As above
MC-C	Sensitivity to gain and delay mismatch	As MC-A; gain 0.25 to 2 dB; delay 0.5 to 10 microseconds	As above
MC-D	Sensitivity to frame selection and observation length	As MC-A; frame gate: all, 30, 20 and 10 dB; length 4096 to 32768 samples	As above and error against the true angle

4.2 Test Environment

All real-array tests were done in a laboratory with little to no background noise. The air temperature was 20 degrees Celsius, taken as the room temperature and not measured during each test. The floor area of the room is about 30 square metres. The room was not anechoic, so reflections from walls, tables and equipment were present and are discussed as a limitation in the section on calibration, uncertainty and validity.

At 20 degrees Celsius the speed of sound is about 343 m/s, which is the value used in the software. The temperature was not logged during the tests, so any difference between the actual and the assumed temperature is a source of error, treated in the section on calibration, uncertainty and validity.

4.3 Equipment

Table 4.2 lists the equipment used.

Table 4.2: Equipment used in the real-array tests

Item	Description
Array	Three SPH0645 I2S microphones on an equilateral triangle of 17 mm side, connected to a Teensy 4.0
Enclosure	Housing for the whole system, with angle markings in steps of 45 degrees
Turntable	Turntable with angle markings in steps of 22.5 degrees, adapted from an online CAD model (Appendix C)
Sound source and recorder	Redmi 14C phone, used to play the sine and speech test sounds. The hand clap was made by hand at the source position.
Computer	ASUS laptop, Intel Core i5 processor, 20 GB RAM, Windows 11
Software	Julia program for acquisition, processing and display and Teensy firmware (Appendix B)
Distance measurement	Measuring tape with 1 mm graduations

4.4 Experimental Layout and Procedure

4.4.1 Layout

The array and its enclosure are mounted on the turntable, and the source is fixed at a chosen distance in front of the array. Two source distances were used, 90 cm and 180 cm. The fixture is described in Chapter 3 (Figure 3.12), and detailed drawings are in Appendix C. The markings of the turntable are in steps of 22.5 degrees and those of the enclosure in steps of 45 degrees, and any angle between the markings can be set by reading the scale. Before each test series the zero of the enclosure is aligned with the zero of the turntable. The top of the turntable then rotates together with the enclosure as one body, so the array turns and the source stays fixed.

The angle convention is the same for the turntable and the software: zero is on the positive x axis and angles increase counter-clockwise.

4.4.2 Procedure

Each test followed the same steps.

1. Set the distance between the array and the source to 90 cm or 180 cm.
2. Align the zero of the enclosure with the zero of the turntable.
3. Select the angle by rotating the turntable to the required marking.
4. Play the test sound from the phone.
5. The system records the sound automatically when the trigger level is exceeded, analyses the recording with the three algorithms and outputs the angle estimates.
6. Record the estimated angles with the true angle, then repeat from the third step for the next angle.

Every condition was measured six times, except the hand clap, which was measured once at each condition. A hand clap is a live transient whose level, duration and spectrum differ from one clap to the next, and the recordings are not stored or replayed. A single clap was therefore measured at each

condition, and the result is an indication of performance, not a statistically repeatable measurement. All the tests, the test signals and the results are recorded in the results workbook, in which the first measurement of each condition and the five repeats are stored in separate columns. Each recording file is named with the signal name, the distance and the angle.

4.5 Test Signals

Table 4.3 lists the test signals.

Table 4.3: Test signals

Signal	**Used in**	**Purpose**
4 kHz sine tone	K1, K2	Narrowband signal inside the analysis band (2 to 10.088 kHz) and below the spatial aliasing limit; the simplest case for the three algorithms
Speech	K2	The phrase ”The salt breeze came across from the sea”, taken from the IEEE list of phrases, which contains enough frequency and time-domain information to test the algorithms on the kind of sound the system is meant to locate.
Clap	K2	Impulsive broadband sound with a very short duration, to test the trigger and the algorithms on a transient (UREQ07, FREQ08). Made by hand at the source position, and measured once at each condition
Broadband signal	K1, MC	Simulated signal with several tones (4, 6 and 9.5 kHz) and noise, used where the real array is not involved

The test signals and all the recorded results are in the results workbook, which is in the code repository (Appendix E).

4.6 Data Processing Parameters

Table 4.4 gives the parameters used by the processing, which are the same for all three algorithms so that the comparison is fair.

Table 4.4: Processing parameters

Parameter	**Value**
Sampling rate	44117.6 Hz (fixed by the Audio library); 44118 Hz in the Julia program
Block and recording length	128 samples per block; 1034 blocks, about 3.0 s
Trigger level	200 counts
Pre-filter	2 kHz high-pass, applied in the frequency domain
Analysis band	2 kHz to 10.088 kHz (spatial aliasing limit $c/2d$)
Window and frames	Hann window, 1024 samples, 50% overlap
Frame gate	Frames below 5% of the peak frame energy are rejected
Speed of sound	343 m/s
Algorithms	Trig + FFT, MUSIC, LUT + cost function

4.7 Performance Metrics

The accuracy requirement is an error of at most 15 degrees. This is 10 degrees for the system plus 5 degrees for the uncertainty of the known angle, which is plus or minus 5 degrees (Chapter 3). Every result is therefore also judged against the limit of 15 degrees.

For a true angle and an estimated angle, the error is the estimate minus the true angle, wrapped to the interval from -180 to 180 degrees: error = ((estimate - true + 180) mod 360) - 180. Wrapping is necessary because 359.9 degrees and 0.1 degrees are only 0.2 degrees apart. The mean and the standard deviation are computed on the wrapped errors, not on the raw angles, otherwise a result that falls on both sides of zero gives a standard deviation that is far too large. For n runs of one condition the metrics are as follows.

- **Bias** is the mean of the wrapped errors.
- **Standard deviation (SD)** is the spread of the wrapped errors about the bias, and measures the repeatability.
- **MAE** is the mean of the absolute wrapped errors.
- **RMSE** is the square root of the mean of the squared wrapped errors, and combines bias and spread.
- **95 percent confidence interval** of the mean error is the bias plus or minus 1.96 times SD divided by the square root of n.
- **Share within 15 degrees** is the percentage of runs whose absolute error is at most 15 degrees, which is the pass criterion of the requirement.
- **p95 and outlier rate** (Monte Carlo only) are the 95th percentile of the absolute error and the percentage of trials with an error larger than 15 degrees.

With six repeats the confidence interval is a rough guide only, and this is stated where the results are discussed.

4.8 Simulation and Monte Carlo Experiments

4.8.1 Simulator for the known-angle tests

The simulator models the three microphones on the 17 mm triangle. For a source at angle theta the delay of microphone 2 relative to microphone 1 is b sin(30 degrees - theta)/c and that of microphone 3 is b cos(theta)/c, with b the side length and c the speed of sound. The signals are generated at 44.1 kHz for 3 s, with tones at 4, 6 and 9.5 kHz for the broadband case, and Gaussian noise of standard deviation 0.01 is added. The same processing chain is applied as for the real array. Each of the six angles is run six times for each algorithm (experiment K1).

4.8.2 Monte Carlo set-up on the measured recordings

The Monte Carlo experiments are applied to the real recordings and not to simulated signals, so that the sensitivities are those of the algorithms working on real sound with the real room, microphones and noise. A recording is perturbed in a controlled way, the algorithm is run again, and the result is compared in two ways. The **error** is the wrapped difference between the estimate and the true angle. The **shift** is the wrapped difference between the estimate of the perturbed recording and the unperturbed (clean) estimate of the same recording and algorithm. The shift measures the sensitivity to the perturbation alone, whereas the error also contains the systematic error that is already present in the clean estimate. The experiments use the speech recordings at 180 cm and at 90 cm, with the six angles of the known-angle tests. Experiments A to C use 20 random trials per recording and level (120 trials per algorithm and level), and experiment D2 uses 200 random windows per recording (1200 per algorithm and length). The random generator is seeded so that the results can be reproduced. The metrics are the bias, SD, RMSE, MAE and p95 of the shift, and the percentage of outliers (shift above 15 degrees). MUSIC returns no estimate when no frequency bin gives a valid number of sources, and the percentage of such trials is reported.

4.8.3 Why only the speech recordings were used

Speech was chosen for the Monte Carlo experiments for four reasons. First, speech is the source the system is meant to locate (UREQ07), so the sensitivities of speech are the ones that matter in use. Second, each experiment needs a signal that varies across time and frequency. Experiment D1 selects frames by their energy, and a steady tone has frames of almost equal energy, so gating would change nothing. Experiment D2 uses windows at random positions, and for a tone every window is the same. Speech has strong and weak frames and a broad spectrum, so both experiments have something to select from. Third, delay and gain mismatch act on the phase and level at each frequency, and a tone tests a single frequency, whereas speech spreads over many bins, so the effect on the fused estimate can be seen. Fourth, the 4 kHz tone and the clap were not suitable. The tone gave the worst real-array results because of the standing-wave pattern in the room (Chapter 5), so its clean estimates are dominated by the room and the perturbations would be hidden. The clap lasts only a few tens of milliseconds, so it has too few frames for gating and too short a signal for windows of 4096 to 32768 samples. Running one signal type also keeps the number of runs manageable, since each level is repeated for six angles, 20 trials and three algorithms. The two distances were both used so that the sensitivities can be compared between them.

4.8.4 Experiment A: noise sensitivity

White Gaussian noise is added to the recorded channels at signal-to-noise ratios from 40 dB down to −5 dB. The SNR is defined in the analysis band (2 kHz to 10.09 kHz): the power of the added noise in this band equals the in-band signal power divided by 10ˆ(SNR/10), where the signal power is averaged over the three channels and over the 3 s window of the recording with the most energy. The noise is independent on each channel and a new noise sequence is drawn for every trial. This shows the SNR at which each algorithm leaves the limit, and the SNR at which MUSIC fails to give an estimate.

4.8.5 Experiment B: microphone position errors

The microphone coordinates used by the algorithm are perturbed randomly by 0.25, 0.5, 1 and 2 mm. Each value is the standard deviation of a zero-mean Gaussian error, drawn independently for the x and y coordinate of each of the three microphones in every trial. The recordings are not changed, so only the geometry assumed by the algorithm is wrong. This represents errors in the placement of the microphones on the board, and shows how accurately the geometry has to be known.

4.8.6 Experiment C: channel gain and delay mismatch

A random gain error of 0.25, 0.5, 1 and 2 dB is applied to the channels, and a random delay error of 0.5, 1, 2, 5 and 10 microseconds. The gain values cover the sensitivity tolerance of the microphones, and the delay values cover the sample-clock and wiring differences. The relative channel gains of the recordings were also measured, as the level of microphones 2 and 3 relative to microphone 1, to compare the real mismatch with the range tested.

4.8.7 Experiment D: frame selection and observation length

Experiment D1 estimates the angle of each frame of the recording and combines the frames, using all frames and then only the frames whose energy is within 30, 20 and 10 dB of the strongest frame. It applies to Trig + FFT and LUT + cost function, which estimate each frame separately. MUSIC forms a covariance over the frames and is not included in D1. Experiment D2 estimates the angle from windows of 4096, 8192, 16384 and 32768 samples (0.09 s to 0.74 s) at random positions in the recording, to show how the observation length affects the result.

4.9 Calibration, Uncertainty and Validity

The main sources of uncertainty in the real-array tests are listed below. The true angle is known to plus or minus 5 degrees, from reading the markings and aligning the two zeros. The turntable markings are in steps of 22.5 degrees, but any angle between them can be set by reading the scale, and the six test angles are only the ones that were measured. The distance was measured with a measuring tape with 1 mm graduations, so it has an uncertainty of a few millimetres. The phone speaker is not an ideal point source and its output at 4 kHz and for speech is not flat. Reflections in the room add to the direct sound. The room temperature was taken as 20 degrees Celsius and was not logged during the tests. The speed of sound changes by about 0.6 m/s for each degree, so a room that was 5 degrees warmer would change the delays by about 1 percent. The three microphones are sampled with the same clock, so they are synchronised and no timing calibration of the channels was needed.

The results therefore describe the performance of the whole system in one room and with one source, not of the algorithms alone. The simulations isolate the algorithms from these effects, and the comparison between the two is made in the results chapter.

4.10 Chapter Summary

The system was tested in a quiet laboratory at about 20 degrees Celsius using a phone as the source and a turntable to set the known angle, at 90 cm and 180 cm. Six angle-based conditions were measured six times for each algorithm and signal, and the errors are summarised with the wrapped error, bias, SD, RMSE, MAE and a confidence interval. Monte Carlo simulation adds the sensitivity to noise, geometry, mismatch, frame selection and observation length. The next chapter presents the results.

Chapter 5

Results and Discussion

This chapter presents the results of the experiments described in Chapter 4 and discusses them. The accuracy requirement (S07) is an error of at most 15 degrees: 10 degrees for the system, plus 5 degrees for the uncertainty of the known angle. Every result is judged against this limit of 15 degrees. Errors are wrapped errors in degrees, as defined in Chapter 4. All the individual measurements are in the results workbook in the project repository (Appendix E).

5.1 Simulation Results

5.1.1 Known-angle accuracy

Table 5.1 gives the accuracy of the three algorithms on simulated signals at the six test angles, with six runs each. All three algorithms estimate the angle correctly. The mean absolute error is 0.61 degrees for MUSIC, 0.64 degrees for Trig + FFT and 0.76 degrees for LUT + cost function, and the largest single error is 2.2 degrees. All 36 conditions meet the 15 degree limit.

Table 5.1: Accuracy on simulated signals (6 angles, 6 runs each, errors in degrees)

Algorithm	Signal	MAE	RMSE	Bias	SD	Max. error	Within 15 deg
MUSIC	4 kHz sine	0.57	0.73	-0.09	0.74	1.48	6/6
Trig + FFT	4 kHz sine	0.62	0.76	-0.08	0.77	1.49	6/6
LUT + cost	4 kHz sine	0.75	0.88	-0.24	0.86	1.80	6/6
MUSIC	Broadband	0.65	0.84	+0.11	0.85	1.49	6/6
Trig + FFT	Broadband	0.66	0.82	-0.02	0.83	1.76	6/6
LUT + cost	Broadband	0.78	0.96	-0.34	0.91	2.22	6/6

The three algorithms differ by less than 0.2 degrees in MAE, which is smaller than the spread between runs (SD about 0.8 degrees), so the simulation cannot separate them on accuracy. The bias is close to zero (between -0.34 and +0.11 degrees), so the errors are random, and they come from the added noise of the simulator. The result for the 22.5 degree case of the Trig + FFT algorithm, 22.4988 degrees in the signal simulation, agrees with this. The 4 kHz sine and the broadband signal give almost the same errors, so the signal type has little effect when the noise is the only impairment. LUT + cost function has a slightly larger error, which is expected because it searches a grid of angles and so has a resolution limit.

At 0 degrees the estimates are written as 359.9 or 360.9 degrees. These are the same direction as 0 degrees, and the statistics in this chapter are computed on the wrapped error so that they do not appear as errors of about 360 degrees.

5.2 Measured Results

5.2.1 Accuracy and repeatability on the real array

Table 5.2 gives the results of the real-array tests for the 4 kHz sine, the speech phrase and the hand clap. Each row combines the six angles with six repeats each (36 measurements). The column labelled repeatability is the standard deviation of the six repeats, averaged over the angles. The clap rows are from a single measurement at each angle, so no repeatability is given.

Table 5.2: Accuracy and repeatability on the real array (errors in degrees)

Algorithm	Signal	Distance	MAE	RMSE	Repeat. SD	Within 15 deg	Max. error
MUSIC	4 kHz sine	90 cm	82.4	94.4	0.80	1/6	156.6
Trig + FFT	4 kHz sine	90 cm	78.3	92.5	0.82	1/6	157.6
LUT + cost	4 kHz sine	90 cm	78.4	94.0	0.86	0/6	174.6
MUSIC	4 kHz sine	180 cm	54.3	59.8	0.74	0/6	96.6
Trig + FFT	4 kHz sine	180 cm	60.0	72.0	0.80	1/6	135.5
LUT + cost	4 kHz sine	180 cm	64.9	69.9	0.78	0/6	116.3
MUSIC	Speech	90 cm	48.7	70.2	0.78	1/6	155.6
Trig + FFT	Speech	90 cm	38.7	49.3	0.69	1/6	105.4
LUT + cost	Speech	90 cm	14.8	19.0	0.78	4/6	39.1
MUSIC	Speech	180 cm	69.8	101.1	0.79	2/6	174.1
Trig + FFT	Speech	180 cm	41.3	52.4	0.90	2/6	101.8
LUT + cost	Speech	180 cm	25.5	26.4	0.81	0/6	36.0
MUSIC	Clap	90 cm	50.7	63.2	n/a	1/6	122.3
Trig + FFT	Clap	90 cm	40.9	45.0	n/a	1/6	63.3
LUT + cost	Clap	90 cm	25.1	29.0	n/a	3/6	52.5
MUSIC	Clap	180 cm	53.6	60.1	n/a	0/6	104.5
Trig + FFT	Clap	180 cm	29.3	45.4	n/a	4/6	101.4
LUT + cost	Clap	180 cm	39.3	53.3	n/a	3/6	79.3

Two things stand out for the sine and the speech. First, the results are very repeatable: the standard deviation of the six repeats is between 0.3 and 1.2 degrees, and 0.8 degrees on average, which is about the same as in simulation. Second, they are not accurate. The MAE is between 15 and 82 degrees, and only 13 of the 72 conditions (18 percent) are within the 15 degree limit. The 4 kHz sine fails almost completely (3 of 36 conditions pass), and the speech phrase does better (10 of 36), but not well enough. For the clap, the errors are of the same size as for speech: the MAE is between 25 and 54 degrees and 12 of the 36 conditions are within the limit, with the best results for LUT + cost function at 90 cm (3 of 6 angles) and Trig + FFT at 180 cm (4 of 6 angles).

Table 5.3 gives the mean error at each angle, so that the pattern of the errors can be seen.

Table 5.3: Mean wrapped error at each true angle on the real array (degrees)

Algorithm	**Signal**	**Distance**	**0**	**22.5**	**45**	**90**	**180**	**270**
MUSIC	4 kHz sine	90 cm	+72.3	+112.7	+78.2	+3.4	+155.5	+72.6
Trig + FFT	4 kHz sine	90 cm	-34.2	+123.8	+72.5	-11.5	+156.0	+71.8
LUT + cost	4 kHz sine	90 cm	-29.1	+109.9	+66.3	-20.8	+174.1	+70.4
MUSIC	4 kHz sine	180 cm	-36.9	+40.0	+49.1	-79.5	+24.8	+95.5
Trig + FFT	4 kHz sine	180 cm	-32.6	+40.6	+51.4	+134.6	+14.7	+86.0
LUT + cost	4 kHz sine	180 cm	-58.3	+42.2	+50.8	+115.3	+42.0	+80.5
MUSIC	Speech	90 cm	+26.5	-18.2	-154.5	+64.0	-6.6	+22.0
Trig + FFT	Speech	90 cm	-7.6	-28.3	-31.5	-25.2	+104.3	+35.1
LUT + cost	Speech	90 cm	+5.9	-8.9	+9.6	-3.7	+37.4	+23.0
MUSIC	Speech	180 cm	+0.1	-172.8	+2.5	+20.3	-167.1	+55.5
Trig + FFT	Speech	180 cm	-6.4	-28.0	-9.8	-44.5	+101.1	+57.9
LUT + cost	Speech	180 cm	+35.1	+15.3	+20.1	-21.4	+29.2	+31.8
MUSIC	Clap	90 cm	-2.3	-29.4	+29.7	-61.4	+59.2	+122.3
Trig + FFT	Clap	90 cm	-29.2	-63.3	-48.5	-45.9	+5.3	-53.0
LUT + cost	Clap	90 cm	+10.6	+25.3	-33.1	+14.7	+52.5	+14.7
MUSIC	Clap	180 cm	-39.9	-23.9	-104.5	-67.7	-55.4	+30.4
Trig + FFT	Clap	180 cm	+13.4	+42.0	-2.2	+101.4	+7.2	+9.8
LUT + cost	Clap	180 cm	-1.3	+0.9	+75.7	+8.4	-79.3	+70.5

5.2.2 Discussion of the real-array accuracy

The combination of small spread and large error shows that the error is systematic: for a given set-up, the system gives nearly the same wrong answer each time. Random noise therefore does not explain it, and this is consistent with the simulation, where the noise gives errors of less than 2.2 degrees. The uncertainty of the known angle (plus or minus 5 degrees) is also far too small to explain errors of tens of degrees. The microphones share one clock, so a timing offset between the channels is not expected, and the delay offsets implied by the errors of the 4 kHz sine change in sign and size with the angle and reach about four samples, which is more than the largest physical delay of 2.2 samples, so a simple constant offset does not fit the data either. The following explanations are consistent with the data, and none has yet been tested separately.

- **Reflections in the room.** A pure tone in a room produces a standing-wave pattern, and the phase at each microphone depends on the position in that pattern. The result is repeatable, but depends on the angle and the distance. This matches the observation that the errors at 90 cm and at 180 cm differ, and that the 4 kHz sine is worse than speech, which has a broad spectrum and averages the pattern.

- **Geometry, channel assignment and angle convention.** An error in the microphone coordinates used by the code, in which microphone is connected to which channel, or in the mapping between the turntable angle and the software angle (the zero position or the direction of rotation) would give errors that depend on the angle. The simulation shows that the algorithms agree with the geometry in the code, but the real array was not checked against it separately.

- **Source effects.** The phone speaker is small and directional at 4 kHz, and is not a point source. At 90 cm and 180 cm the wave front is nearly plane for a 17 mm array, so a near-field effect is small.

- **Alignment of the set-up.** If the array is not on the rotation axis of the turntable, or the zeros are not well aligned, the angle seen by the array differs from the marked angle, but this effect is limited to the uncertainty of the known angle unless the offset is large.

The data gives some support for the first two explanations. The errors of the three algorithms are of similar size for the same signal and distance, which points to the data rather than to the algorithms, since the algorithms process the phase differences in different ways. Speech reduces the error for Trig + FFT and LUT + cost function, and for MUSIC at 90 cm (but not at 180 cm), as expected for a broadband source. The error is not a simple function of the true angle, and the angles that are near the correct value, such as 90 degrees at 90 cm with MUSIC (error +3.4 degrees), are not consistent between algorithms or distances. The most likely cause is therefore a combination of room reflections and an error in the geometry or the angle mapping. The Recommendations chapter lists the checks that would separate them.

5.2.3 Effect of signal type

The speech phrase gives a mean absolute error of 39.8 degrees against 69.7 degrees for the 4 kHz sine, averaged over the algorithms, distances and angles. The broadband content of speech means that the algorithms use many frequency bins, so the error at one frequency (for example from a standing wave) is averaged with the others. This agrees with the literature in Chapter 2, which reports that broadband and speech sources are more robust than a single tone in reverberant rooms. The best real-array result is LUT + cost function on speech at 90 cm, with an MAE of 14.8 degrees, which is just inside the 15 degree limit on average, and 4 of 6 angles within it. At 180 cm the same combination has an MAE of 25.5 degrees, so the result is not robust. The clap does not give a better result than speech, although it is a broadband transient with a short duration.

5.2.4 Hand clap

The hand clap is a short broadband transient. Table 5.2 and Table 5.3 give the results of the single measurement at each angle. The clap errors are as large and as irregular as those of the speech: the MAE is 25.1 degrees for LUT + cost function and 50.7 degrees for MUSIC at 90 cm, and 29.3 degrees for Trig + FFT, 39.3 degrees for LUT + cost function and 53.6 degrees for MUSIC at 180 cm. 12 of the 36 conditions are within 15 degrees, which is similar to speech (10 of 36) and much better than for the 4 kHz sine (3 of 36), but still not a pass. The error of a single clap also varies strongly from angle to angle (for example 101 degrees at 90 degrees for Trig + FFT at 180 cm, and 7 degrees at 180 degrees for the same case). A short event gives only a few usable frames, so a single estimate is much less averaged than that of a 3 s speech phrase, which may explain the irregular errors. Because only one clap was measured at each condition, no standard deviation or confidence interval is given for it, and the clap results are less reliable than those of the sine and the speech.

5.2.5 Effect of distance

The average MAE of the sine and the speech is 56.9 degrees at 90 cm and 52.6 degrees at 180 cm. The difference is small and the sign changes between algorithms and signals, so the distance does not have

a consistent effect. The errors at the two distances are different at the same angle, which is expected if reflections are the cause, because moving the source changes the interference pattern.

5.2.6 Monte Carlo analysis of the measured speech recordings

The Monte Carlo experiments of Chapter 4 were run on the speech recordings at 180 cm and at 90 cm. Table 5.4gives the errors of the clean (unperturbed) estimates, which are the reference for the shift. They are of the same kind as the speech rows of Table 5.3,but are from a single recording of each angle. The clean estimates of the LUT + cost function differ from the speech rows of the measured tables because the Monte Carlo code is not identical to the script used for the measured tables. It analyses the loudest 3 s window of each recording, not the whole recording, and the LUT script takes the tone frequency for all frames from the strongest bin of the first frame of the window. For a speech signal, which is not a single tone, that frequency depends on where the window starts, so the estimates change. The code also uses a grid step of 0.1 degrees and a speed of sound of 343 m/s. The estimates of MUSIC and Trig + FFT are less affected because they choose their frequency bins separately in each frame.

Table 5.4: Error of the clean estimates in the Monte Carlo experiments (degrees)

Algorithm	**Distance**	**0**	**22.5**	**45**	**90**	**180**	**270**
Trig + FFT	90 cm	-6.4	-31.3	-31.0	-25.1	+105.8	+34.9
MUSIC	90 cm	+26.4	-17.8	-154.5	+63.7	-7.4	+22.0
LUT + cost	90 cm	+23.6	+7.9	+6.4	-23.0	+89.4	+12.3
Trig + FFT	180 cm	-8.6	-23.5	-9.7	-47.0	+96.3	+57.8
MUSIC	180 cm	-0.1	-172.7	+2.6	+20.0	-167.2	+55.7
LUT + cost	180 cm	+36.6	-13.0	+6.8	-45.1	+59.6	+48.9

Because these clean estimates already contain large systematic errors, the Monte Carlo results are reported as the shift of the estimate caused by each perturbation: the RMSE of the shift over all trials and angles. A shift of 10 degrees or less is taken as acceptable, since 10 degrees is the part of the budget allowed for the system. Figure 5.1 and Figure 5.2 summarise the results, and the numbers are in the tables below. An asterisk in a table means that MUSIC returned no estimate in part of the trials.

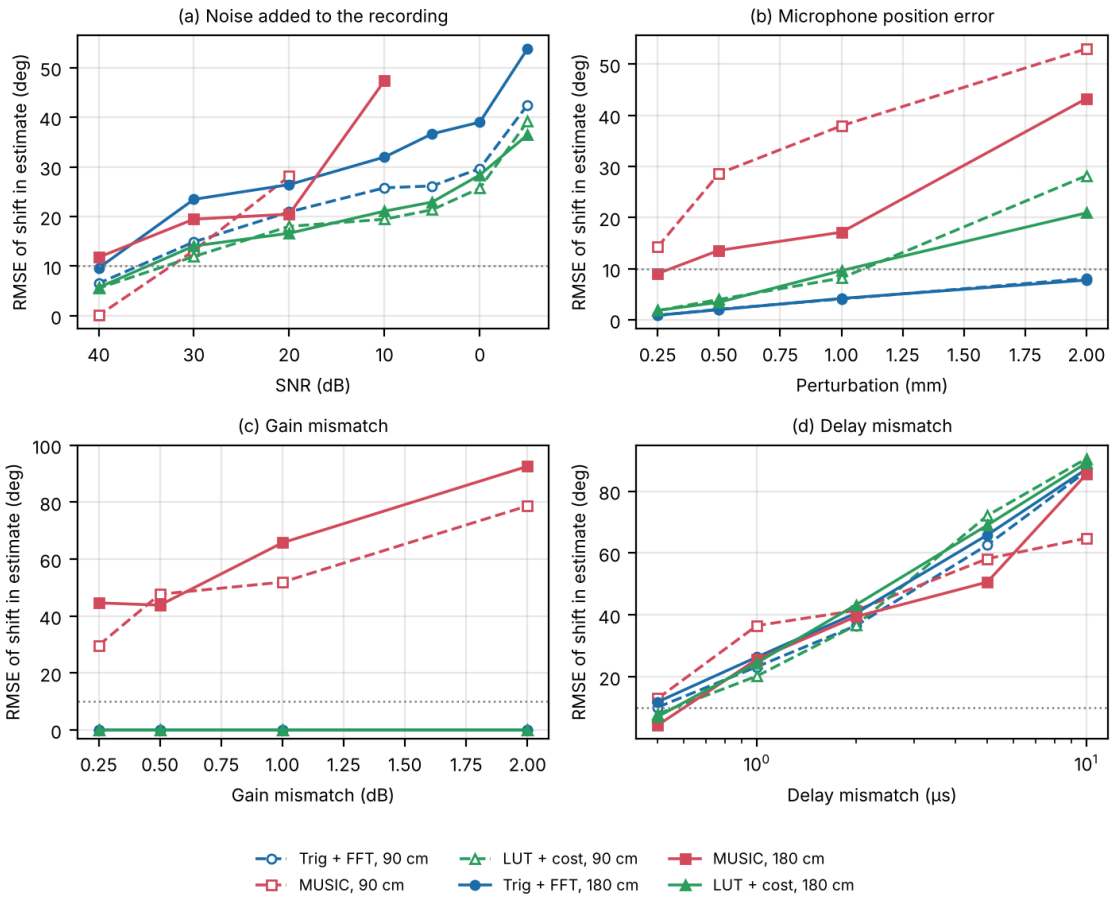


Figure 5.1: Shift of the estimate under noise, position error, gain mismatch and delay mismatch (solid: 180 cm, dashed: 90 cm; the dotted line is 10 degrees)

5.2.7 Noise sensitivity

Table 5.5 gives the shift against the SNR. All three algorithms are sensitive to noise added to the recording: even at 40 dB the shift is between 0.1 and 12 degrees, at 20 dB it is between 17 and 28 degrees, and at 30 dB or lower no algorithm keeps the shift below 10 degrees. LUT + cost function is the most stable at low SNR, with a shift of 19 to 21 degrees at 10 dB and 37 to 39 degrees at -5 dB, followed by Trig + FFT. MUSIC gives the smallest shift at 40 dB at 90 cm (0.1 degrees), but it fails abruptly when the noise increases. At 10 dB some trials give no estimate at 90 cm, at 5 dB only a third of the trials at 180 cm give an estimate, and at 0 dB and below it gives none. This is most likely because the noise raises the small eigenvalues of the covariance matrix, so that the source-counting test finds too many sources and no bin is usable.

Table 5.5: Shift of the estimate against the SNR (RMSE in degrees; * = some trials without an estimate)

SNR (dB)	Trig + FFT 90 cm	Trig + FFT 180 cm	MUSIC 90 cm	MUSIC 180 cm	LUT + cost 90 cm	LUT + cost 180 cm
40	6.5	9.6	0.1	11.8	5.6	5.7
30	14.9	23.5	13.1	19.5	11.9	14.1
20	20.9	26.4	28.1	20.5	18.0	16.6
10	25.8	32.0	62.1*	47.4	19.4	21.1
5	26.2	36.6	no estimate	2.9*	21.4	22.9
0	29.7	39.0	no estimate	no estimate	25.7	28.4
-5	42.5	53.8	no estimate	no estimate	39.1	36.5

The fact that the shift is not small even at 40 dB shows that the estimates on real speech depend strongly on small changes in the data, which fits the systematic and reflection-dominated errors found above. A more robust estimate would need a way to reject the frames in which the phase is not dominated by the direct sound, as in the frame gating below.

5.2.8 Microphone position errors

Table 5.6 gives the shift against the position error. The two phase-based methods are much less sensitive than MUSIC. Trig + FFT stays below 10 degrees up to 2 mm (8 degrees), LUT + cost function up to 1 mm (8 to 10 degrees), and MUSIC is already at 9 to 14 degrees at 0.25 mm and reaches 43 to 53 degrees at 2 mm. For a 17 mm baseline, 1 mm is 6 percent of the side length. The microphones must therefore be placed to within about 1 mm for the phase-based methods and to a fraction of a millimetre for MUSIC, which is demanding for a hand-assembled board.

Table 5.6: Shift of the estimate against the microphone position error (RMSE in degrees)

Error (mm)	Trig + FFT 90 cm	Trig + FFT 180 cm	MUSIC 90 cm	MUSIC 180 cm	LUT + cost 90 cm	LUT + cost 180 cm
0.25	0.9	0.9	14.2	9.0	1.8	1.8
0.5	2.1	2.0	28.6	13.6	4.0	3.4
1	4.1	4.2	37.9	17.1	8.2	9.6
2	8.1	7.8	53.0	43.3	28.2	20.9

5.2.9 Gain and delay mismatch

Table 5.7 gives the shift against the gain and the delay mismatch. The two phase-based methods are not affected by gain mismatch at all (the shift is zero to numerical precision), because they use only the phase of each bin. MUSIC is very sensitive: the shift is 30 to 45 degrees at 0.25 dB and 79 to 93 degrees at 2 dB, because the gain changes the covariance matrix and its eigenvalues, and so the estimated noise subspace. The measured channel gains of the recordings differ from microphone 1 by between -0.5 and +0.2 dB (microphone 2) and between -0.1 and +1.0 dB (microphone 3) at 90 cm and between -1.4 and +0.5 dB (microphone 2) and between -0.4 and +0.9 dB (microphone 3) at 180 cm, so the real mismatch is in the range in which MUSIC is affected.

Table 5.7: Shift of the estimate against the gain mismatch and the delay mismatch (RMSE in degrees)

Level	**Trig + FFT 90 cm**	**Trig + FFT 180 cm**	**MUSIC 90 cm**	**MUSIC 180 cm**	**LUT + cost 90 cm**	**LUT + cost 180 cm**
Gain 0.25 dB	0	0	29.7	44.6	0	0
Gain 0.5 dB	0	0	47.7	43.9	0	0
Gain 1 dB	0	0	51.9	65.7	0	0
Gain 2 dB	0	0	78.7	92.6	0	0
Delay 0.5 μs	10.2	11.9	13.0	4.4	8.0	7.1
Delay 1 μs	23.3	26.4	36.6	25.7	20.2	24.5
Delay 2 μs	36.6	40.6	41.5	39.5	36.7	43.3
Delay 5 μs	62.6	65.8	58.2	50.6	72.1	69.0
Delay 10 μs	86.4	87.1	64.8	85.5	90.5	89.1

The delay mismatch affects all three methods in the same way. A delay error of 0.5 microseconds already causes 4 to 13 degrees, 1 microsecond causes 20 to 37 degrees and 2 microseconds causes 37 to 43 degrees. This is because the largest delay between two microphones of a 17 mm baseline is only 50 microseconds, so 1 microsecond is 2 percent of the whole range and 0.04 of a sample period. The delay between the channels must be known to a fraction of a microsecond. The microphones share the same clock, so a mismatch of this size between the channels is not expected from the sampling, but delays from differences in the wiring or the processing of the channels are not excluded.

5.2.10 Frame selection

Table 5.8 gives the mean absolute error of the combined estimate against the true angle when only the strongest frames are used, with the number of angles within 15 degrees in brackets. Selecting the strongest frames reduces the error of the real-array estimate. For Trig + FFT the error falls from 39 and 41 degrees to 24 and 20 degrees at 90 cm and 180 cm, and for LUT + cost function from 27 and 35 degrees to 22 and 17 degrees, and the number of angles within 15 degrees rises from 2 to 4 of 6 for LUT + cost function at 180 cm. This agrees with the expected behaviour of speech: the frames with the most energy have the best signal-to-noise ratio and are the most dominated by the direct sound, so their phase is the most reliable. The gain from the gate is smaller at 90 cm, and it is not always steady (LUT + cost function at 90 cm is best at 20 dB). The 10 dB gate keeps only about 15 to 22 percent of the frames, so the gate has to be balanced against the amount of data. The 5 percent power gate of the real-time program (about 13 dB) lies between the 10 dB and 20 dB cases. A tighter gate is therefore likely to improve the real-time accuracy.

Table 5.8: Mean absolute error against the true angle with frame gating (degrees, angles within 15 degrees in brackets)

Gate	**Trig + FFT 90 cm**	**Trig + FFT 180 cm**	**LUT + cost 90 cm**	**LUT + cost 180 cm**
all	39.1 (1/6)	40.5 (2/6)	27.1 (3/6)	35.0 (2/6)
30 dB	26.1 (1/6)	23.8 (2/6)	20.5 (3/6)	26.4 (2/6)
20 dB	26.1 (1/6)	20.9 (3/6)	19.4 (3/6)	22.1 (2/6)
10 dB	23.5 (3/6)	20.1 (3/6)	22.3 (2/6)	16.5 (4/6)

5.2.11 Observation length

Table 5.9 gives the RMSE against the true angle for windows of 4096 to 32768 samples. The error is between 65 and 85 degrees for all algorithms and lengths, and it does not fall as the window is made longer, so windows of up to 0.74 s are too short for a reliable estimate of this speech phrase, which has short pauses and a few strong syllables. The estimate must be made from the whole recording of about 3 s, as the real-time program does. MUSIC also fails to give an estimate in 4 to 19 percent of the windows. The shift against the clean estimate, which is the estimate from the whole recording, is between 52 and 102 degrees. It is far above the 10 degree limit, so a window of up to 0.74 s does not reproduce even the estimate of the whole recording. The angle that a short window gives depends on which part of the phrase it covers, because the estimates of the individual frames spread widely (see frame-selection section). The whole-recording estimate averages over this spread, which is why it is more stable than any short window and why a longer observation time is needed.

Table 5.9: RMSE against the true angle for windows of different length (degrees)

Length (samples)	**Trig + FFT 90 cm**	**Trig + FFT 180 cm**	**MUSIC 90 cm**	**MUSIC 180 cm**	**LUT + cost 90 cm**	**LUT + cost 180 cm**
4096	68.0	67.7	74.2	76.4	65.1	69.5
8192	65.0	68.5	72.2	82.1	68.5	64.9
16384	80.2	76.4	80.1	85.4	83.1	75.2
32768	78.2	71.9	69.6	80.3	75.0	77.5

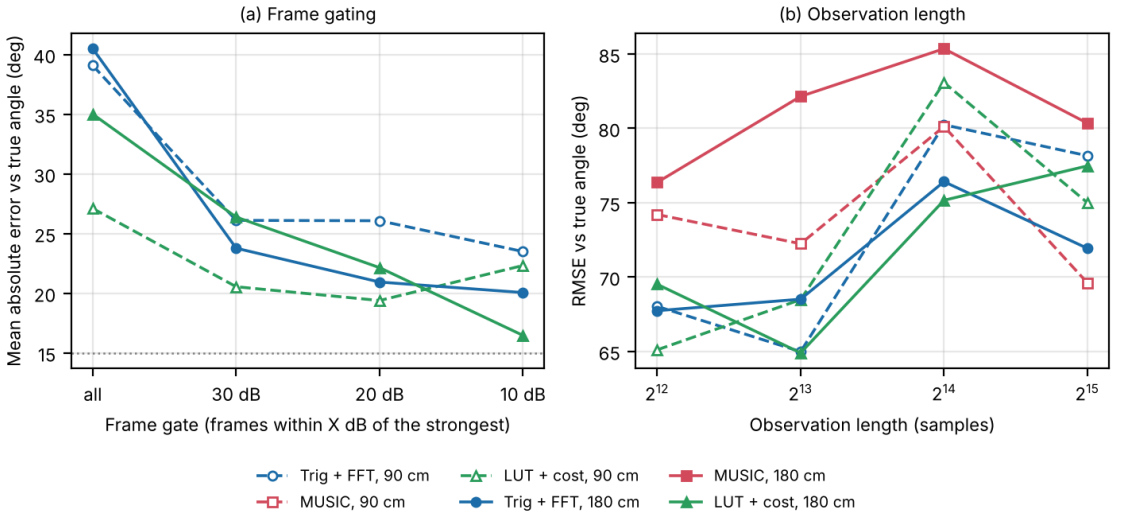


Figure 5.2: Error of the combined estimate with frame gating (a) and with the observation length (b) (solid: 180 cm, dashed: 90 cm)

5.2.12 Error budget

Table 5.10 summarises the shift for one representative level of each perturbation, so that their sizes can be compared. A short window gives the largest shift, followed by a delay mismatch of 2 microseconds and noise at 10 dB. A position error of 0.5 mm gives the smallest shift for the phase-based methods, and gain mismatch matters only for MUSIC. The phase-based methods, Trig + FFT and LUT + cost function, are therefore the more robust choices, and Trig + FFT is the best against position error.

Table 5.10: Error budget: shift of the estimate for representative levels (RMSE in degrees)

Perturbation	Trig 90 cm	Trig 180 cm	MUSIC 90 cm	MUSIC 180 cm	LUT 90 cm	LUT 180 cm
Noise, SNR 10 dB	25.8	32.0	62.1	47.4	19.4	21.1
Position error 0.5 mm	2.1	2.0	28.6	13.6	4.0	3.4
Gain mismatch 1 dB	0	0	51.9	65.7	0	0
Delay mismatch 2 microseconds	36.6	40.6	41.5	39.5	36.7	43.3
Window of 8192 samples	59.9	60.7	83.3	98.0	63.5	58.5

5.3 Simulated Against Measured Results

The simulation gives a mean absolute error of 0.6 to 0.8 degrees and the real array gives 15 to 82 degrees. The repeatability is almost the same in both (about 0.8 degrees). The difference is therefore not in the random noise, which the simulator models, but in effects that the simulator does not include: reflections, errors in the real microphone positions or the channel assignment, and the response of the phone speaker. The simulation shows that the algorithms and the software chain are correct for an ideal array, and the measurements show that the real system has an extra error source that has to be removed before the accuracy requirement can be met. The Monte Carlo results give the sizes of the effects that can be excluded: a position error of 1 mm or a gain mismatch causes at most a few degrees for the phase-based methods, so it cannot account for errors of tens of degrees on its own, but a delay error of about 1 microsecond or more can.

5.4 Algorithm Comparison

Table 5.11 compares the three algorithms. In simulation they have the same accuracy within the noise. On the real array, MUSIC, Trig + FFT and LUT + cost function have an MAE of 63.8, 54.6 and 45.9 degrees, and 4, 5 and 4 of the 24 conditions within the limit. The differences are small compared with the common error, so the accuracy data do not allow one algorithm to be called the best. The Monte Carlo results separate them better: Trig + FFT is the least sensitive to position errors, LUT + cost function is the most stable against noise, and both are unaffected by gain mismatch, whereas MUSIC is the most sensitive to position errors, gain mismatch and noise, and gives no estimate at low SNR. The LUT + cost function is chosen as the best algorithm. It had the lowest mean absolute error on the real array (45.9°, against 54.6° for Trig + FFT and 63.8° for MUSIC). Its simulated error of 0.76° is only about 0.1° above the other two, which is negligible, and its computation time of 2.2 s is far inside the 15 s design budget and the 1 min limit of S08. Trig + FFT is the fastest at 0.9 s and is the alternative if processing speed ever becomes the constraint.

Table 5.11: Comparison of the three algorithms

Measure	Trig + FFT	MUSIC	LUT + cost	Source
Simulation MAE (degrees)	0.64	0.61	0.76	Table 5.1
Real-array MAE, sine and speech (degrees)	54.6	63.8	45.9	Table 5.2
Conditions within 15 degrees, sine and speech	5 of 24	4 of 24	4 of 24	Table 5.2
Shift at 40 dB SNR (degrees)	6.5 to 9.6	0.1 to 11.8	5.6 to 5.7	Table 5.5
Shift at 1 mm position error (degrees)	4.1	17 to 38	8 to 10	Table 5.6
Shift at 1 dB gain mismatch (degrees)	0	52 to 66	0	Table 5.7
Computation time per estimate	0.9 s	1.5 s	2.2 s	Measured on the laptop

The real-array rows use the 4 kHz sine and the speech recordings, at both distances (24 conditions per algorithm). The hand clap is left out because it has no repeatability measure.

5.5 Real-Time Performance

A recording takes about 3.0 s (1034 blocks of 128 samples at 44 117.6 Hz) and the 781-byte packets are sent over USB serial. The measured time from the sound to the displayed angle is 54.5 s: 3.0 s for the recording, 20 s for the transfer, 1.5 s for the processing and 30 s for the display update. This is within the 1 minute allowed by S08, with a margin of 5.5 s, but it is more than three times the 15 s that was budgeted for the stages. The processing time depends on the algorithm: 0.9 s for Trig + FFT, 1.5 s for MUSIC and 2.2 s for LUT + cost function, so it is not what limits the latency. The transfer and the display update account for 50 of the 54.5 s, and they depend on the speed and memory of the laptop. Because the stages run one after the other, a new angle appears about once per 54.5 s, which is about 1.1 estimates per minute. The margin is small, so a slower laptop could exceed the limit.

5.6 Acceptance Test Results

Table 5.12 summarises the acceptance tests.

Table 5.12: Results of the acceptance test procedures

ATP ID	Pass criterion	Measured result	Pass or fail
ATP01	Three or more microphones, positions recorded	Three microphones on a 17 mm equilateral triangle	Pass
ATP02	No relative offset beyond 1 sample	The delays measured with the source at several known positions agreed with the delays expected from the geometry at every position, so there is no offset between the channels. The three microphones also share one clock source.	Pass
ATP03	No data lost or repeated	Blocks are lost in the first recording after startup and none in the recordings after it. The first recording is treated as a warm-up and is not used for measurements.	Pass (after the first recording)
ATP04	Tones at correct frequencies, no aliasing	The tones appear at exactly the expected frequencies, with no aliasing	Pass
ATP05	Error within 15 degrees	Simulation: all 36 conditions pass (maximum 2.2 degrees). Real array: 13 of 72 conditions pass (MAE 15 to 82 degrees)	Pass (simulation); fail (real array)
ATP06	Error statistics for each source type, within 15 degrees	Sine and speech: statistics in Tables 5.2 and 5.3, mostly outside the limit. Clap: 12 of 36 conditions within the limit (single measurement per condition)	Fail
ATP07	Display updates at least as often as S08	Time from sound to displayed angle 54.5 s, against 60 s allowed by S08	Pass
ATP08	Multichannel data match the chosen settings	Simulator produced data that gave the expected angles in all cases	Pass
ATP09	Same code runs on recorded and simulated data	The same MUSIC, Trig + FFT and LUT + cost functions were run on the simulated and the recorded data without changes	Pass
ATP10	Each recording identifies its test conditions	Recording names contain the signal, the distance and the angle	Pass

5.7 Limitations

The real-array results are for one room, one source (a phone speaker), two distances and six angles, with six repeats, so the confidence intervals are wide and the results cannot be used to claim performance in other rooms. Only six angles were tested, although any angle can be set. The Monte Carlo experiments use one speech phrase. The cause of the systematic error on the real array has not been isolated, so the real-array results describe the present state of the prototype, not the limit of the method. The temperature was taken as 20 degrees and not logged. The clap results are from a single live clap at each condition, so they have no standard deviation or confidence interval and are less reliable than the sine and speech results.

Chapter 6

Conclusions

6.1 Summary of the Work

An acoustic direction finding system was designed, built and tested. It uses three SPH0645 I2S microphones on a 17 mm equilateral triangle, a Teensy 4.0 that records the three channels and sends them over USB, and a Julia program that estimates and displays the azimuth. Three algorithms were implemented: Trig + FFT, MUSIC and LUT + cost function. A simulator was built to test them, and the complete system was tested at known angles on a turntable at 90 cm and 180 cm with a 4 kHz sine, a speech phrase and a hand clap. A Monte Carlo analysis on the recorded speech measured the sensitivity of the algorithms to noise, microphone position errors, gain and delay mismatch, frame selection and observation length.

6.2 Conclusions

- **The design meets its functional aims.** The three channels are sampled together at 44117.6 Hz, the data are transferred without loss, and the direction is estimated and displayed for a single source.
- **In simulation all three algorithms meet the accuracy requirement.** The mean absolute error is 0.61 degrees for MUSIC, 0.64 degrees for Trig + FFT and 0.76 degrees for LUT + cost function, with a maximum error of 2.2 degrees, against a limit of 15 degrees (10 degrees for the system and 5 degrees for the uncertainty of the known angle). The algorithms cannot be separated on accuracy in the noise-only simulation.
- **On the real array the requirement is not yet met.** Only 13 of 72 test conditions are within 15 degrees, and the mean absolute error is 15 to 82 degrees. The estimates are very repeatable (standard deviation 0.8 degrees on average), which shows that the error is systematic and not random.
- **Speech is more accurate than a pure tone.** The mean absolute error is 39.8 degrees for speech and 69.7 degrees for the 4 kHz sine. The best real-array result is LUT + cost function on speech at 90 cm, with an MAE of 14.8 degrees.
- **The distance had no consistent effect** between 90 cm and 180 cm.

- **The phase-based methods are more robust than MUSIC.** In the Monte Carlo analysis, Trig + FFT and LUT + cost function are unaffected by gain mismatch and keep the shift below 10 degrees for position errors up to 2 mm and 1 mm respectively, whereas MUSIC shifts by 30 to 45 degrees for a gain mismatch of 0.25 dB and gives no estimate at an SNR of 0 dB or lower. All three are sensitive to a delay error between the channels: 1 microsecond shifts the estimate by 20 to 37 degrees.

- **Using only the strongest frames improves the real-array estimate.** With the 10 dB frame gate the mean absolute error of Trig + FFT falls from 39 and 41 degrees to 24 and 20 degrees (90 cm and 180 cm), and that of LUT + cost function from 27 and 35 degrees to 22 and 17 degrees. Windows shorter than 0.74 s did not give reliable estimates.

- **Impulsive sound.** The hand clap gives errors of the same size as speech (MAE 25 to 54 degrees, 12 of 36 conditions within 15 degrees, from a single measurement per condition), so the system does not meet the accuracy requirement for impulsive sounds either.

- **Best algorithm:** LUT + cost function. It has the lowest MAE on the real array for the sine and speech recordings (45.9 degrees, against 54.6 for Trig + FFT and 63.8 for MUSIC), and it is not affected by a gain mismatch of 1 dB (0 degrees, against 52 to 66 degrees for MUSIC). Its shift for a 1 mm position error is 8 to 10 degrees, which is within the 10 degree allowance, whereas MUSIC shifts by 17 to 38 degrees. It is the slowest of the three at 2.2 s per estimate, but this is small against the 54.5 s total latency and well inside the 60 s of S08. The differences in accuracy between the algorithms are small compared with the spread of the real-array errors, and none of them meets 15 degrees on the real array, so the choice rests on robustness more than on accuracy.

Table 6.1: Aims, findings and evidence

Aim	**Finding**	**Evidence**
Build a working three-microphone system	Built and operating; data transferred and displayed	Chapter 3
Estimate the azimuth within 15 degrees (S07)	Met in simulation; not met on the real array	Tables 5.1 and 5.2
Compare three algorithms	Equal in simulation; no clear winner on the real array	Table 5.11
Test sine, speech and impulsive sources	Sine, speech and clap tested; none meets the limit	Tables 5.2 and 5.3

6.3 Contribution

The project adds a low-cost working acoustic direction finding system with a documented design, three implemented algorithms, a simulator, a Monte Carlo analysis on recorded speech, and a test method with a known angle. The evaluation shows that an ideal array of this size meets the accuracy limit and that the remaining work lies in finding and removing the systematic error in the real array, which the repeatability of the measurements makes possible.

Chapter 7

Recommendations

The recommendations follow from the results and apply to the use and further development of this system.

7.1 Design Recommendations

- Check the geometry and the channel order directly: place the source on the line through each pair of microphones, where the delay is largest (about 2.2 samples) and its sign is known, and compare the measured sign and size of the delay with the expected value. Correct any swapped channel or wrong coordinate in the code.
- Check the microphone coordinates and the angle convention of the code against the real array, for example by measuring the board and by comparing the estimate with the known angle for a source on each axis.
- Use a broadband source for the accuracy tests and for use of the system. Speech and other broadband sounds were much more accurate than a pure tone.
- Prefer Trig + FFT or LUT + cost function over MUSIC for use on the real array: they are unaffected by gain mismatch and less sensitive to position errors and noise. If MUSIC is kept, normalise the gain of each channel before forming the covariance matrix, since the measured channel gains differ by up to about 1.4 dB. Choose the final algorithm after the clap and computation-time results.
- Tighten the frame gate of the real-time program from 5 percent of the peak power (about 13 dB) to about 10 dB, and test the effect on the real array, since the Monte Carlo analysis shows the largest gain at this setting.
- Place the microphones to within about 1 mm, and calibrate the delay between the channels to a fraction of a microsecond, since these errors have a large effect on the estimate.

7.2 Operating Recommendations

- Align the zeros of the enclosure and the turntable before each series and re-check them at the end.

- Record the room temperature for each test and use it to set the speed of sound.
- Keep large reflecting objects away from the array and the source, and test with the source at several distances.

7.3 Evaluation Recommendations

- Reduce the uncertainty of the known angle (now plus or minus 5 degrees), for example with finer markings and a pointer, so that more of the 15 degree allowance is left for the system.
- Repeat the tests in a room with more absorption, or outdoors, to find out how much of the error comes from reflections.
- Test more angles than the six used here, including angles between the markings, and more repeats, so that the confidence intervals are narrower.
- Use the same sound for every repeat of a test. A live source such as a hand clap differs from one event to the next, which adds variability that is not caused by the system. Play a stored recording of the sound through the same speaker at a fixed level, so that the repeats are comparable and the standard deviation and confidence interval show only the variability of the system.
- Log the temperature and the sound level with each recording.

Chapter 8

Future Work

The following extensions were outside the scope of this project and are suggested for future research.

- **Calibration of the real array.** Find and remove the systematic error: measure the channel delays, calibrate the geometry, and repeat the tests. This is the first step before any of the extensions below.
- **Elevation and three-dimensional estimation.** A fourth microphone that is not in the plane of the others would allow the elevation to be estimated as well as the azimuth. The algorithms already take the microphone coordinates as an input, so the main change is in the geometry and the search over two angles.
- **Multiple sources.** MUSIC can in principle resolve more than one source, but with three microphones only two sources can be separated. Localising more would need a larger array.
- **Moving sources and tracking.** A tracking filter, for example a Kalman filter, could smooth the estimates and follow a moving source. The effect of the source motion within the 3 s recording would have to be studied.
- **Learning-based methods.** A neural network trained on recordings from the real array could learn the systematic effects of the room and the hardware. It would need a large set of labelled recordings, which the turntable method can produce, and a comparison with the classical methods used here.
- **Embedded real-time processing.** Running the direction finding on the Teensy itself would remove the dependence on the laptop and shorten the delay between the sound and the display.
- **Wider evaluation.** More rooms, distances and sources, including outdoor tests, and a larger array to improve the low-frequency resolution.

Bibliography

[1] M. A. Sahin, M. Ali, J. Park, and G. Destgeer, "Fundamentals of acoustic wave generation and propagation," in *Acoustic Technologies in Biology and Medicine* (A. Ozcelik, R. Becker, and T. J. Huang, eds.), pp. 1--36, Wiley-VCH, 2023. doi: 10.1002/9783527841325.ch1.

[2] F. E. Toole, E. A. G. Shaw, G. A. Daigle, and M. R. Stinson, "The physical nature of sound," in *Standard Handbook of Audio and Radio Engineering* (J. C. Whitaker and K. B. Benson, eds.), ch. 1.1, New York, NY, USA: McGraw-Hill, 2nd ed., 2002. [Online]. Available: https://sbe.org/handbook/fundamentals/Audio/Audio-The_Physical_Nature_of_Sound.pdf.

[3] A. V. Oppenheim, R. W. Schafer, and J. R. Buck, *Discrete-Time Signal Processing*. Upper Saddle River, NJ, USA: Prentice Hall, 2nd ed., 1999.

[4] L. Bouafif, K. Ouni, and N. Ellouze, "Implementation of a speech coding strategy for auditory implants," *The International Arab Journal of Information Technology*, vol. 7, pp. 388–394, 2010.

[5] L. R. Rabiner and R. W. Schafer, *Theory and Applications of Digital Speech Processing*. Upper Saddle River, NJ, USA: Prentice Hall, 2011.

[6] B. McFee, "2.4. quantization." in *Digital Signals Theory*. [Online]. Available: https://brianmcfee.net/dstbook-site/content/ch02-sampling/Quantization.html. [Accessed: Oct. 25, 2026].

[7] R. G. Lyons, *Understanding Digital Signal Processing*. Upper Saddle River, NJ, USA: Prentice Hall, 3rd ed., 2011. [Online]. Available: https://www.pearson.com/. [Accessed: Oct. 25, 2026].

[8] J. W. Cooley and J. W. Tukey, "An algorithm for the machine calculation of complex Fourier series," *Mathematics of Computation*, vol. 19, pp. 297--301, Apr. 1965. doi: 10.2307/2003354.

[9] F. J. Harris, "On the use of windows for harmonic analysis with the discrete Fourier transform," *Proceedings of the IEEE*, vol. 66, pp. 51--83, Jan. 1978. doi: 10.1109/PROC.1978.10837.

[10] L. Bouafif, K. Ouni, and N. Ellouze, "Implementation of a speech coding strategy for auditory implants," *Int. Arab J. Inf. Technol.*, vol. 7, no. 4, pp. 388--394, 2010.

[11] M. N. O. Sadiku, S. M. Musa, and S. R. Nelatury, "Correlation: A brief introduction," *International Journal of Electrical Engineering Education*, vol. 51, pp. 93--99, Apr. 2014. doi: 10.7227/IJEEE.51.2.1.

[12] J. S. Bendat and A. G. Piersol, *Random Data: Analysis and Measurement Procedures*. Hoboken, NJ, USA: Wiley, 4th ed., 2010. doi: 10.1002/9781118032428.

[13] C. H. Knapp and G. C. Carter, "The generalized correlation method for estimation of time delay," *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 24, no. 4, pp. 320--327, 1976.

[14] T. Fiehler, C. Saraceno, G. Steinmeyer, and U. Wittrock, "Pitfall in autocorrelation measurements of laser radiation," *Optics Express*, vol. 32, pp. 36811--36823, Oct. 2024. doi: 10.1364/OE.533567.

[15] S. M. Kay, *Fundamentals of Statistical Signal Processing: Estimation Theory*, vol. 1. Englewood Cliffs, NJ, USA: Prentice Hall, 1993.

[16] P. Horowitz and W. Hill, *The Art of Electronics*. Cambridge University Press, 3rd ed., 2015.

[17] H. Nyquist, "Thermal agitation of electric charge in conductors," *Physical Review*, vol. 32, pp. 110--113, 1928.

[18] J. B. Johnson, "Thermal agitation of electricity in conductors," *Physical Review*, vol. 32, pp. 97--109, 1928.

[19] A. K. Boyat and B. K. Joshi, "A review paper: Noise models in digital image processing," *Signal & Image Processing: An International Journal*, vol. 6, pp. 63--75, Apr. 2015. doi: 10.5121/sipij.2015.6206.

[20] W. Kester, "Taking the mystery out of the infamous formula, "SNR = 6.02N + 1.76 db," and why you should care." Analog Devices tutorial MT-001. https://www.analog.com/MT-001.

[21] InvenSense, "AN-1112: Microphone specifications explained." Application note. https://invensense.tdk.com/?p=6991.

[22] M. Brandstein and D. Ward, eds., *Microphone Arrays: Signal Processing Techniques and Applications*. Berlin, Germany: Springer, 2001. doi: 10.1007/978-3-662-04619-7.

[23] J. Benesty, J. Chen, and Y. Huang, *Microphone Array Signal Processing*. Springer, 2008.

[24] B. Chhaglani, C. Zakaria, A. Lechowicz, J. Gummeson, and P. Shenoy, "FlowSense: Monitoring airflow in building ventilation systems using audio sensing," *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies*, vol. 6, no. 1, Art. no. 5, pp. 1–26, Mar. 2022, doi: 10.1145/3517258.

[25] E. M. Salomons and S. A. Janssen, "Practical ranges of loudness levels of various types of environmental noise, including traffic noise, aircraft noise, and industrial noise," *International Journal of Environmental Research and Public Health*, vol. 8, pp. 1847–1864, 2011, doi: 10.3390/ijerph8061847.

[26] N. J. Kasdin, "Discrete simulation of colored noise and stochastic processes and $1/f^{\alpha}$ power law noise generation," *Proceedings of the IEEE*, vol. 83, no. 5, pp. 802--827, 1995.

[27] E. Milotti, "1/f noise: A pedagogical review." arXiv preprint physics/0204033, 2002. doi: 10.48550/arXiv.physics/0204033.

[28] G. C. Carter, "Coherence and time delay estimation," *Proceedings of the IEEE*, vol. 75, pp. 236--255, Feb. 1987. doi: 10.1109/PROC.1987.13723.

[29] J.-Y. Lu, D. Ye, and W.-P. Ma, “Time delay estimation based on variational mode decomposition,” *Advances in Mechanical Engineering*, vol. 9, no. 1, pp. 1–12, Jan. 2017, doi: 10.1177/1687814016688587.

[30] B. F. Cron and C. H. Sherman, “Spatial-correlation functions for various noise models,” *The Journal of the Acoustical Society of America*, vol. 34, no. 11, pp. 1732–1736, Nov. 1962, doi: 10.1121/1.1909110.

[31] G. W. Elko, “Spatial coherence functions for differential microphones in isotropic noise fields,” in *Microphone Arrays: Signal Processing Techniques and Applications* (M. Brandstein and D. Ward, eds.), Springer, 2001.

[32] A. Schwarz and W. Kellermann, “Coherent-to-diffuse power ratio estimation for dereverberation,” *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 23, no. 6, pp. 1006–1018, Jun. 2015, doi: 10.1109/TASLP.2015.2418571.

[33] J. Chen, J. Benesty, and Y. Huang, “Time delay estimation in room acoustic environments: an overview,” *EURASIP Journal on Advances in Signal Processing*, vol. 2006, p. 26503, 2006. doi: 10.1155/ASP/2006/26503.

[34] H. Kuttruff, *Room Acoustics*, 5th ed. Abingdon, U.K.: Spon Press, 2009. ISBN: 978-0-415-48021-5.

[35] J. B. Allen and D. A. Berkley, “Image method for efficiently simulating small-room acoustics,” *The Journal of the Acoustical Society of America*, vol. 65, no. 4, pp. 943--950, 1979.

[36] D. C. Rife and R. R. Boorstyn, “Single tone parameter estimation from discrete-time observations,” *IEEE Transactions on Information Theory*, vol. 20, pp. 591--598, Sept. 1974. doi: 10.1109/TIT.1974.1055282.

[37] A. A. Salama, “Direction of Arrival Estimation: A Tutorial Survey of Classical and Modern Methods,” arXiv preprint arXiv:2508.11675, 2025, doi: 10.48550/arXiv.2508.11675.

[38] G. Jekateryńczuk and Z. Piotrowski, “A survey of sound source localization and detection methods and their applications,” *Sensors*, vol. 24, no. 1, p. 68, 2024. doi: 10.3390/s24010068.

[39] A. Khan, A. Waqar, B. Kim, and D. Park, “A review on recent advances in sound source localization techniques, challenges, and applications,” *Sensors and Actuators Reports*, vol. 9, p. 100313, 2025. doi: 10.1016/j.snr.2025.100313.

[40] A. Abdallah and H. Tork, “Acoustic threat detection and direction finding system,” *IOP Conference Series: Materials Science and Engineering*, vol. 1172, no. 1, p. 012033, 2021. doi: 10.1088/1757-899X/1172/1/012033.

[41] D. Desai and N. Mehendale, “A review on sound source localization systems,” *Archives of Computational Methods in Engineering*, vol. 29, pp. 4631--4642, 2022. doi: 10.1007/s11831-022-09747-2.

[42] J. Blauert, *Spatial Hearing: The Psychophysics of Human Sound Localization*. MIT Press, revised ed., 1997.

[43] H. L. Van Trees, *Optimum Array Processing: Part IV of Detection, Estimation, and Modulation Theory*. Wiley, 2002.

[44] H. Krim and M. Viberg, "Two decades of array signal processing research: the parametric approach," *IEEE Signal Processing Magazine*, vol. 13, no. 4, pp. 67--94, 1996.

[45] Z. Heydari and A. Mahabadi, "Real-time TDOA-based stationary sound source direction finding," *Multimedia Tools and Applications*, vol. 82, no. 26, pp. 39929–39960, 2023, doi: 10.1007/s11042-023-14741-2.

[46] M. Ali Pour and Z. Habibzadeh, "Direction Estimation of Sound Sources Using Microphone Arrays and Signal Strength," arXiv preprint arXiv:2507.03466, 2025, doi: 10.48550/arXiv.2507.03466.

[47] M. Wikström, U. Ahnström, J. Falk, and P. Händel, "Implementation of an acoustic location-finding system for TDOA measurements," 2003.

[48] P. Stoica and A. Nehorai, "MUSIC, maximum likelihood, and {Cram\'er--Rao} bound," *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 37, no. 5, pp. 720--741, 1989.

[49] C. A. Balanis, *Antenna Theory: Analysis and Design*. Wiley, 4th ed., 2016.

[50] D. H. Johnson and D. E. Dudgeon, *Array Signal Processing: Concepts and Techniques*. Prentice Hall, 1993.

[51] D. J. Torrieri, "Statistical theory of passive location systems," *IEEE Transactions on Aerospace and Electronic Systems*, vol. 20, no. 2, pp. 183--198, 1984.

[52] R. B. Langley, "Dilution of precision," *GPS World*, vol. 10, no. 5, pp. 52--59, 1999.

[53] J. O. Smith, III, *Physical Audio Signal Processing*. W3K Publishing, 2010. [Online]. Available: https://ccrma.stanford.edu/~jos/pasp/.

[54] J. H. DiBiase, H. F. Silverman, and M. S. Brandstein, "Robust localization in reverberant rooms," in *Microphone Arrays: Signal Processing Techniques and Applications* (M. Brandstein and D. Ward, eds.), ch. 8, Springer, 2001.

[55] J. Capon, "High-resolution frequency-wavenumber spectrum analysis," *Proceedings of the IEEE*, vol. 57, no. 8, pp. 1408--1418, 1969.

[56] R. O. Schmidt, "Multiple emitter location and signal parameter estimation," *IEEE Transactions on Antennas and Propagation*, vol. 34, no. 3, pp. 276--280, 1986.

[57] R. Roy and T. Kailath, "ESPRIT---estimation of signal parameters via rotational invariance techniques," *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 37, no. 7, pp. 984--995, 1989.

[58] H. Wang and M. Kaveh, “Coherent signal-subspace processing for the detection and estimation of angles of arrival of multiple wide-band sources,” *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 33, no. 4, pp. 823--831, 1985.

[59] H. Hung and M. Kaveh, “Focusing matrices for coherent signal-subspace processing,” *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 36, no. 8, pp. 1272--1281, 1988.

[60] M. Wax and T. Kailath, “Detection of signals by information theoretic criteria,” *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 33, no. 2, pp. 387--392, 1985.

[61] Ö. Yılmaz and S. Rickard, “Blind separation of speech mixtures via time-frequency masking,” *IEEE Transactions on Signal Processing*, vol. 52, no. 7, pp. 1830–1847, Jul. 2004, doi: 10.1109/TSP.2004.828896.

[62] Knowles Corporation, “SPH0645LM4H-B: I²S Output Digital Microphone,” datasheet, Rev. C, Feb. 2017. [Online]. Available: https://www.knowles.com/docs/default-source/model-downloads/sph0645lm4h-b-datasheet-rev-c.pdf.

[63] Philips Semiconductors, “I2S bus specification,” 1996. Revised June 5, 1996.

[64] W. Kester, “Converting Oscillator Phase Noise to Time Jitter,” Analog Devices, Tutorial MT-008, Rev. A, Oct. 2008. [Online]. Available: https://www.analog.com/media/en/training-seminars/tutorials/MT-008.pdf.

[65] NXP Semiconductors, *i.MX RT1060 Processor Reference Manual*, Rev. 3, Jul. 2021. [Online]. Available: https://www.pjrc.com/teensy/IMXRT1060RM_rev3.pdf.

[66] Adafruit Industries, “Adafruit I2S MEMS Microphone Breakout—SPH0645LM4H,” Product ID 3421. [Online]. Available: https://www.adafruit.com/product/3421. [Accessed: Oct. 28, 2026].

[67] T. I. Laakso, V. V{\"a}lim{\"a}ki, M. Karjalainen, and U. K. Laine, “Splitting the unit delay,” *IEEE Signal Processing Magazine*, vol. 13, no. 1, pp. 30--60, 1996.

[68] R. Scheibler, E. Bezzam, and I. Dokmani{\'c}, “Pyroomacoustics: A Python package for audio room simulation and array processing algorithms,” in *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 351--355, 2018.

[69] J. Bezanson, A. Edelman, S. Karpinski, and V. B. Shah, “Julia: A fresh approach to numerical computing,” *SIAM Review*, vol. 59, no. 1, pp. 65--98, 2017.

[70] Big Simi, “Manual turning table (fully 3D printable),” MakerWorld, 2025. [Online]. Available: https://makerworld.com/en/models/1614489-manual-turning-table-fully-3d-printable. No licence is stated on the model page. [Accessed: Oct. 1, 2026].

Appendix A

Pseudocode for the Simulations

This appendix gives pseudocode for the signal simulator, the three direction-of-arrival algorithms and the Monte Carlo framework described in the software design section of Chapter 3. The full Julia code is in the repository listed in Appendix E. The azimuth is counter-clockwise from the positive x axis, and microphone 1 is the reference.

A.1 Signal and Noise Simulation

The signal simulation generates the three microphone signals for a known source angle and adds white Gaussian noise to each channel.

```
Algorithm B.1: signal and noise simulation
constants: fs = 44100 Hz, duration T = 3 s
           b = 0.017 m (side of the triangle), c = 343 m/s
           tone frequencies f1 = 4000, f2 = 6000, f3 = 9500 Hz
input:     true azimuth theta
t     <- 0, 1/fs, 2/fs, ..., T - 1/fs
tau21 <- b * sin(30 deg - theta) / c
tau31 <- b * cos(theta) / c
src   <- sin(2 pi f1 t) + sin(2 pi f2 t) + sin(2 pi f3 t)
mic1  <- src + 0.01 * randn(length(t))
mic2  <- sin(2 pi f1 (t - tau21)) + 0.01 * randn(length(t))
mic3  <- sin(2 pi f1 (t - tau31)) + 0.01 * randn(length(t))
return mic1, mic2, mic3
```

A.2 Trig + FFT Algorithm

```
Algorithm B.2: Trig + FFT estimate
constants: N = 1024, hop = N/2, Hann window w, band [Fmin, f1]
input:     mic1, mic2, mic3
for each frame i = 1 ... Nf
    S1, S2, S3 <- real FFT of w * (frame i of mic1, mic2, mic3)
```

```
    k      <- index of the strongest in-band bin of |S1|^2
    fk     <- k * fs / N
    lambda <- c / fk
    phi12  <- angle(S2[k] * conj(S1[k]))
    phi13  <- angle(S3[k] * conj(S1[k]))
    s      <- clamp(-phi12 * lambda / (2 pi b), -1, 1)
    cs     <- clamp(-phi13 * lambda / (2 pi b), -1, 1)
    sin_th <- (cs - 2 s) / sqrt(3)
    theta_i <- atan2(sin_th, cs) mod 2 pi
end
theta_hat <- atan2(mean(sin(theta_i)), mean(cos(theta_i))) mod 2 pi
return theta_hat
```

The relation used in the loop follows from the delays of the array: cs is cos(theta) and s is sin(30 degrees - theta), so sin(theta) = (cs - 2 s)/sqrt(3).

A.3 LUT + Cost Function Algorithm

```
Algorithm B.3: LUT + cost function
input:  x (N x 3 samples), fs, microphone positions p1, p2, p3
grid    <- 0, step, 2 step, ..., 360 - step   (step = 0.1 deg)
u       <- (cos(grid), sin(grid))
tau21   <- (p1 - p2) . u / c
tau31   <- (p1 - p3) . u / c
y       <- high-pass(x, 2 kHz), oversampled by the factor osf
S       <- STFT of y (1024-sample Hann frames, 50% overlap)
f_lut   <- frequency of the strongest in-band bin of frame 1
for each frame i
    k     <- index of the strongest in-band bin of |S1|^2
    f     <- f_lut          // or the frame's own bin frequency
    phi12 <- angle(S2[k] * conj(S1[k]))
    phi13 <- angle(S3[k] * conj(S1[k]))
    cost  <- wrap(-2 pi f tau21 - phi12)^2
           + wrap(-2 pi f tau31 - phi13)^2
    theta_i <- grid[argmin(cost)]
end
return circular mean of theta_i
```

A.4 MUSIC Algorithm

```
Algorithm B.4: MUSIC
input:  x (N x 3), fs, positions p1, p2, p3
y       <- high-pass(x, 2 kHz);  S <- STFT of y
```

```
energy[b] <- sum over frames and microphones of |S[b, i, m]|^2
bins    <- in-band bins with energy > 0.3 * (largest in-band energy)
grid    <- 0, 0.1, ..., 359.9 deg;   tau21, tau31 as in B.3
fused   <- zeros over the grid
for each bin b in bins, with frequency f
    X  <- 3 x Nf matrix of S[b, :, :]
    R  <- X X^H / Nf
    eigen-decompose R (eigenvalues in ascending order)
    K  <- number of eigenvalues larger than 1% of the largest
    if K = 3, skip this bin
    En <- eigenvectors of the (3 - K) smallest eigenvalues
    a(theta) <- [1, exp(-j 2 pi f tau21), exp(-j 2 pi f tau31)]
    P(theta) <- 1 / ||En^H a(theta)||^2
    fused    <- fused + energy[b] * P / max(P)
end
return the grid angle with the largest fused value
```

A.5 Noise, Mismatch and Geometry Perturbations

```
Algorithm B.5: perturbations used by the Monte Carlo experiments
add_noise(x, snr_db):        // experiment A
    P    <- mean power of x inside the analysis band
    var  <- P / 10^(snr_db/10) * (fs/2) / (Fhi - Fmin)
    return x + sqrt(var) * randn(size(x))
perturb_geometry(P, sigma_mm):   // experiment B
    return P + sigma_mm * 1e-3 * randn(3, 2)
mismatch(x, gain_db, delay_us):  // experiment C
    X <- real FFT of each channel
    X[:, m] <- X[:, m] * 10^(gain_db[m]/20) * exp(-j 2 pi f d[m])
               with d[m] = delay_us[m] * 1e-6 * fs  (in samples)
    return inverse FFT of X
```

A.6 Monte Carlo Driver

```
Algorithm B.6: Monte Carlo experiments
load recordings; true angle <- second field of each file name
segment <- loudest 3 s window of each recording
clean[rec, algo] <- estimate with no perturbation
A: for snr in levels, for trial in 1..n, for rec
       estimate of each algorithm on add_noise(segment, snr)
B: for sigma in levels, for trial in 1..n
       P' <- perturb_geometry(P_nominal, sigma)
```

```
       estimate of each algorithm on the recording with P'
C: for each of gain sigma and delay sigma, for trial in 1..n
       draw random gain / delay errors, apply mismatch()
D1: per-frame estimates of Trig + FFT and LUT + cost,
    with energy gates
D2: for window length L, for trial in 1..n
       cut a random window with energy within 20 dB of the loudest
       estimate of each algorithm on the window
for every row: err   <- wrap180(estimate - true angle)
               delta <- wrap180(estimate - clean estimate)
summaries: bias, standard deviation, RMSE, mean absolute error,
           95th percentile, percentage of |err| > 15 degrees
write CSV tables, one Excel workbook and the plots
```

Appendix B

Pseudocode for the Firmware and Real-Time Program

This appendix gives pseudocode for the Teensy firmware and for the Julia program that receives, processes and displays the data. The full code is in the repository listed in Appendix E.

B.1 Teensy Firmware

```
Algorithm C.1: Teensy 4.0 firmware
constants: block = 128 samples, TRIGGER = 200 counts
           RECORD_BLOCKS = 1034 (about 3.0 s at 44117.6 Hz)
storage:   blocks 0..599 in RAM2, 600..1033 in RAM1
           each block = mic1[128], mic2[128], mic3[128]
setup:  start the serial port, allocate audio memory
        arm: clear and start the three record queues
loop:
    if not triggered
        if all three queues hold a block
            m_k <- mean of block k, for k = 1, 2, 3
            if any sample differs from its block mean by > TRIGGER
                store the three blocks as block 0
                triggered <- true
            free the three blocks
    else
        record:  LED on
                 for n = 1 ... RECORD_BLOCKS - 1
                     wait until all three queues hold a block
                     copy the three blocks into block n; free them
                 stop and clear the queues; LED off
        send:    packet <- "START" | counter | total | 768 bytes
                 for n = 0 ... RECORD_BLOCKS - 1
```

```
                    fill counter = n and the samples of block n
                    write the 781-byte packet to USB serial
        wait 5 s;  arm again
```

B.2 Host Acquisition

```
Algorithm C.2: receive one recording (Julia listener)
buf <- empty byte buffer
loop while running
    find "START" in buf, reading the port when more bytes are needed
    total <- block count from the packet header
    read until buf holds total * 781 bytes
        (stop if the stream pauses for more than 1 s)
    raw <- zeros(total * 128, 3)
    for each packet k = 0 ... total - 1
        check the marker; counter c <- block counter
        place its 128 x 3 samples at rows c*128+1 ... (c+1)*128
        count the packet as received
    dropped <- total - received
    return raw, dropped, total
```

B.3 Real-Time Direction Estimate

```
Algorithm C.3: estimate_doa (real-time program)
constants: fs = 44118, b = 0.017 m, c = 343 m/s
           analysis band 2000 ... 10088 Hz, cut-off 2000 Hz
           N = 1024, hop = 512, Hann window
           gate = 5% of the peak power
           grid = 0, 0.01, ..., 359.99 deg
x <- raw samples as floating point, mean removed
high-pass filter: zero the FFT bins below the cut-off, inverse FFT
for each frame i
    S1, S2, S3 <- real FFT of the windowed frame of each channel
    k <- strongest in-band bin of |S1|^2;  peak_i <- its power
thr <- gate * max(peak_i)
for each frame with peak_i >= thr
    f     <- frequency of bin k
    phi12 <- angle(S2[k] * conj(S1[k]))
    phi13 <- angle(S3[k] * conj(S1[k]))
    kk    <- -2 pi f b / c
    cost  <- wrap(kk sin(30 deg - grid) - phi12)^2
           + wrap(kk cos(grid) - phi13)^2
```

```
    theta_i <- grid[argmin(cost)]
if no frame passed the gate, return no result
return circular mean of theta_i, number of frames used
```

B.4 Display

```
Algorithm C.4: display update
draw the microphones scaled by the baseline
start the listener task
for each recording received
    status <- "Processing"
    angle, frames <- estimate_doa(raw)
    if no result: show "no usable frames"
    else: draw a line from the origin at the estimated angle,
          show "Angle of Arrival: <angle> CCW (<frames> frames)"
    draw the waveform of each channel (mean removed, decimated by 20)
    status <- "Done", with the number of dropped blocks if any
    optionally save the recording as a WAV file named with the angle
    clear the serial buffer
close the listener when the window is closed
```

Appendix C

CAD Model of the Final System

This appendix shows the CAD model of the enclosure and turntable from the main viewing directions of the finished system. The final system design section of Chapter 3 refers to them.

The detailed CAD drawings, the STL files and the code are available in the project repository at `https://github.com/MFKMOR011/Acoustic-Direction-Finding-System`.

C.1 CAD Model of the Enclosure

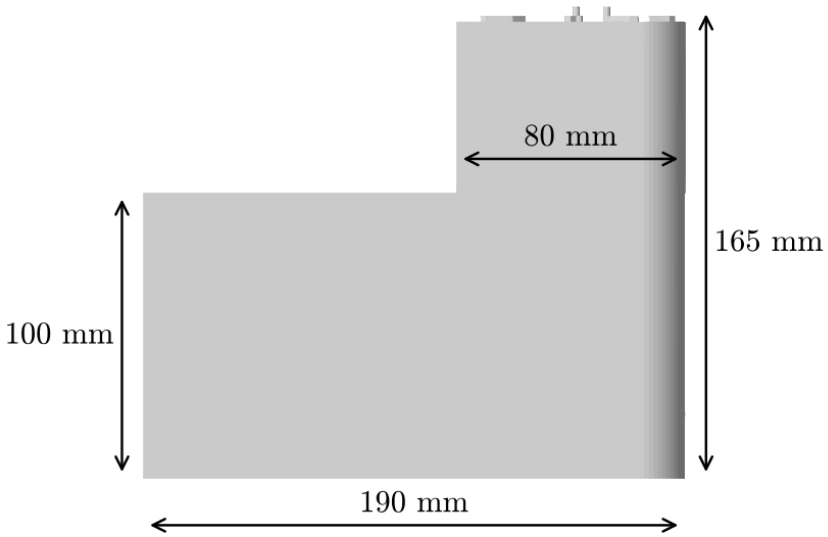


Figure C.1: Enclosure, front view with dimensions



Figure C.2: Enclosure, back view

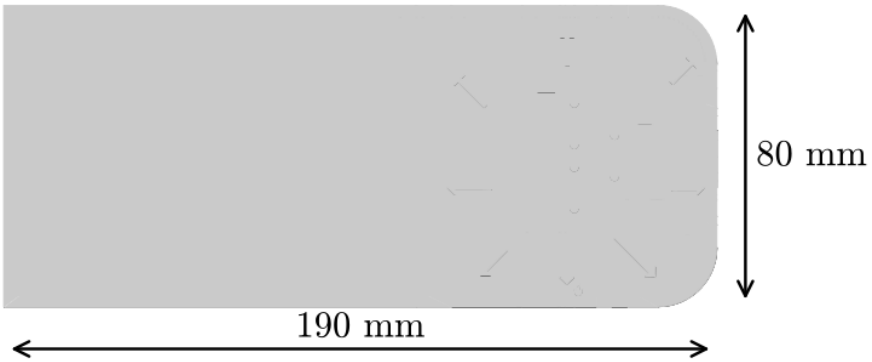


Figure C.3: Enclosure, top view with dimensions, showing the microphone cut-outs and the angle markings

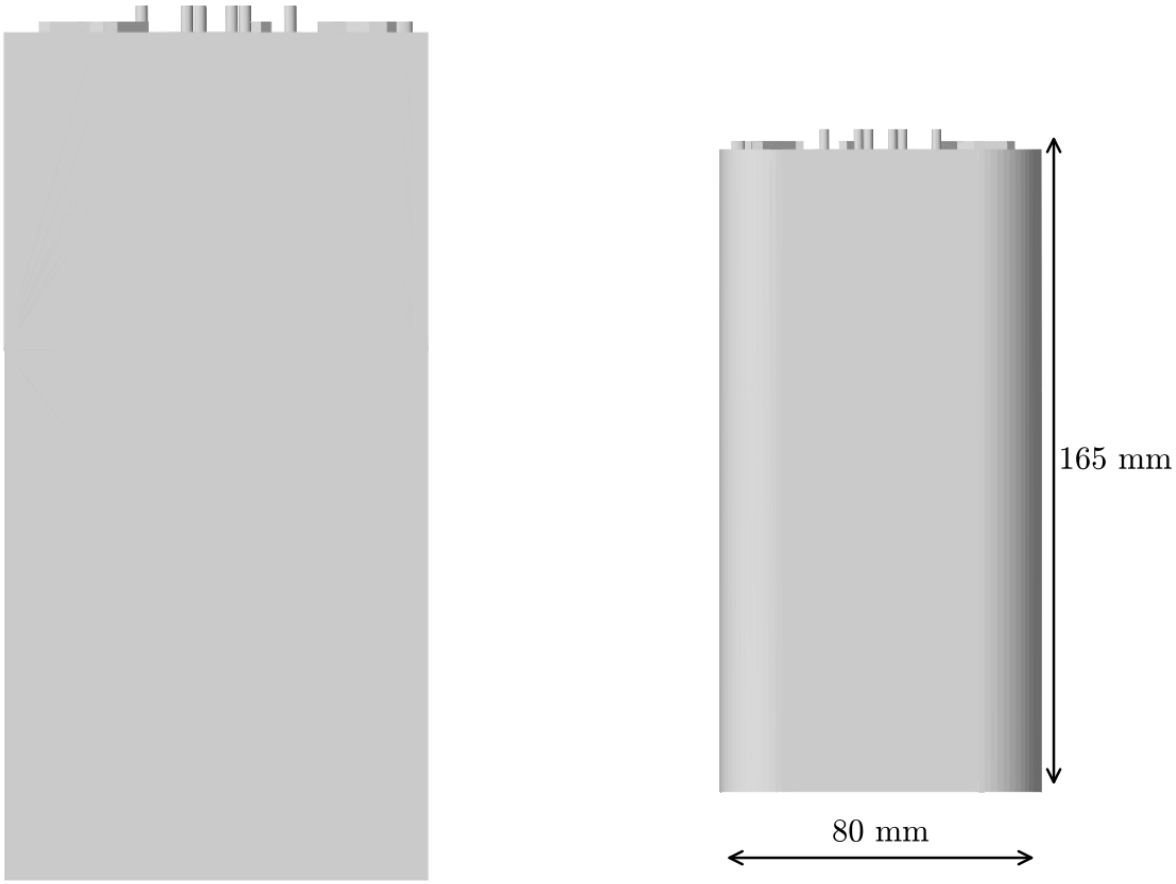


Figure C.4: Enclosure, left and right views

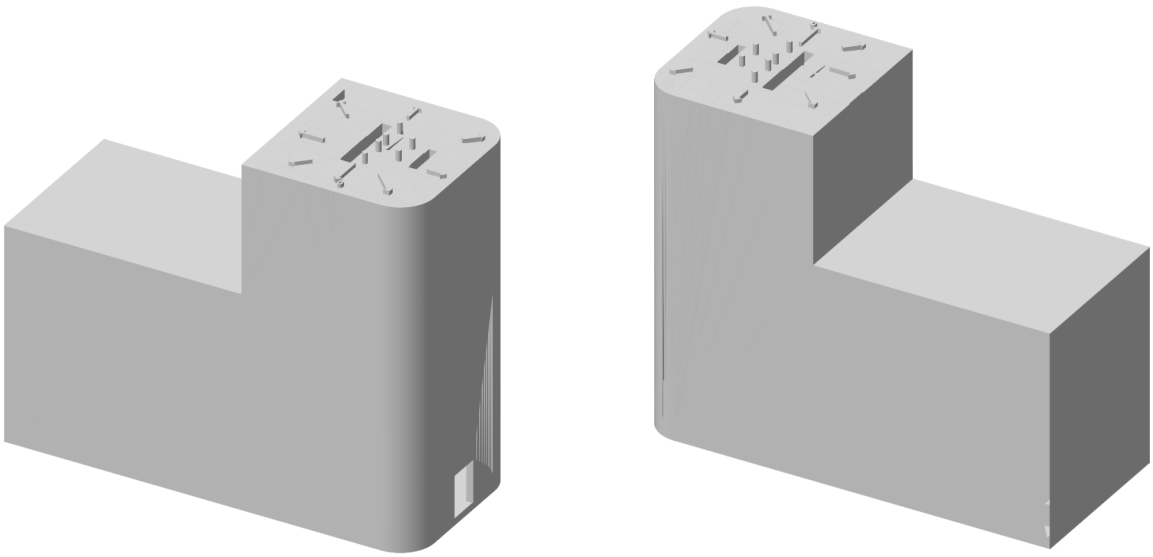


Figure C.5: Enclosure, isometric views from the front and from the back

C.2 CAD Model of the Turntable

The turntable model was found online: “Manual turning table (fully 3D printable),” MakerWorld (model 1614489), by Big Simi (2025), accessed 1 October 2026. The model page states no licence, so the model is credited to its author and used here only for academic work. It was adapted for this project by adding angle markings in steps of 22.5 degrees in the CAD software. Figure C.6 shows the turntable with its two parts separated and Figure C.7 to Figure C.9 show the detailed views.

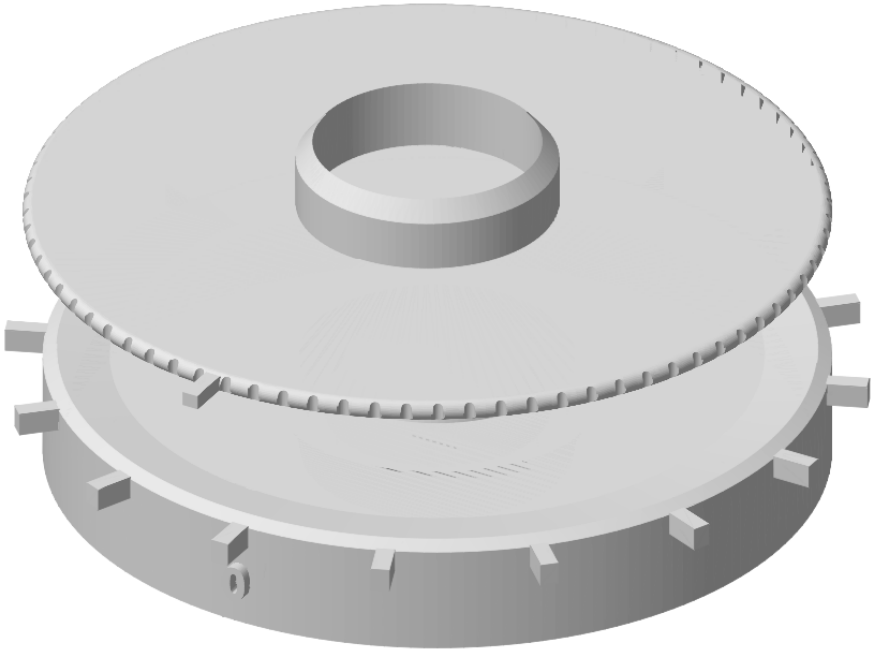


Figure C.6: Turntable, isometric view with the top plate lifted above the base

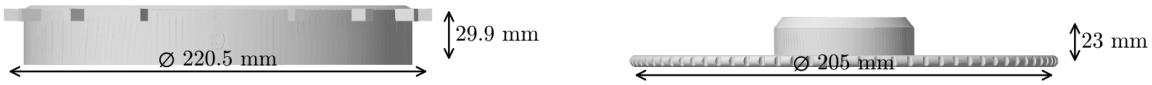


Figure C.7: Turntable, front views with dimensions: base (left) and top plate (right)

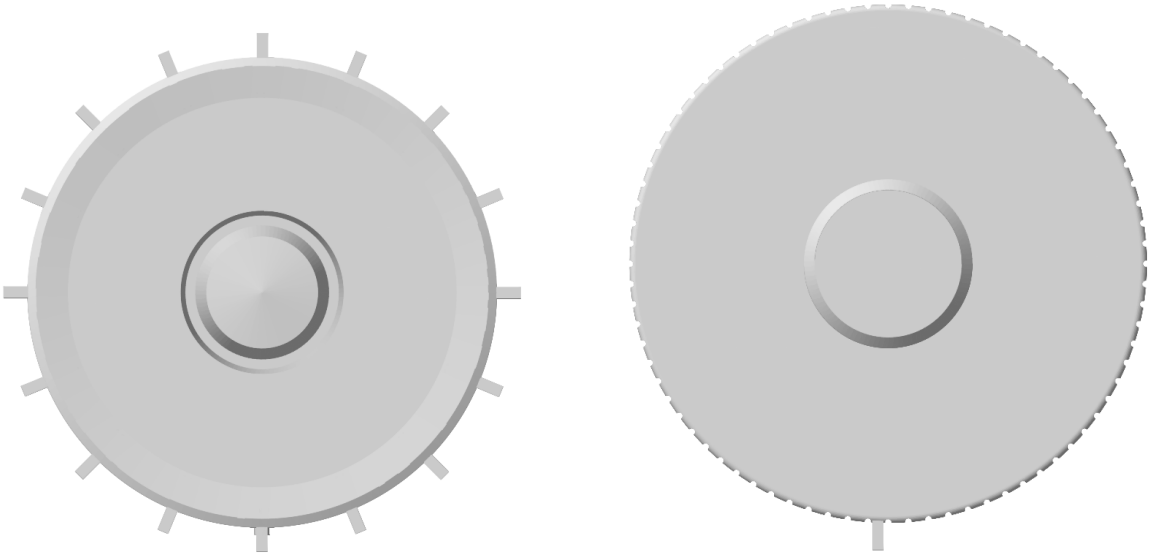


Figure C.8: Turntable, top views: base with the markings every 22.5 degrees (left) and top plate (right)

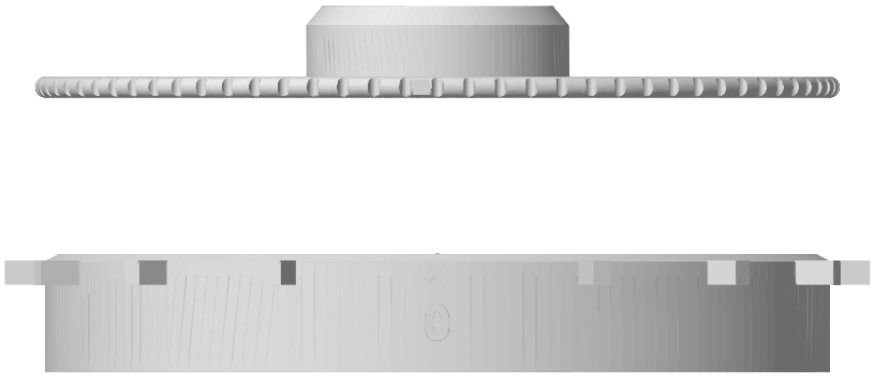


Figure C.9: Turntable, side view with the top plate lifted above the base

Appendix D

Datasheets of the Selected Components

This appendix contains the datasheets of the components chosen in Chapter 3. Table D.1 lists them and the sections of the report that rely on them.

Table D.1: Datasheets and the sections that cite them

Component	Document	Used in
SPH0645 digital MEMS microphone	Knowles SPH0645LM4H-B datasheet [62]	Microphone selection; array geometry; final design parameters
I2S microphone breakout board	Adafruit I2S MEMS microphone breakout (SPH0645LM4H) product page and pinout [66]	Microphone selection; wiring of the array
Teensy 4.0 board	PJRC Teensy 4.0 product page and pinout [65]	Acquisition hardware; sampling parameters
Teensy 4.0 processor	NXP i.MX RT1060 processor reference manual, serial audio interface (SAI) chapter [65]	Acquisition hardware

D.1 SPH0645 Microphone

The values of the SPH0645LM4H-B microphone [62] used in the design are:

- Signal-to-noise ratio: 65 dB(A)
- Sensitivity: -26 dBFS at 94 dB SPL, 1 kHz
- Supply voltage: 1.62 V to 3.6 V
- Output: I2S, 24-bit frame with 18-bit data
- Clock range: 1.024 MHz to 4.096 MHz
- Directivity: omnidirectional

The full datasheet is in the code repository, in the folder `datasheets`.

D.2 Teensy 4.0 Board and Processor

The values of the Teensy 4.0 board and its processor [65] used in the design are:

- Processor clock: 600 MHz (ARM Cortex-M7)
- RAM: 1 MB
- I2S pins used: pin 21 (bit clock), pin 20 (frame clock) and pin 8 (data in)
- USB speed: USB 2.0 high speed, 480 Mbit/s

The pinout diagrams of the front and the back of the board are shown in Figure D.1. The processor reference manual is too large to include and is in the code repository, in the folder `datasheets`.

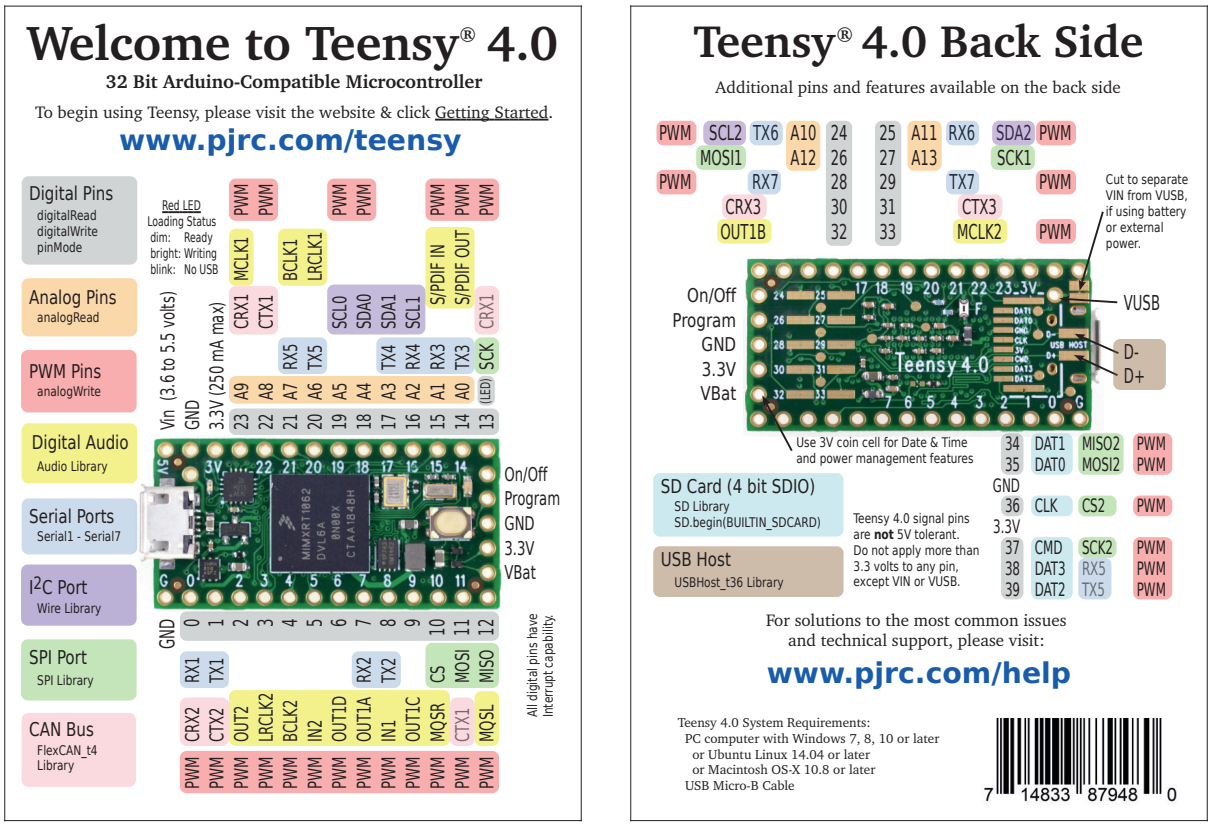


Figure D.1: Pinout of the Teensy 4.0, front (left) and back (right) [65]

D.3 Other Components

The remaining parts are standard and have no datasheet of their own. The Teensy is connected to the laptop with a standard USB cable that supports data transfer, not only charging. The microphones and the status LED were wired on a breadboard. The status LED is a green LED with a series resistor.

Appendix E

Code Repository and Use of AI

E.1 Code Repository

The code for the simulations, the real-time program and the firmware is in a public repository: `https://github.com/MF`
Table E.1 lists its contents.

Table E.1: Contents of the code repository

Folder or file	**Contents**
`firmware/`	Teensy 4.0 sketch that records the three channels and sends them over USB serial
`realtime/`	Julia programs (`realtime_trig.jl`, `realtime_music.jl`, `realtime_lut.jl`) that receive the data, estimate the angle and display it
`simulation/`	Signal simulation scripts for Trig + FFT, MUSIC and LUT + cost function
`montecarlo/`	Monte Carlo notebook (`MonteCarlo.ipynb`) and the result tables it writes to the `results` folder
`recordings/`	Recorded data, with the distance, the true angle and the signal in the file name (for example `180cm_90_d_man_sin_4000hz.wav`)
`cad/`	CAD files of the enclosure, and the changes made to the turntable

E.2 Use of AI in the Report

An AI assistant, Claude, was used while writing this report. It was used to improve and edit text that the author supplied or described, to convert the chapters to LaTeX and LyX, to edit the figures from the author's data and files, and to review tables and code. The examples below show what was submitted, what came back and what was used.

- **Front matter.** Submitted: "I now want you to edit and fix the wording of the abstract, acknowledgements and introduction", with the existing text of each. Received: edited versions of the three sections. Used: the edited abstract and introduction, checked against the results, with the grammar fixed, and the edited acknowledgements.
- **Figures.** Submitted: "I want you to edit these figures so that they fit the theme of this report". Received: nine updated versions of the diagrams of the design and the analysis. Used: the black-

and-white version, after the author rejected the first set and asked for “a different theme, just white and black, in the same font as the document”.

- **Table review.** Submitted: the tables, with the request “the table is too large, overlapping, and some of the information is missing, can you format it better”. Received: better formatted tables. Used: the corrected LaTeX syntax for the tables.

Appendix F

Ethics Clearance

The ethics clearance letter for this project (EEE4022S, student number MFKMOR011) is included on the following page.

UNIVERSITY OF CAPE TOWN
IYUNIVESITHI YASEKAPA • UNIVERSITEIT VAN KAAPSTAD

PRE-SCREENING QUESTIONNAIRE OUTCOME LETTER

STU-EBE-2026-PSQ002618

2026/08/20

Dear Morena Mofokeng,

Your Ethics pre-screening questionnaire (PSQ) has been evaluated by your departmental ethics representative. Based on the information supplied in your PSQ, it has been determined that you do not need to make a full ethics application for the research project in question.

You may proceed with your research project titled:

Acoustic Direction Finding System

Please note that should aspect(s) of your current project change, you should submit a new PSQ in order to determine whether the changed aspects increase the ethical risks of your project. It may be the case that project changes could require a full ethics application and review process.

Regards,

Faculty Research Ethics Committee

Appendix G

Graduate Attribute Appendix

This appendix describes how each of the six graduate attributes (GA1, GA4, GA5, GA6, GA8 and GA9) was met in this project. The descriptions refer to the chapters of this report where the evidence is found.

G.1 GA1: Problem Solving

- **Problem formulation.** The problem was stated in Chapter 1: estimate the azimuth of a sound source with the smallest practical array, three microphones on a 17 mm equilateral triangle, with an accuracy of 15 degrees (S07) and a time from sound to displayed angle of at most 1 minute (S08).
- **Literature.** Chapter 2 reviews the physics of sound, signal processing, sound source localisation, array geometry, direction-of-arrival algorithms, hardware, simulation and the sources of error, and ends with what the literature implies for the design.
- **First principles.** The relation between the delay and the angle was derived from the plane-wave geometry of a microphone pair and extended to the three-microphone array. The spatial aliasing limit, the angular accuracy of a pair, a line and a triangle, and the Cram
- **Mathematics and statistics.** The analysis uses the discrete Fourier transform, eigendecomposition and numerical search, and the results are described with the bias, the standard deviation, the MAE, the RMSE and the fraction of estimates within 15 degrees. Monte Carlo experiments quantify the effect of noise, position error, gain mismatch and delay mismatch.
- **Substantiated conclusions.** The simulation gives errors below 1 degree, whereas the real array gives errors of tens of degrees. The report analyses the possible causes with the Monte Carlo experiments and concludes that effects not included in the simulation, such as the microphone positions, the channel mismatch and the room, must be removed before the accuracy requirement is met.
- **Design concept.** The solution was kept small, low-cost and reusable: standard MEMS microphones, a Teensy 4.0 powered over USB, and a 3D-printed enclosure and turntable that can be reprinted, so that no special equipment or consumables are needed.

G.2 GA4: Investigations, Experiments and Data Analysis

- **Choice of method.** The minimum number of microphones that gives a full-circle azimuth without front--back ambiguity is three, and the methods in the literature were compared for this array before three were selected.
- **Experiment protocol.** Chapter 4 sets out the protocol: three source types (a 4 kHz sine, a speech phrase and a hand clap), two source distances (90 cm and 180 cm), six test angles (0, 22.5, 45, 90, 180 and 270 degrees) set on a turntable, and six repeats for each condition, with the steps of the procedure written down.
- **Simulation and real array.** The algorithms were first tested on simulated data with a known angle (36 conditions) and then on the real array (the sine and speech recordings and the clap), which separates errors of the algorithms from errors of the hardware.
- **Monte Carlo experiments.** Four experiments (A to D) vary the noise, the microphone positions, the gain and delay mismatch, and the frame and observation length, and an error budget compares the size of each effect.
- **Results and comparison.** Chapter 5 gives the results in tables and figures against defined metrics, compares simulation with measurement and compares the three algorithms on accuracy, robustness and computation time. The acceptance test procedures are reported with a pass or fail for each.
- **Conclusions.** The conclusions and recommendations follow from the results, and the limitations of the experiments are stated.

G.3 GA5: Use of Engineering Tools

- **Tools used.** Julia (with the FFTW, linear algebra, plotting and data-frame libraries) for the algorithms, the simulation, the Monte Carlo experiments and the real-time display; the Arduino environment with a Teensy 4.0 for the firmware; CAD and 3D printing for the enclosure and the turntable; and LyX and LaTeX for the report. The code is kept in a public repository.
- **Decisions from datasheets.** The microphone spacing, the sampling and the I2S settings follow from the datasheets of the SPH0645 microphone and the Teensy 4.0 processor, for example the clock range of the microphone and the half-wavelength limit of the spacing, which sets the highest usable frequency (Chapters 2 and 3 and the datasheet appendix).
- **Limitations.** The limitations of each tool and platform are stated in the report, for example the small 17 mm aperture, the fixed sampling rate of the audio library and the speed of the serial link.
- **Benchmark.** The computation time per estimate was measured for each algorithm in Julia on the laptop: 0.9 s for Trig + FFT, 1.5 s for MUSIC and 2.2 s for LUT + cost function.
- **Software and hardware.** The algorithms run in Julia on the laptop, and the Teensy only acquires and sends the data. Running them on the microcontroller was outside the scope of the project and is listed in Chapter 8.

- **AI tools.** An AI assistant was used and is disclosed in the declaration and in the appendix on the use of AI. Its output was checked before use, and its errors were found and corrected, for example a wrong conclusion about the latency requirement and a figure whose convention differed from the equations.

G.4 GA6: Professional and Technical Communication

- **Report structure.** The report follows the structure of the course template, with an abstract, a nomenclature and a list of abbreviations, eight chapters and appendices, and consistent notation and cross-references.
- **Diagrams and drawings.** The report contains block diagrams of the system, the firmware and the signal pipeline, diagrams of the geometry, CAD views of the enclosure and turntable, photographs of the finished system and screenshots of the real-time display.
- **Repeatability.** The procedures for the experiments, the set-up of the turntable and the settings of the firmware and software are written so that another person can repeat them. The code, firmware, recordings and CAD files are in the code repository (see the code repository in the report).
- **Presentation of results.** Results are presented in tables and figures with the same metrics throughout, and the text states what each one shows.
- **Supervisor meetings.** Weekly meetings with supervisor.

G.5 GA8: Individual Working

- **Responsibility.** The author carried out the design, the building, the programming, the debugging, the experiments, the analysis and the writing of this report.
- **Independent decisions.** Technical decisions, such as the choice of the array, the algorithms, the sampling settings, the enclosure and the test conditions, were made by the author and are justified in Chapters 2 to 4.
- **Use of the supervisor.** Problems were first analysed and tried independently, and the supervisor was consulted when more expertise was needed, with a defined problem, what had been tried and a proposed next step.
- **Equipment.** The equipment used in the labs was handled responsibly and returned in working condition.
- **Integrity and ethics.** Ethics clearance was obtained (see the ethics clearance appendix). All sources are cited in the IEEE style, the turntable model is credited to its author, and the use of an AI assistant is disclosed in the plagiarism declaration and in the appendix on the use of AI.

G.6 GA9: Independent Learning Ability

- **Gaps in knowledge.** The author identified gaps in array signal processing, embedded I2S audio capture on the Teensy, Monte Carlo analysis and Julia programming, and closed them through self-directed study of papers, datasheets and reference manuals, which are cited in Chapter 2 and in the list of references.
- **Reflection.** After each milestone the approach was adjusted. For example, the large errors of the real array led to the Monte Carlo experiments that were designed to find which error source could explain them, and the simulation was used to check the algorithms before the hardware was blamed.
- **Feedback.** Feedback from the supervisor was acted on, and the changes made in response are recorded in the log and in the report.